

机器视觉技术

人工智能学院
南开大学



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



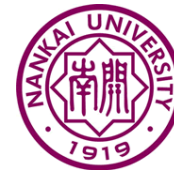
南開大學

Nankai University

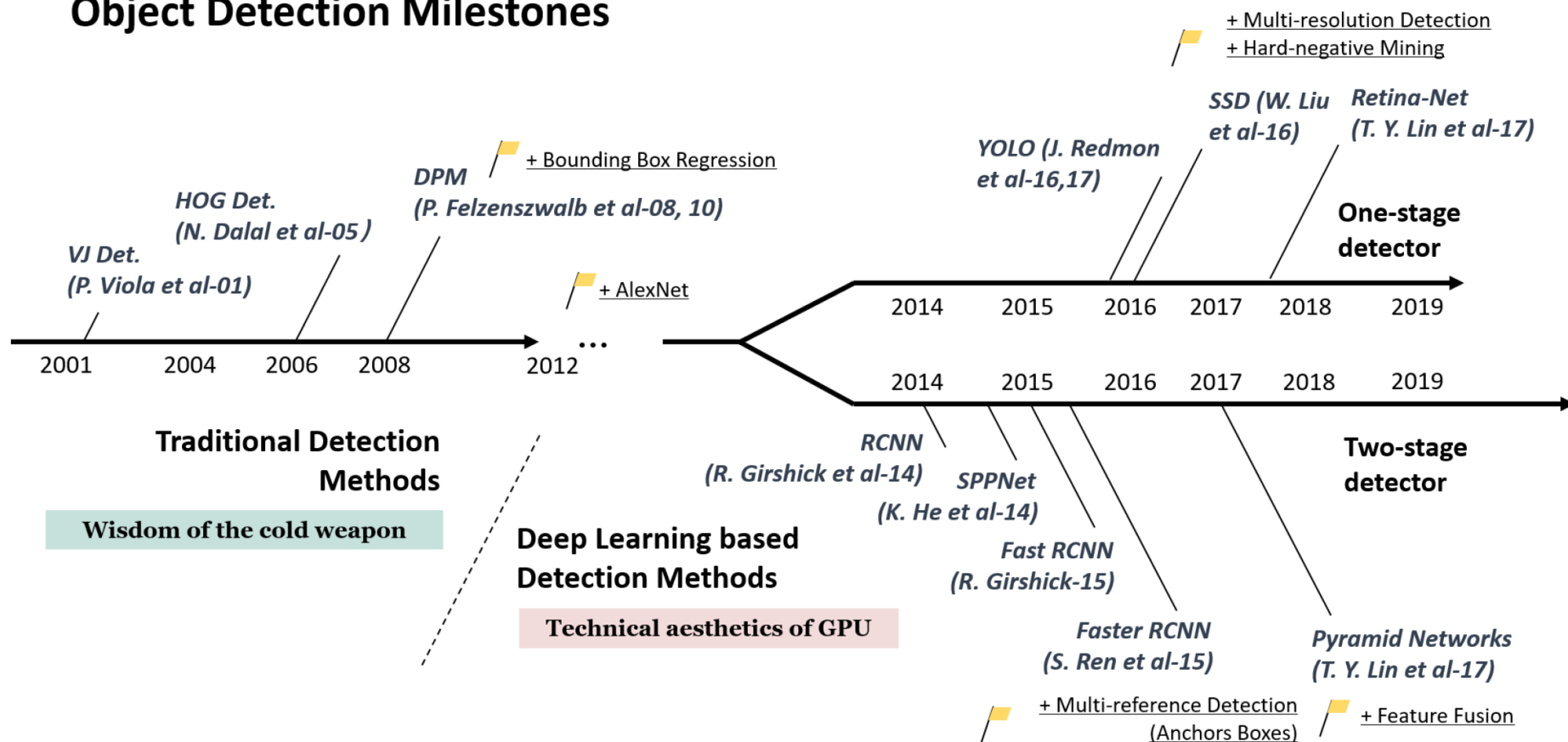
主要内容

- 第六章 卷积神经网络
- 第七章 图像分类
- 第八章 目标检测与图像分割
 - 8.1 目标检测
 - 8.1.1 定位+分类
 - 8.1.2 Two-stage目标检测方法
 - 8.1.3 One-stage目标检测方法
 - 8.2 图像分割
 - 8.2.1 语义分割
 - 8.2.2 实例分割

8.1 目标检测



Object Detection Milestones



Zou Z , Shi Z , Guo Y , et al. Object Detection in 20 Years: A Survey[J]. 2019.

8.1.1 定位+分类

- 目标检测：对于有多个目标的图像，检测出所有目标所处的位置及其类别
- 定位+分类：对于仅有一个目标的图像，检测出该目标所处的位置以及该目标的类别，定位+分类是分类到目标检测的一个过渡问题
 - 分类：给出类别标签——**分类**
 - 定位：给出目标所在的外界矩形框/包围框(bounding box) 四元组 (x, y, w, h) ——**回归**



→ CAT

a) 分类问题

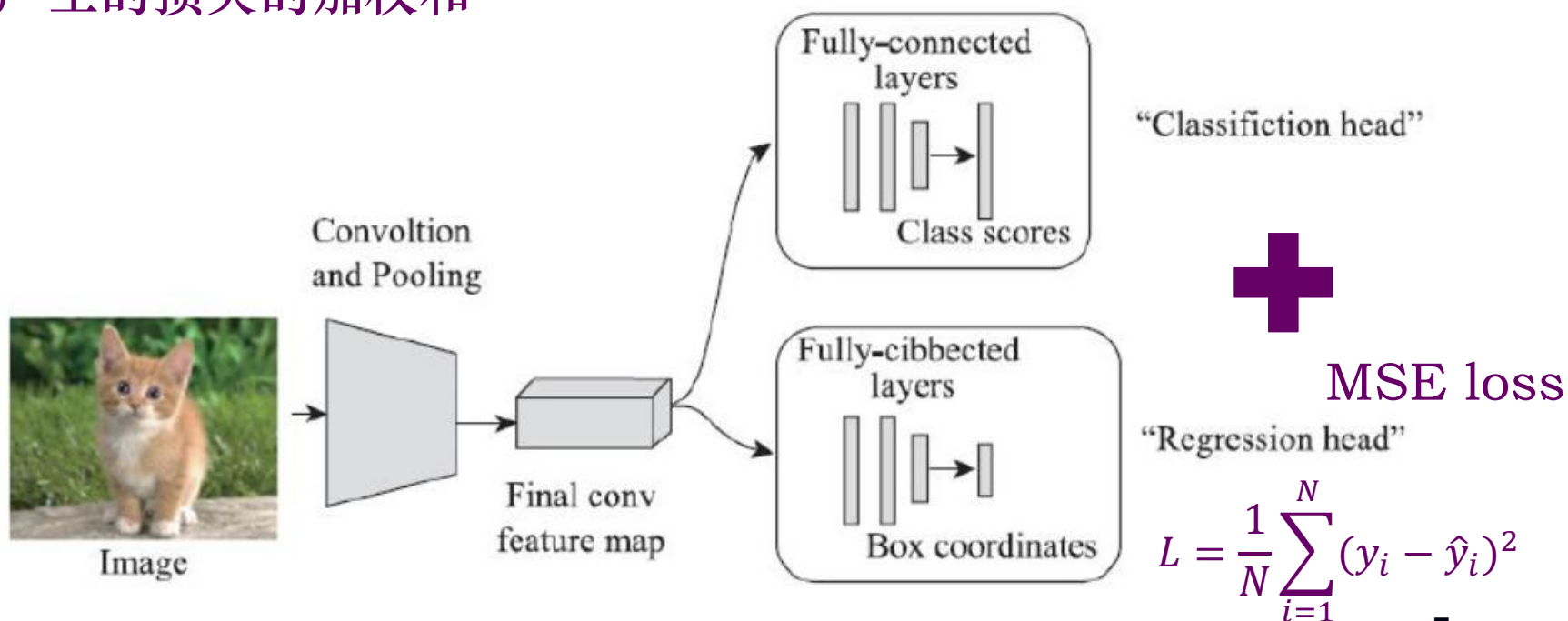


→ (x, y, w, h)

b) 定位问题

8.1.1 定位+分类

- 在分类网络最后一个卷积层的特征层(feature map)上添加“regression head”（回归头）
- 训练阶段：同时训练“分类头”和“回归头”，为了同时训练分类和定位两个问题，最终损失函数是分类和定位两个“head”产生的损失的加权和



8.1.1 定位+分类

- 在分类网络最后一个卷积层的特征层(feature map)上添加“regression head” (回归头)
- 训练阶段：同时训练“分类头”和“回归头”，为了同时训练分类和定位两个问题，最终损失函数是分类和定位两个“head”产生的损失的加权和
- 预测阶段：同时使用分类和回归“head”得到分类+定位的结果
 - 分类预测出的结果就是C个类别
 - 回归预测的结果可能有两种：一种是类别无关，输出1个包围框四元组(x, y, w, h)，共4个值；一种是类别相关，输出 $4 * C$ 个值

从分类到目标检测

- 定位+分类的问题：图像中有多个目标？
- 从滑窗到候选框
 - 设计大量的不同尺度和长宽比的“滑窗”，通过CNN来完成预测
 - 快速定位到目标的潜在区域/候选区域(proposals)，减少计算量



Dog? NO
Cat? NO
Background? YES



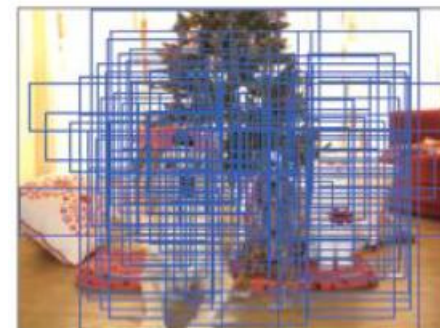
Dog? YES
Cat? NO
Background? NO



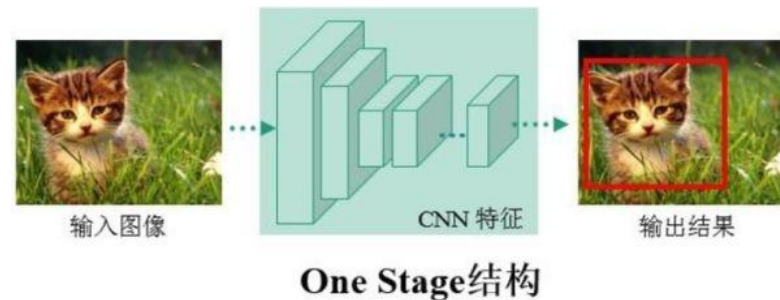
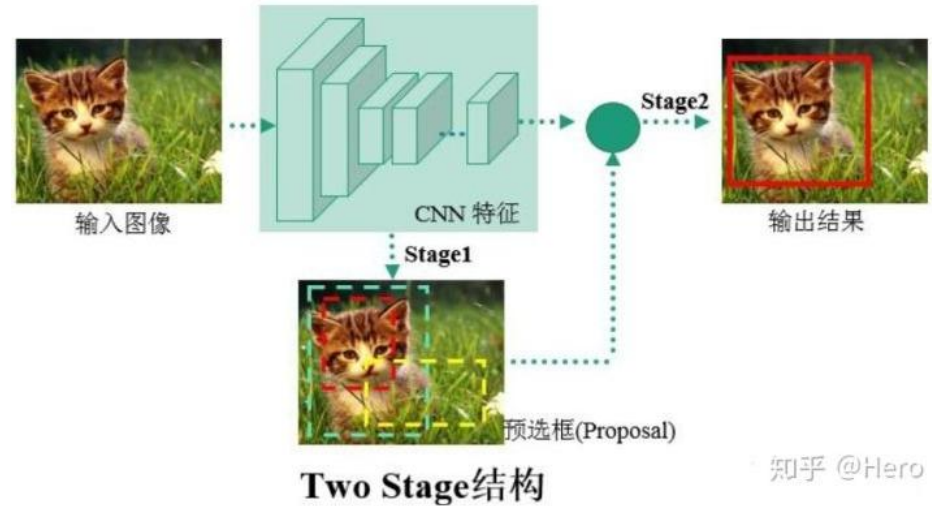
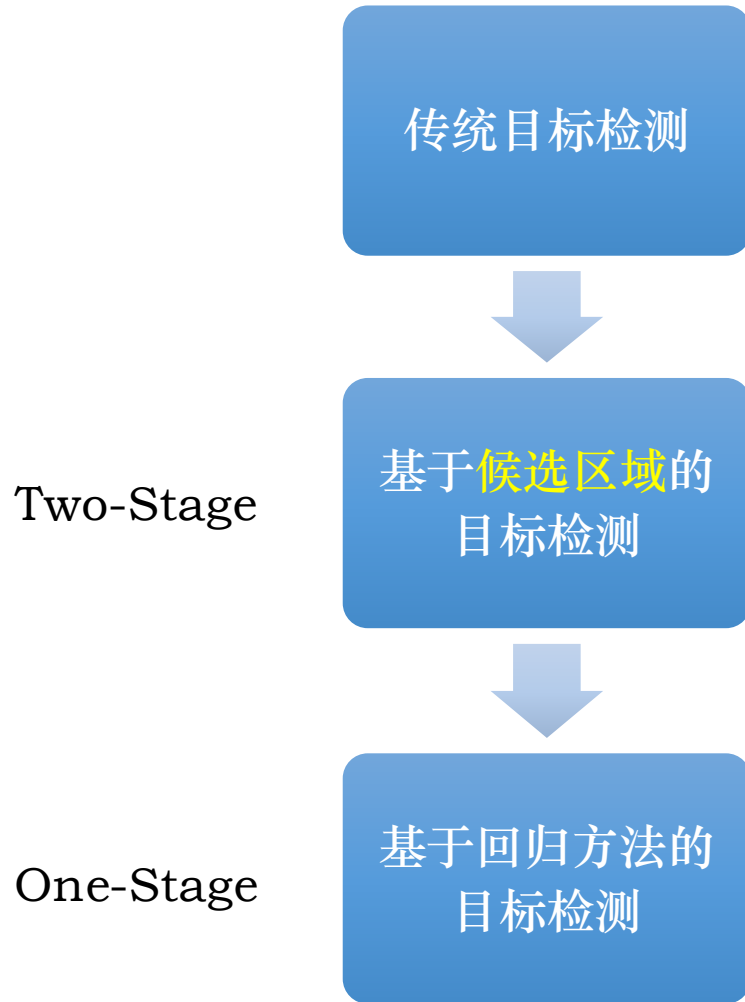
Dog? YES
Cat? NO
Background? NO



Dog? NO
Cat? YES
Background? NO



8.1 目标检测



PASCAL VOC 数据集



➤ PASCAL VOC 数据集

Visual Object Classes Challenge



- 网址: <http://host.robots.ox.ac.uk/pascal/VOC/>
- 是计算机视觉领域最经典、影响力最深远的公开数据集之一
- 由欧盟资助的 PASCAL 网络组织发起, 从 2005 年到 2012 年每年都会举办挑战赛, 旨在为图像分类、目标检测、语义分割等任务提供一个统一的评估基准
- PASCAL VOC 数据集共包含 20 个目标类别, 涵盖了人物、动物、交通工具和室内用品四大类
- VOC2007: 包含约9,963张标注图像, 共标注了24,640个物体
- VOC2012: 包含约11,530张标注图像, 共标注了27,450个物体

PASCAL VOC 数据集



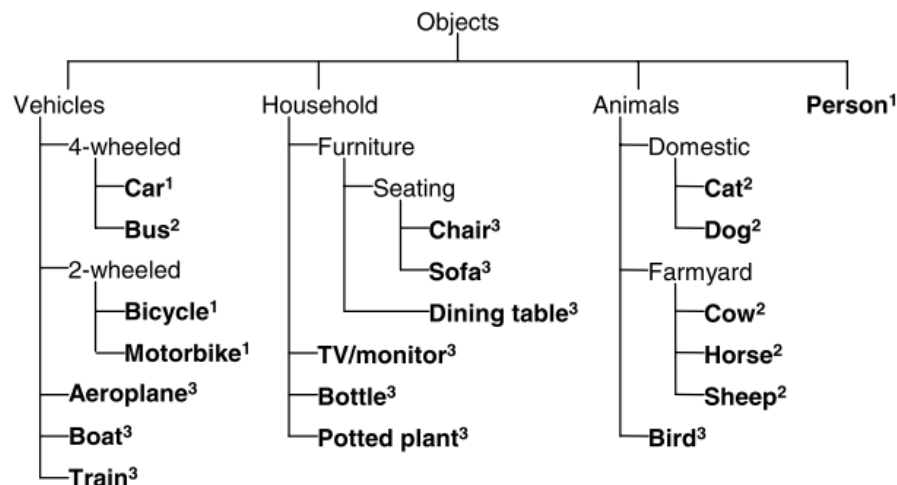
➤ PASCAL VOC 数据集

Visual Object Classes Challenge

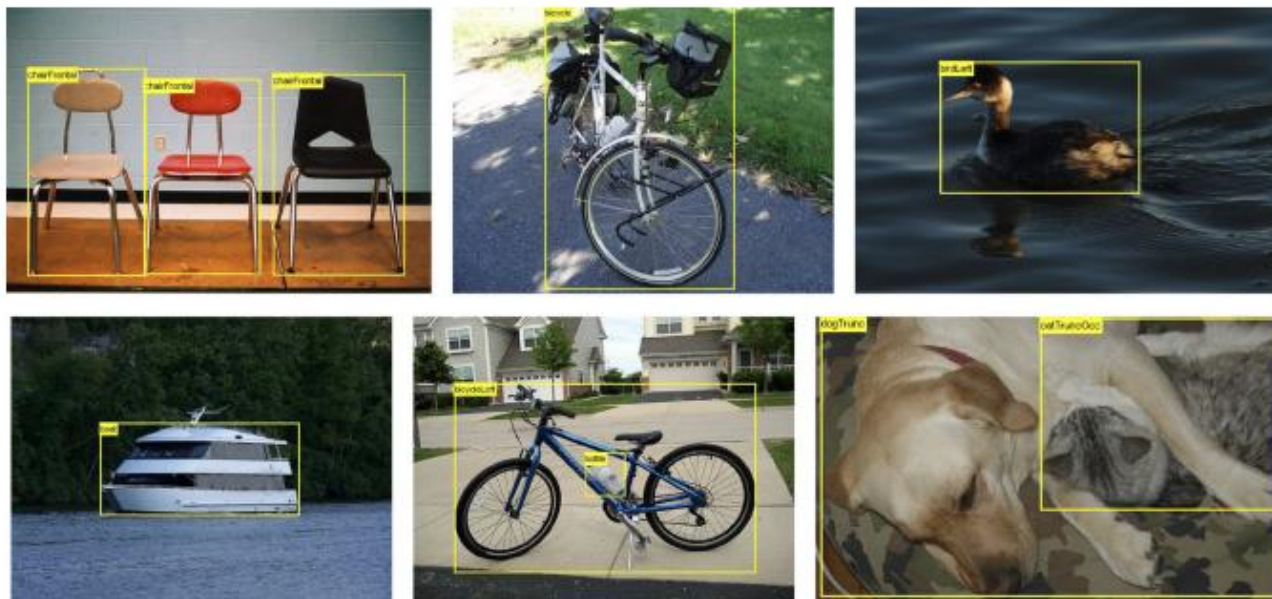
➤ 网址: <http://host.robots.ox.ac.uk/pascal/VOC/>

➤ 任务

- Object Classification
- Object Detection
- Object Segmentation
- Human Layout
- Action Classification



PASCAL VOC 数据集

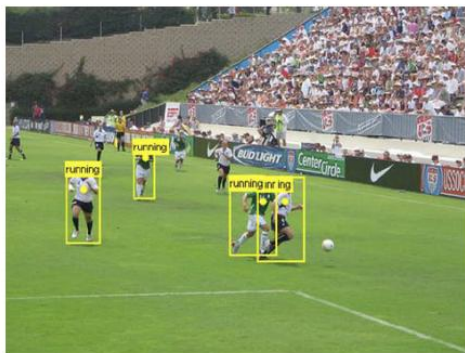


(a) Classification and detection

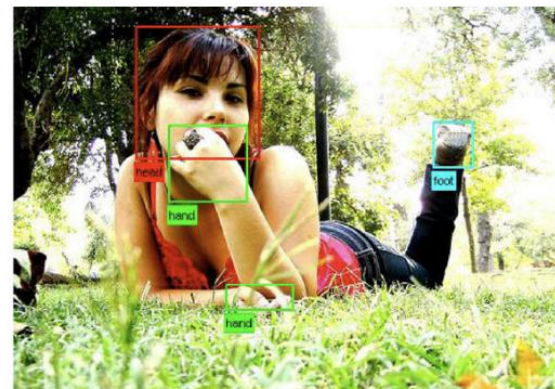
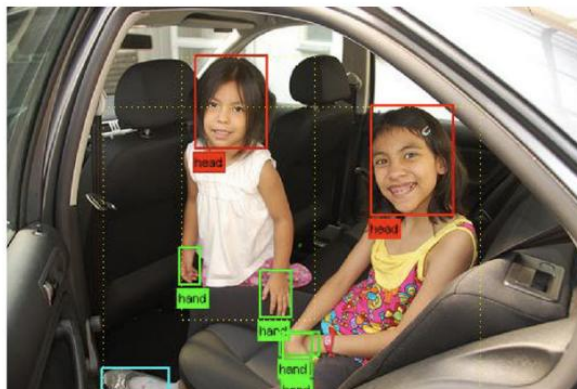
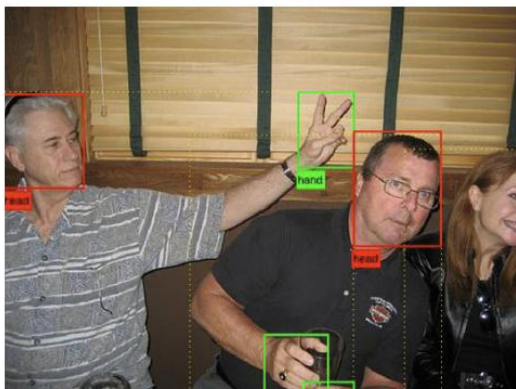


(b) Segmentation

PASCAL VOC 数据集



(c) Action classification



(d) Person layout

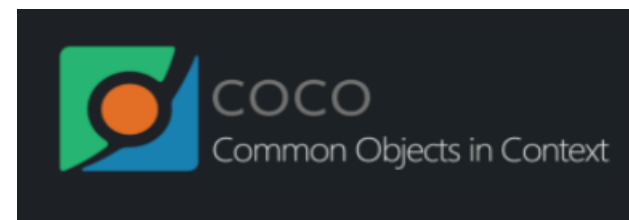
COCO数据集



➤ COCO(Common Objects in Context)

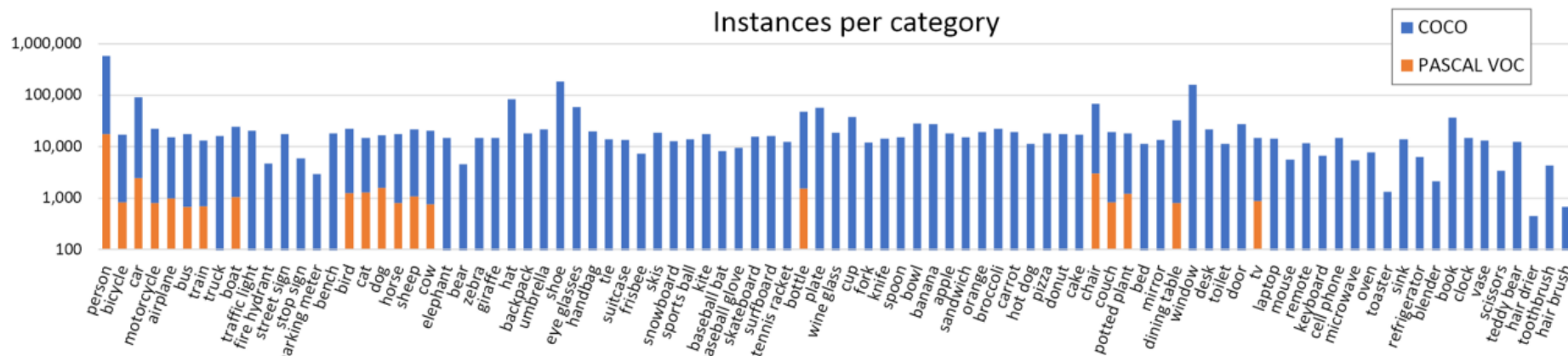
➤ 网址: <https://cocodataset.org/>

<https://arxiv.org/pdf/1405.0312.pdf>



➤ 由微软研究院在2014年发布, 旨在推动目标检测、图像分割以及场景理解等技术的发展

➤ 包含80个常见的物体类别, 涵盖人、动物、交通工具、日常用品等, COCO 2017共包含约 12.3 万张图像, 标注超过 150 万个物体实例



COCO数据集



➤ 任务



Object Detection Task



Stuff Segmentation Task



Panoptic Segmentation Task



The man at bat readies to swing at the pitch while the umpire looks on.

Captioning Challenge



Keypoint Detection Task

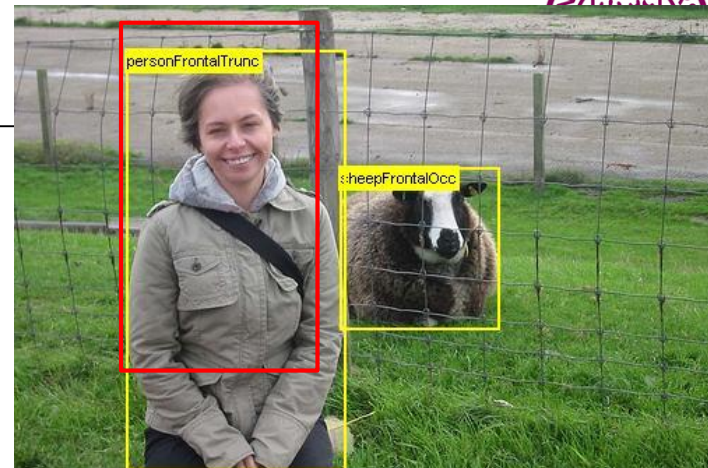


DensePose Task

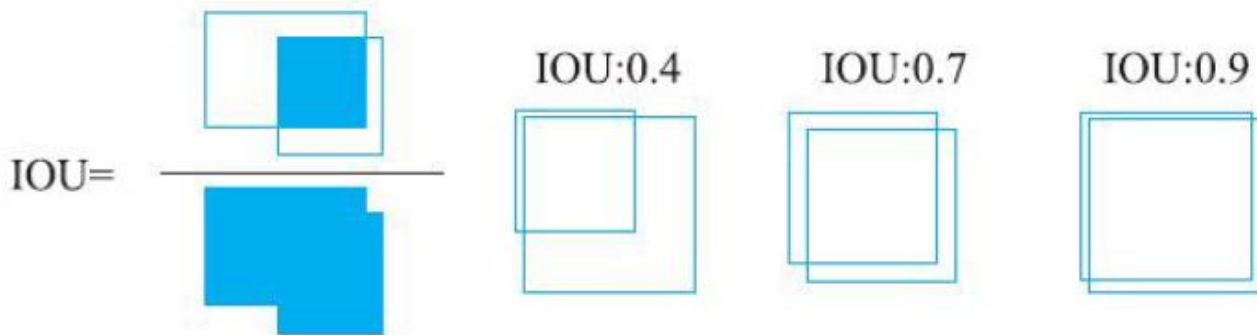
评价指标简介

➤ 评价指标

- 精确率 $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- 召回率 $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- PR曲线: Precision-Recall曲线
- AP: PR 曲线下的面积(AUC, Area Under Curve)
- mAP(mean Average Precision)



➤ 交并比 IoU



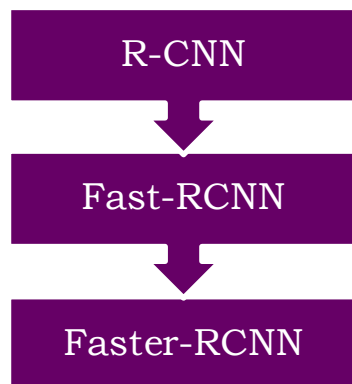
评价指标简介

- COCO数据集中的AP/AP50/AP75
 - COCO数据集中的AP指mAP
 - AP后面的数字代表IOU阈值:
 - AP50代表在IoU阈值大于0.5的情况下求AP (Pascal VOC指标)
 - IoU=.50:.05:.95 指的是IoU从0.5开始间隔0.05一直取值到0.95，然后求均值
 - AP50>AP75

Average Precision (AP):

AP	% AP at IoU=.50:.05:.95 (primary challenge metric)
AP ^{IoU=.50}	% AP at IoU=.50 (PASCAL VOC metric)
AP ^{IoU=.75}	% AP at IoU=.75 (strict metric)

8.1.2 Two-stage目标检测方法



Rich feature hierarchies for accurate object detection and semantic segmentation

Ross Girshick Jeff Donahue Trevor Darrell Jitendra Malik
UC Berkeley

Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun

Fast R-CNN

Ross Girshick
Microsoft Research

Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun

<https://arxiv.org/abs/1311.2524>

<https://arxiv.org/abs/1406.4729v4>

<https://arxiv.org/abs/1504.08083>

<https://arxiv.org/abs/1506.01497>

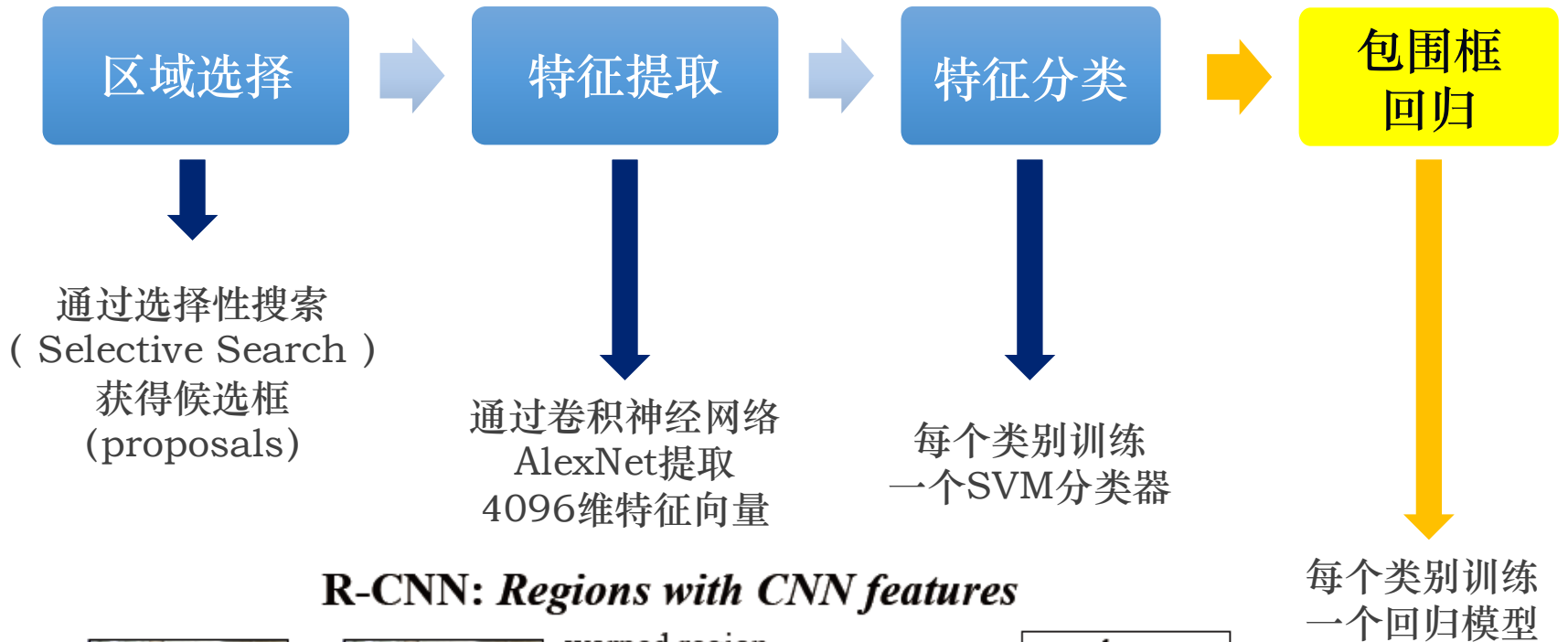
1) R-CNN

➤ Selective search方法

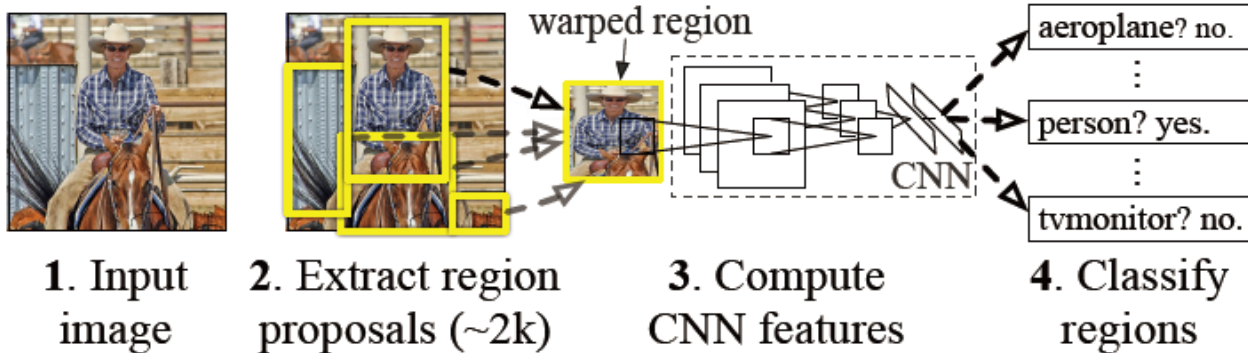
- 是一种从图像中选出潜在物体候选框的方法
- 和滑动窗口相比，选择性搜索有效地去除冗余候选区域，使得计算量大大的减小



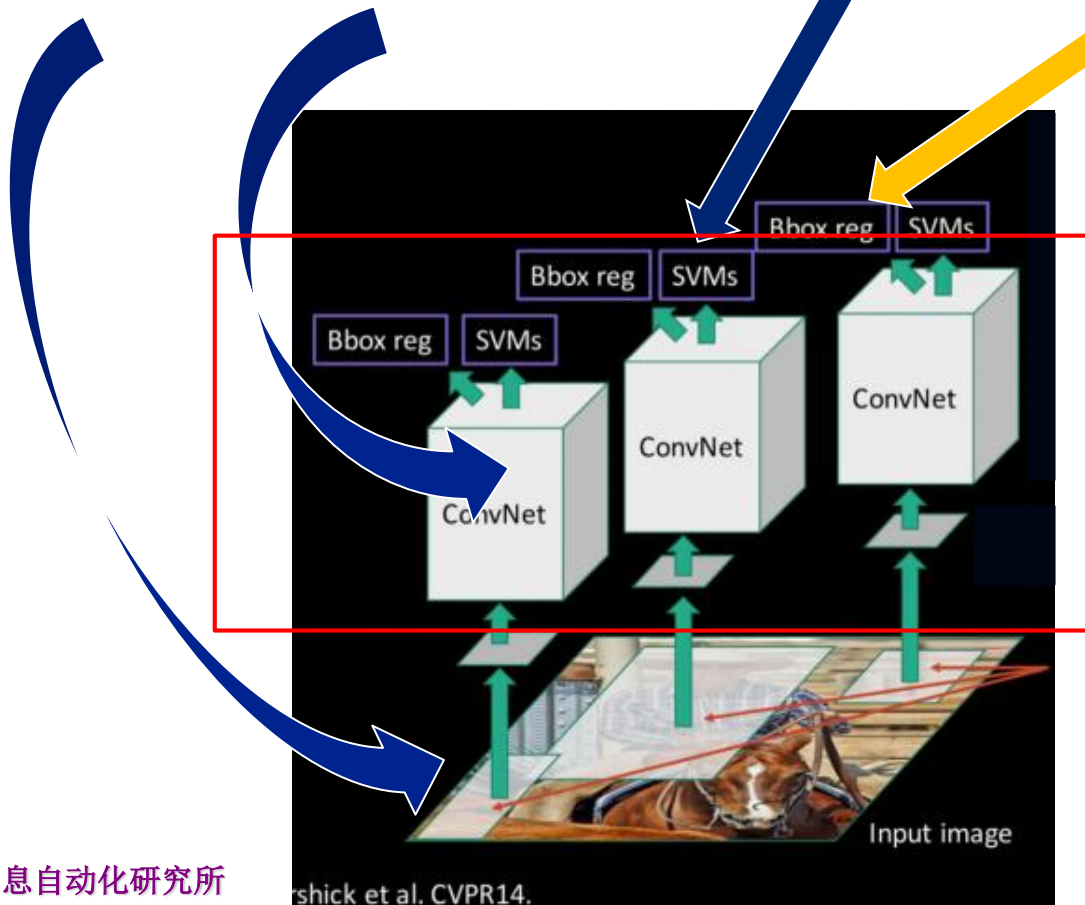
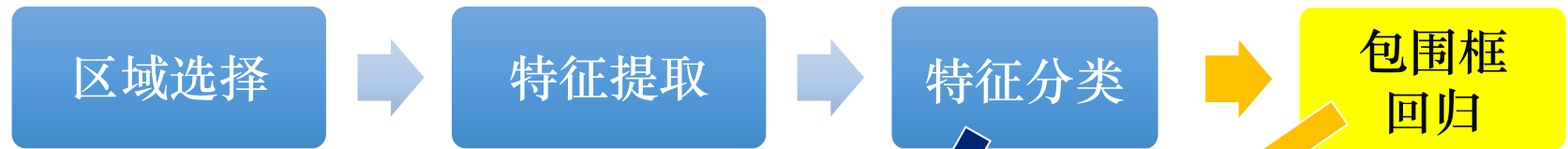
1) R-CNN



R-CNN: *Regions with CNN features*



1) R-CNN



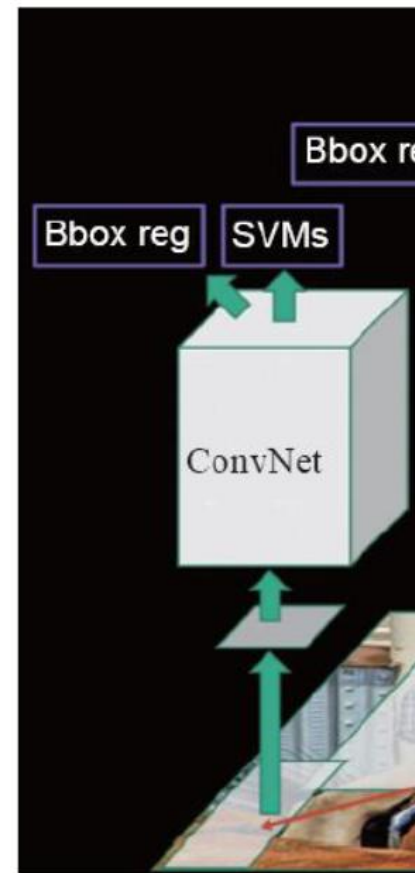
1) R-CNN

➤ 算法效果

- R-CNN在ILSVRC2013数据集上的mAP由24.3%提升到了31.4%，**第一次有了质的改变**

➤ R-CNN存在问题

- 重复计算：不论是训练还是预测，都需要对selective search出来的2000个ROI全部通过CNN的Forward过程来获取特征，这个过程花费的时间会非常长
- SVM模型：卷积神经网络的特征提取器和用来预测分类的SVM是分开的，也就是特征提取的过程不会因SVM和回归的调整而更新
- 操作流程复杂：每一步是分开的，如特征提取器通过Softmax分类获得，最终分类结果由SVM获得，矩形框的位置则是通过回归方式获得



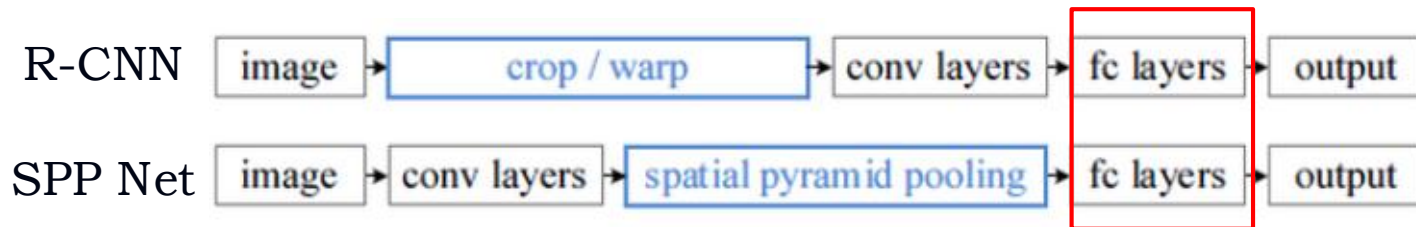
2) Fast R-CNN

➤ Fast R-CNN

- 首先是速度，2000个ROI的CNN特征提取占用了大量的时间，是否可以使用更好的方法（比如，共享卷积层）来同时处理这2000个ROI？
- 其次是CNN的特征不会因SVM和回归的调整而更新，且R-CNN的操作流程比较复杂，是否有更好的方式使得训练过程成为端到端的？

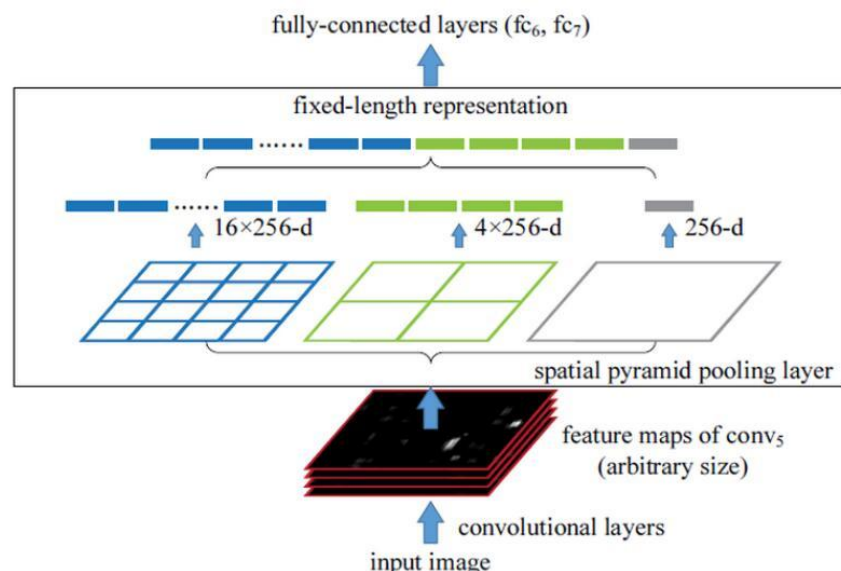
SPP Net

- 面临的问题：CNN要求输入图像必须被强制缩放到固定尺寸，主要原因为网络末端的全连接层需要固定长度的输入向量。
- SPP (Spatial Pyramid Pooling)
 - 解决方案：先做卷积，在最后的卷积层和全连接层之间，插入一个 SPP 层
 - 作用：无论前面的卷积层输出多大尺寸的特征图（Feature Map），SPP 层都能将其转化为**固定长度的特征向量**，从而完美适配全连接层



➤ SPP (Spatial Pyramid Pooling)

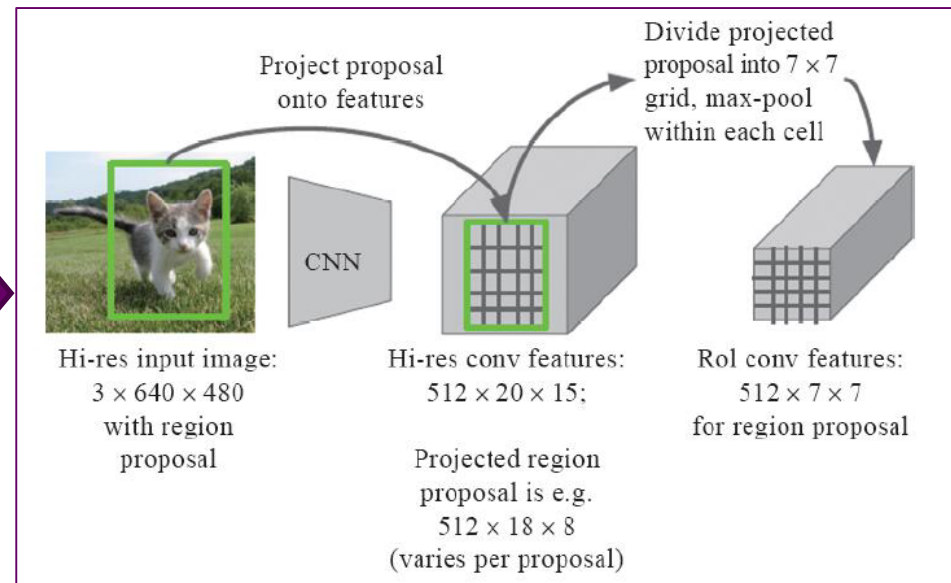
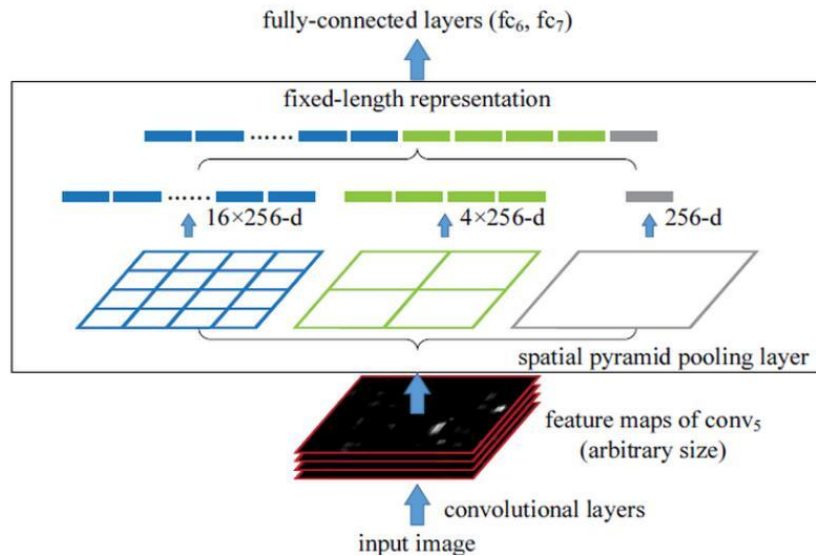
- 预设金字塔池化层级为 $[4 \times 4, 2 \times 2, 1 \times 1]$ ，设卷积层输出的特征图通道数为 256
 - 4×4 层级：将特征图均匀划分为 16 个网格，对每个网格做最大池化，得到 16 个特征值。
 - 2×2 层级：将特征图划分为 4 个网格，同样做最大池化，得到 4 个特征值
 - 1×1 层级：直接对整个特征图做全局最大池化，得到 1 个特征值
- 拼接输出：将这三部分的结果拼接起来，最终得到一个固定长度为 $(16 + 4 + 1) \times 256 = 21 \times 256$ 的特征向量
- 因为池化操作可以根据输入特征图的大小自适应地调整窗口和步长，无论输入尺寸如何，经过 SPP 层后，输出的特征向量长度是固定的



2) Fast R-CNN

➤ ROI 池化

- 坐标映射：将候选框的坐标等比例映射缩小到特征图的尺度上
- 划分网格：将映射到特征图上候选框区域在水平和竖直方向上均匀地划分为 7×7 个网格，如果候选框的长宽不能被7整除，实际计算时通常会进行取整处理
- 最大池化：从每个网格中取出数值最大的特征值，形成固定大小 7×7 特征图

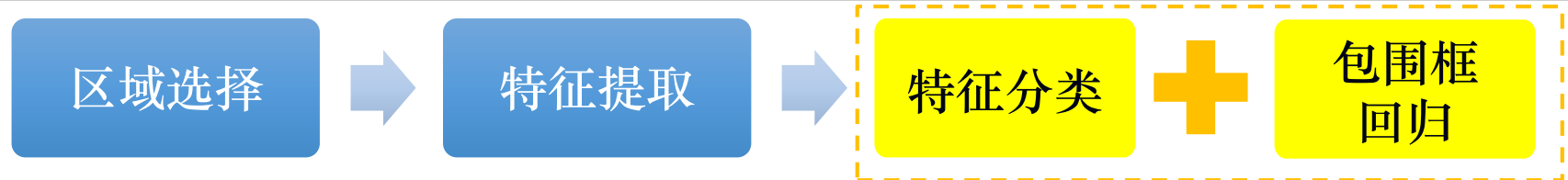


2) Fast R-CNN

➤ Fast R-CNN

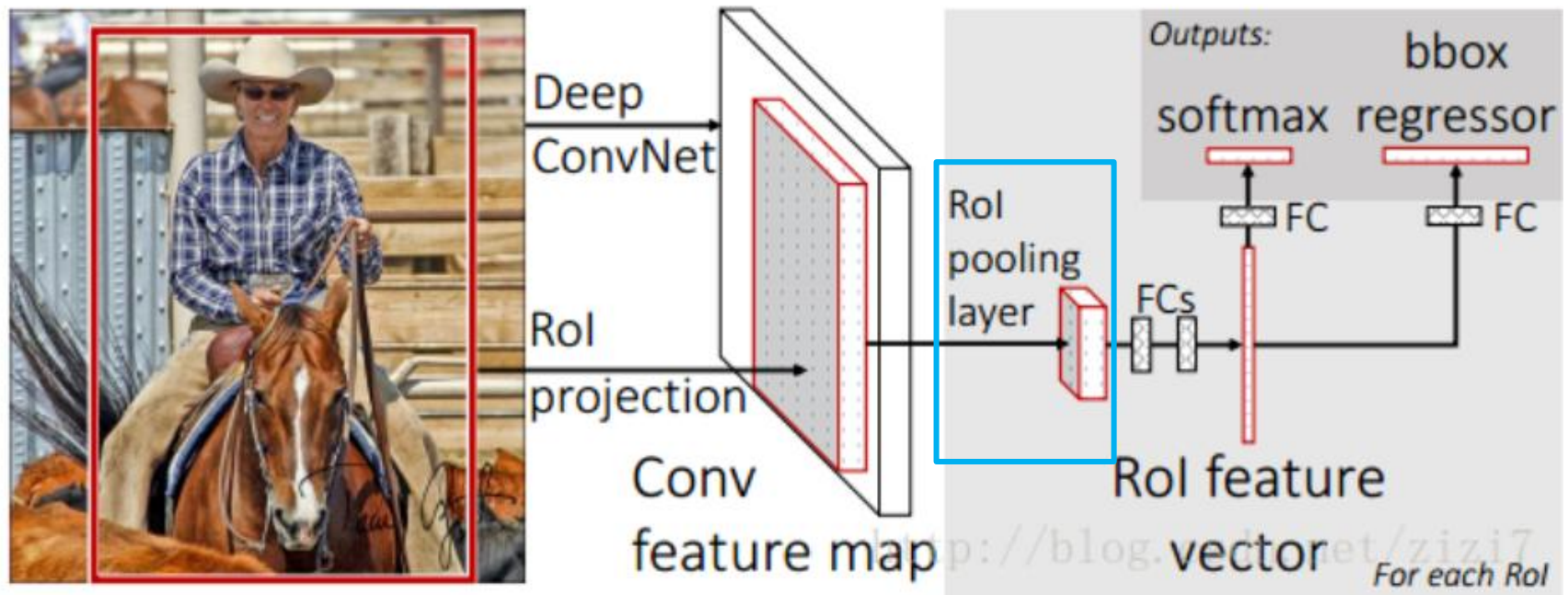
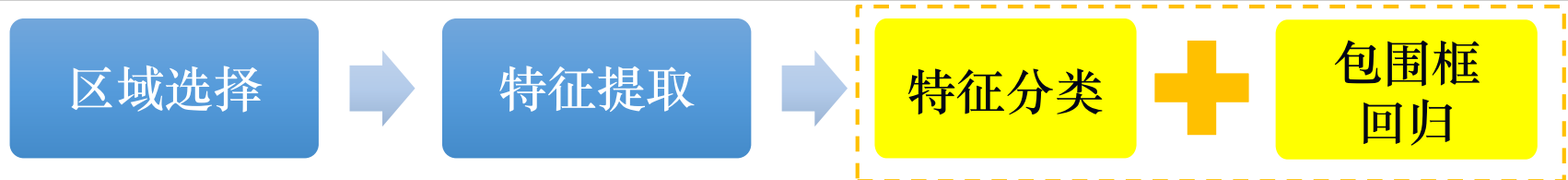
- 首先是速度，2000个ROI的CNN特征提取占用了大量的时间，是否可以使用更好的方法（比如，共享卷积层）来同时处理这2000个ROI？ --解决！
- 其次是CNN的特征不会因SVM和回归的调整而更新，且R-CNN的操作流程比较复杂，是否有更好的方式使得训练过程成为端到端的？

2) Fast R-CNN

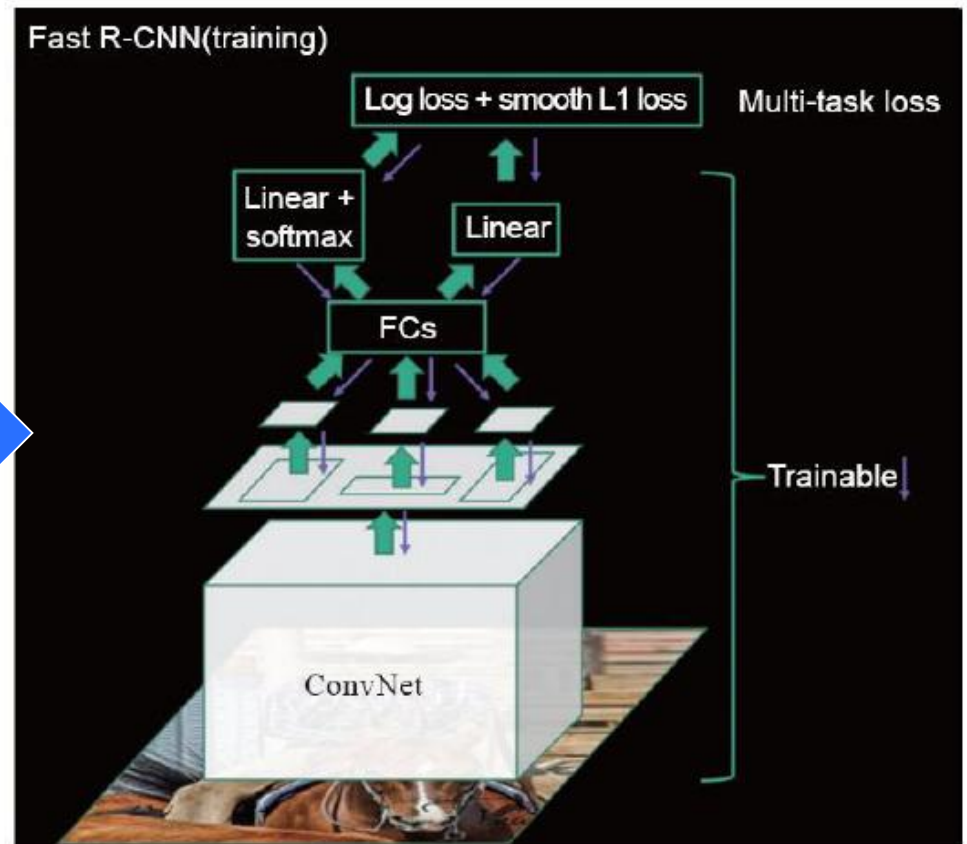
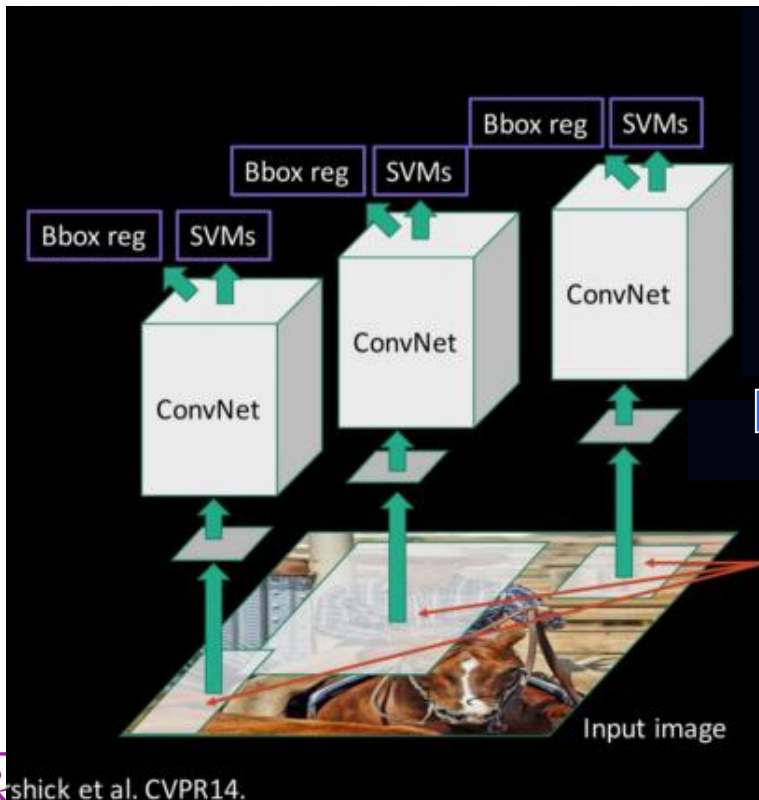
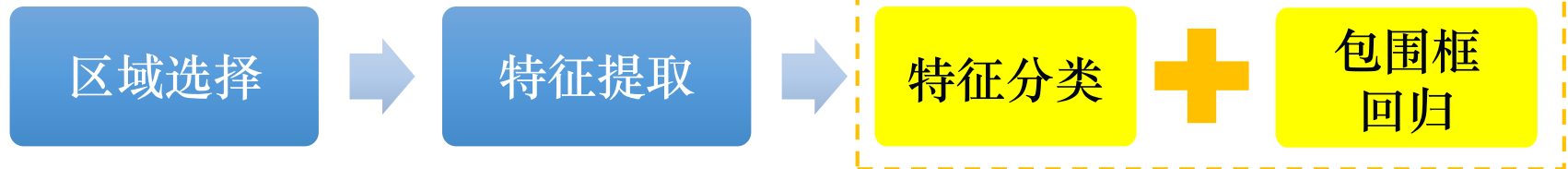


- 首先通过selective search方法生成约2K个候选框，连同图像一起输入到CNN网络
- 在最后一个卷积层后求取候选框位置的映射关系，使用ROI池化层将每个候选框的图像特征统一到相同大小
- 最后通过两个全连接(FC) 层，一个FC层后接softmax实现分类，一个FC层后接包围框回归得到类别修正后的位置

2) Fast R-CNN



2) Fast R-CNN

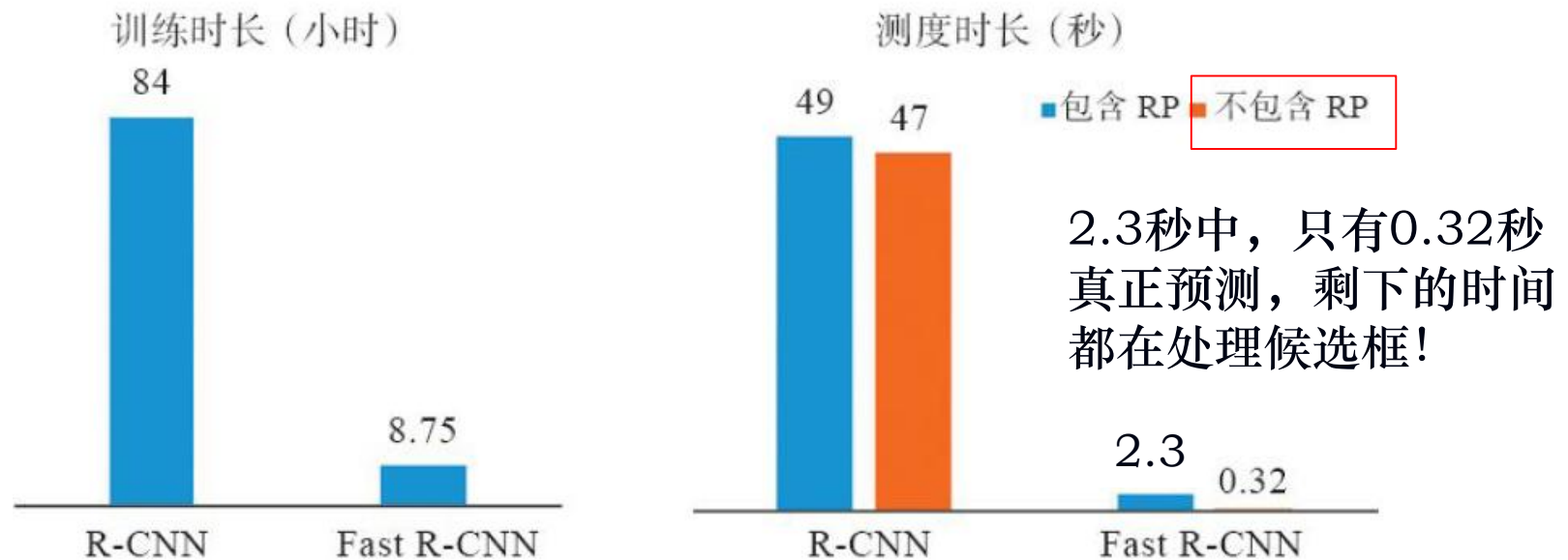


2) Fast R-CNN

➤ Fast R-CNN vs R-CNN

R-CNN: 许多候选框 -> CNN -> 得到每个候选框的特征 -> 分类+回归

Fast R-CNN: 一张图像 -> CNN -> 得到每张候选框的特征 -> 同时分类回归

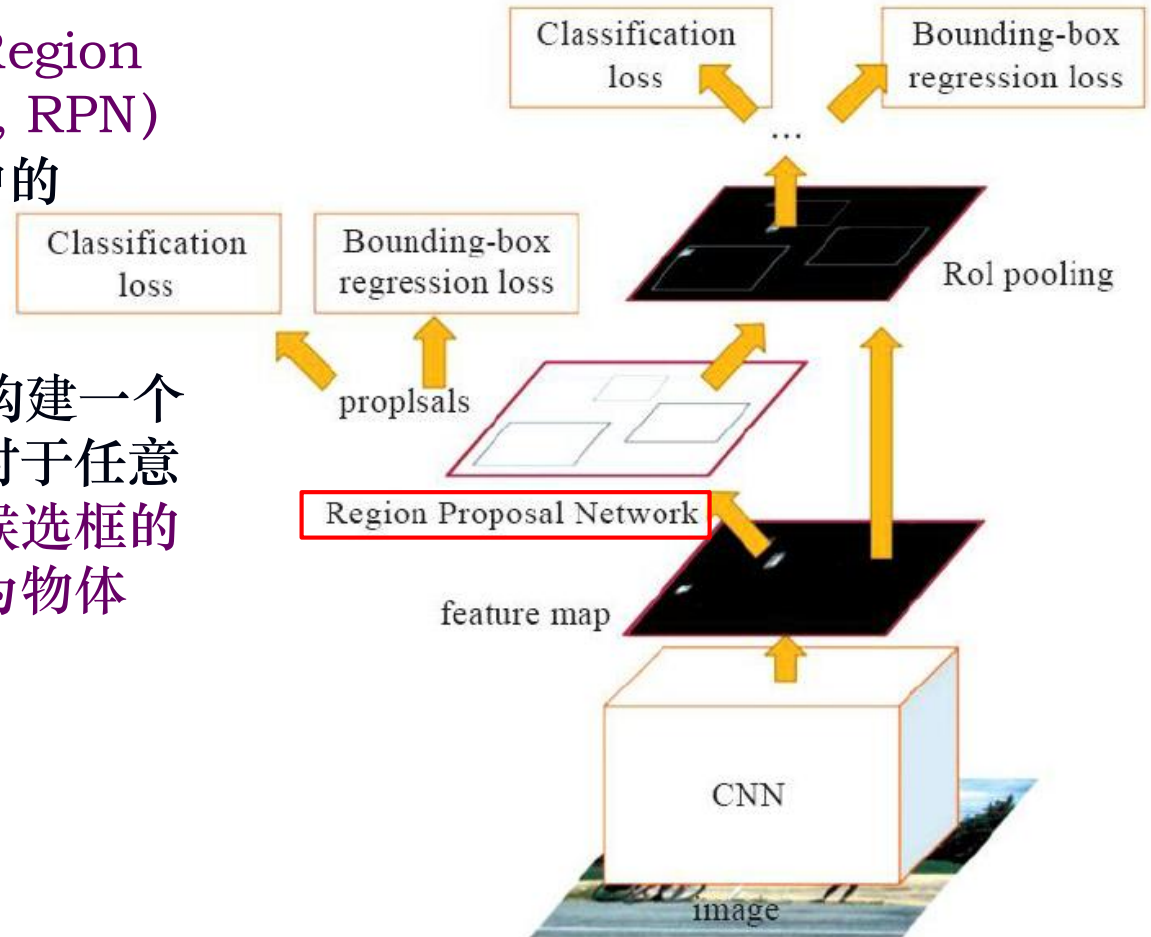


问题：处理候选框时间过长！！！！

3) Faster R-CNN

➤ Faster R-CNN算法思想

- 用区域候选网络 (Region proposal network, RPN) 代替Fast R-CNN中的selective search
- RPN的核心思想：构建一个小的全卷积网络，对于任意大小的图像，输出候选框的具体位置及其是否为物体（二分类）



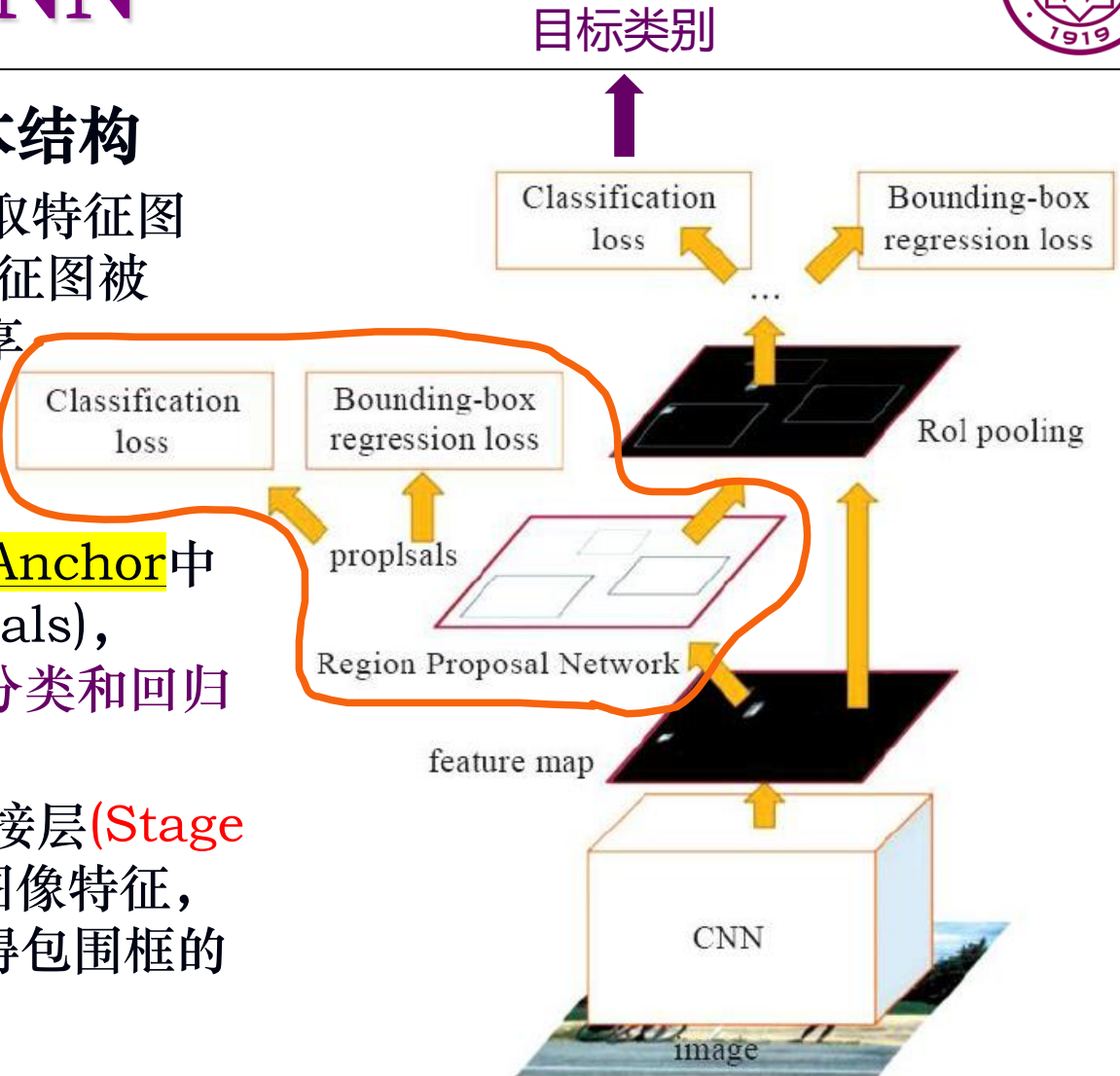
3) Faster R-CNN

➤ Faster R-CNN基本结构

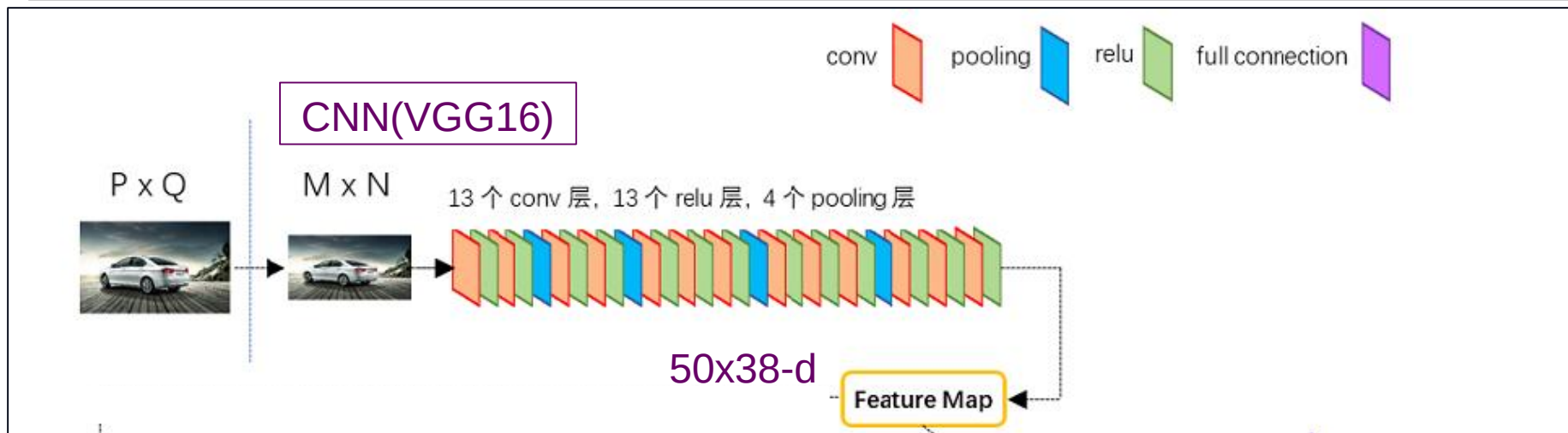
- CNN: 从图像中提取特征图 (feature map), 特征图被 RPN和全连接层共享

是否为候选区域 ←
代替selective search

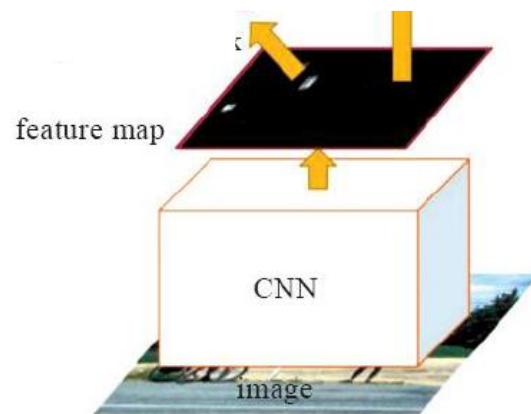
- RPN(Stage 1): 从Anchor中产生候选框(proposals), RPN中包含独立的分类和回归模块 (全连接层)
- ROI pooling+全连接层(Stage 2): 生成候选框的图像特征, 判断目标类别并获得包围框的精确位置



Faster R-CNN: 特征提取网络结构

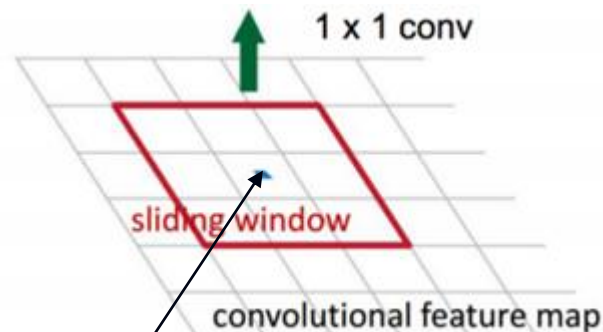
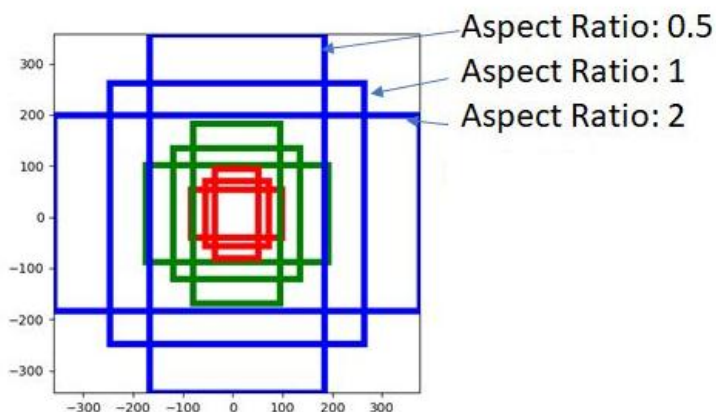


- 采用VGG16作为特征提取的模块，输入图像调整为 800×600 大小
- 所有conv层: $\text{kernel_size}=3$, $\text{pad}=1$, $\text{stride}=1$
- 所有pooling层: $\text{kernel_size}=2$, $\text{pad}=0$, $\text{stride}=2$
- 800×600 的特征图经过CNN，固定变为 $(800/16) \times (600/16) = 50 \times 38$ ，深度: $512/256$



Anchor

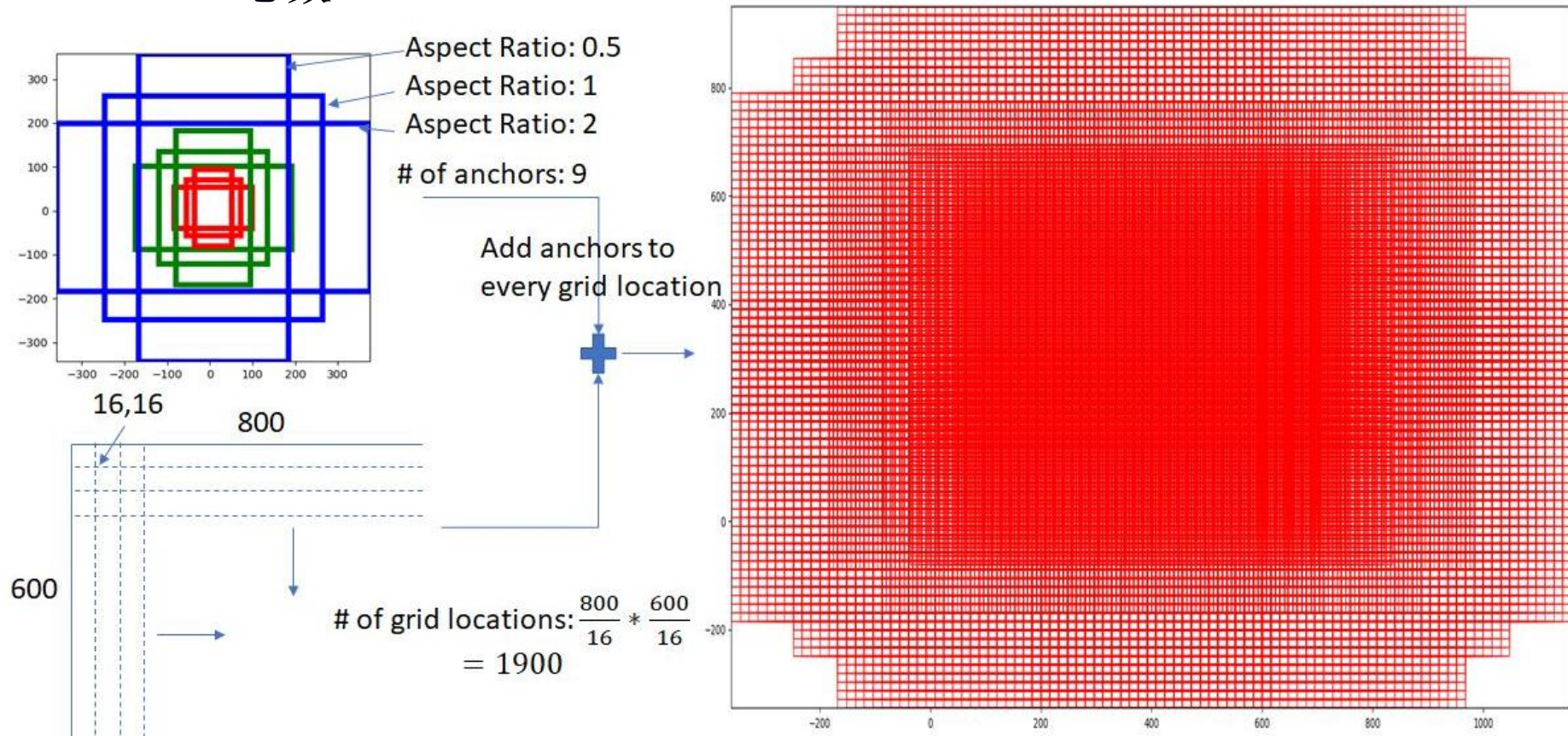
- Anchor是一组预先定义好的、在原图上位置和大小固定的矩形参考框（候选框的候选框）
- 在特征图的每一个像素点（空间位置）上，Faster R-CNN 都会预设 9 个不同大小和形状的 Anchor
 - 三个面积尺寸： $128^2, 256^2, 512^2$ (单位像素，对应原图大小)
 - 三种不同的长宽比例：1:1, 1:2, 2:1



Anchor



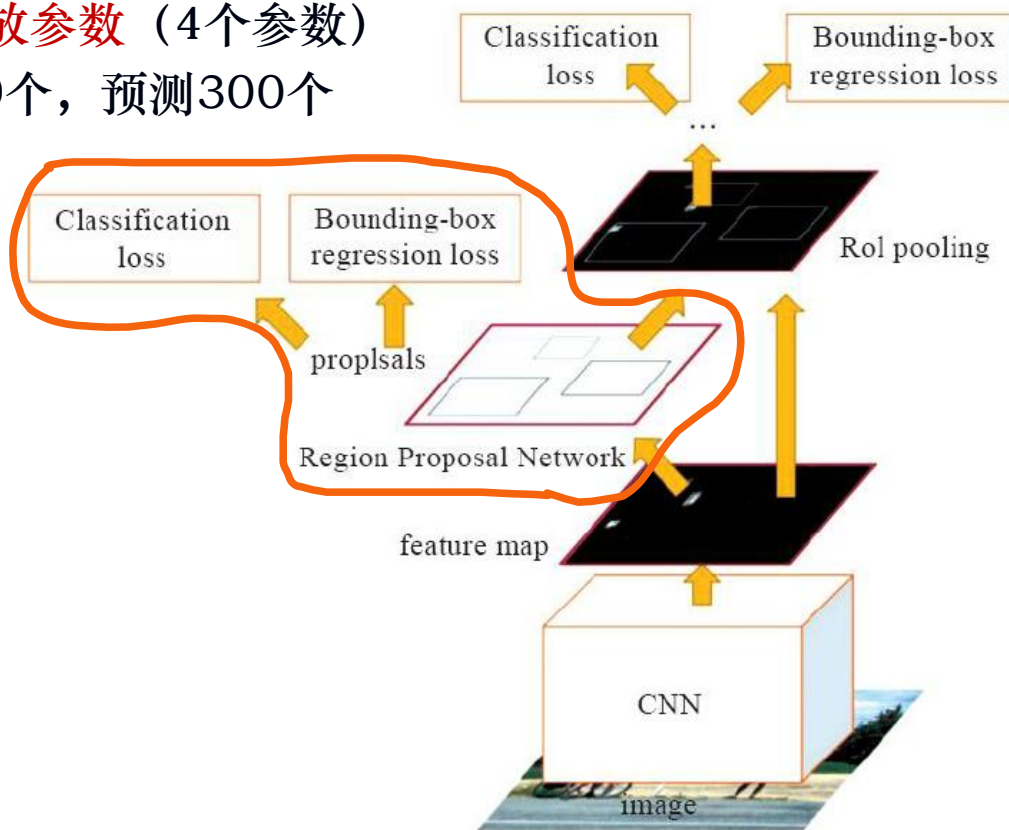
- 这些Anchor以该特征点在原始图像上的映射位置为中心，均匀地铺满整张图像
- Anchor总数: $50 \times 38 \times 9 = 17100$



RPN: 从Anchors中获得候选框

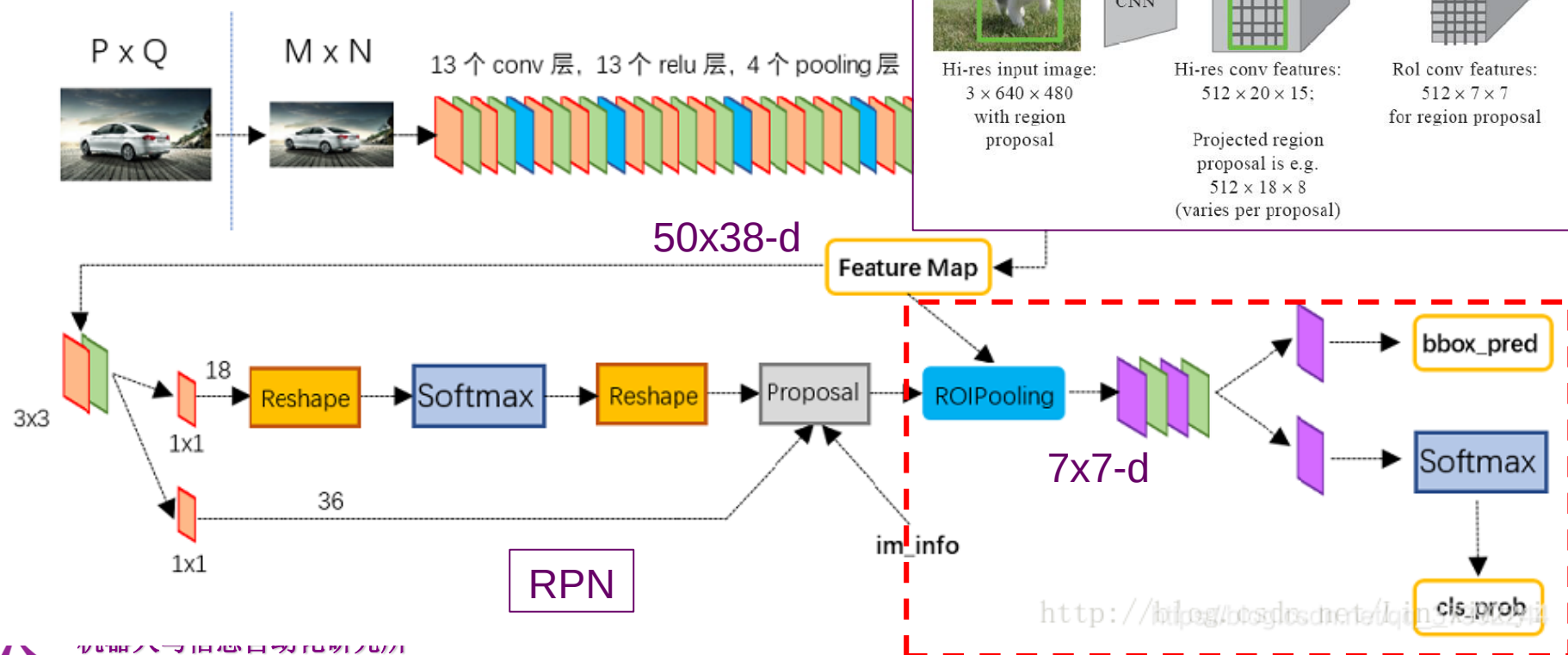
- RPN输入：特征图+17100个Anchors原始图像信息
- RPN输出：候选框信息
 - 分类网络：前景/背景概率（2个参数）
 - 回归网络：包围框的**平移缩放参数**（4个参数）
 - 候选框输出数量：训练2000个，预测300个

- RPN的作用：对于所有的Anchor图像，RPN把明显是背景的扔掉，把像物体的留下来并进行微调，最终输出高质量的候选区域（Proposals）交给后续的 Fast R-CNN 模块去做具体的分类和更精细的回归



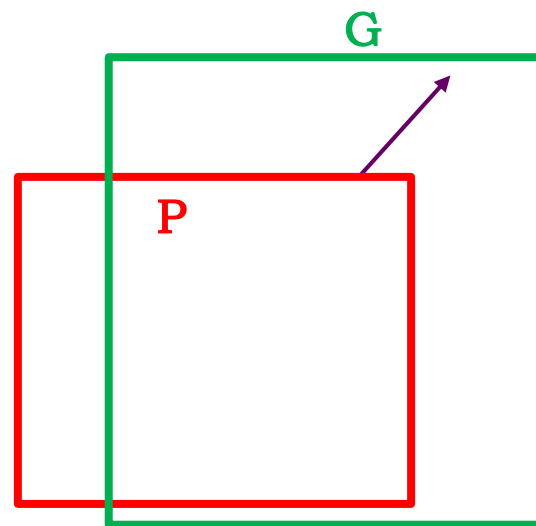
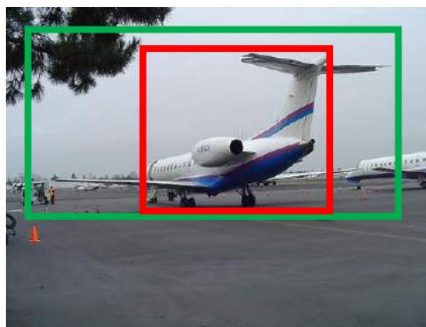
Faster R-CNN: 网络结构

- 基于候选框进行目标检测
- ROI pooling和Fast R-CNN相同，每个候选区域通过ROI pooling缩放到 7×7 大小的特征图



包围框回归：包围框映射

- 通过包围框回归(Bounding-box regression)提高定位准确率
- 窗口使用四维向量 $[x, y, w, h]$ 来表示
- 预测偏移量的优势：偏移量通常是相对于候选框而言的，因为候选框本身就已经离真实物体很近了，所以模型只需要预测一个非常微小的调整值，神经网络学习这种微调任务更轻松，收敛速度也更快
- 包围框映射：寻找一种关系，使得输入的原始候选框P(Proposals, 红色)映射到真实框G(Ground truth, 绿色)



包围框回归：包围框映射

- 设原始候选框 $P = [P_x, P_y, P_w, P_h]$ ，目标真实标注框 $G = [G_x, G_y, G_w, G_h]$
- 候选框 P 映射到真实框 G ，对它进行中心点的平移以及宽高的缩放，设变换参数为 d_x, d_y, d_w, d_h ：

$$G_x = P_x + P_w \cdot d_x(P), G_y = P_y + P_h \cdot d_y(P)$$

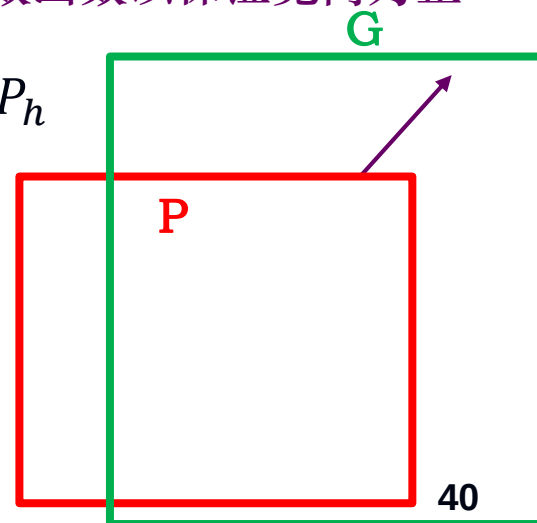
用候选框的宽高进行归一化，消除不同物体尺度带来的影响

$$G_w = P_w \cdot \exp(d_w(P)), G_h = P_h \cdot \exp(d_h(P))$$

引入指数函数以保证宽高为正

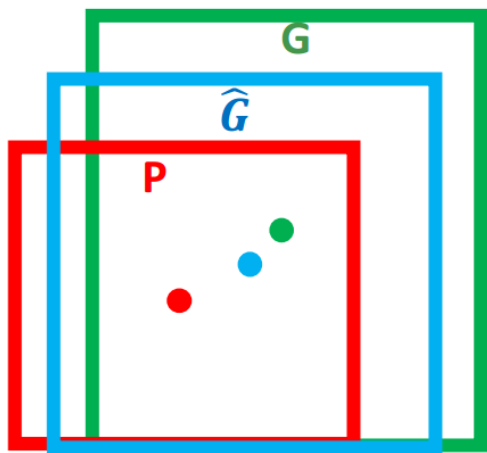
- 目标平移量： $d_x = (G_x - P_x)/P_w, d_y = (G_y - P_y)/P_h$

- 目标缩放比： $d_w = \log(G_w/P_w), d_h = \log(G_h/P_h)$



包围框回归

- 目标：寻找映射关系，使得候选包围框 **P** (Proposals, 红色) 经过映射得到一个跟真实框 **G** (Ground truth, 绿色) 更接近的预测包围框 $\hat{P}$ (蓝色) $\hat{P} = [x, y, w, h]$
 - 真实框 **G** 相对于候选包围框 **P** 的映射参数： $t^* = [t_x^*, t_y^*, t_w^*, t_h^*]$
 - 预测包围框 $\hat{P}$ 相对于候选包围框 **P** 的映射参数： $t = [t_x, t_y, t_w, t_h]$
- 回归结果



$$t_x^* = (G_x - P_x)/P_w, t_y^* = (G_y - P_y)/P_h$$

$$t_w^* = \log(G_w/P_w), t_h^* = \log(G_h/P_h)$$

$$t_x = (x - P_x)/P_w, t_y = (y - P_y)/P_h$$

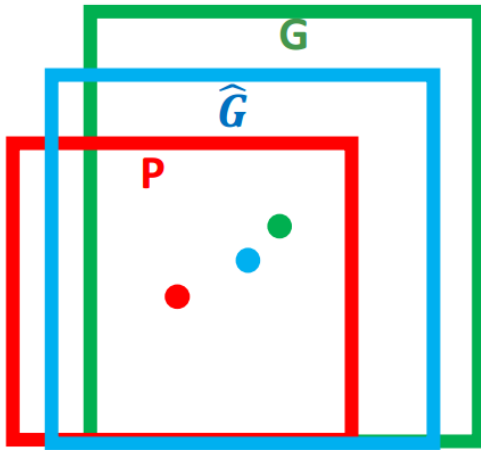
$$t_w = \log(w/P_w), t_h = \log(h/P_h)$$

真值
计算得到

网络预测
得到

包围框回归

- 目标：寻找映射关系，使得候选包围框 **P** (Proposals, 红色) 经过映射得到一个跟真实框 **G** (Ground truth, 绿色) 更接近的预测包围框 $\hat{P}$ (蓝色) $\hat{P} = [x, y, w, h]$
 - 真实框 **G** 相对于候选包围框 **P** 的映射参数： $t^* = [t_x^*, t_y^*, t_w^*, t_h^*]$
 - 预测包围框 $\hat{P}$ 相对于候选包围框 **P** 的映射参数： $t = [t_x, t_y, t_w, t_h]$
- 回归结果



```
class FastRCNNPredictor(nn.Module):
    """ ... """
    def __init__(self, in_channels, num_classes):
        super(FastRCNNPredictor, self).__init__()
        self.cls_score = nn.Linear(in_channels, num_classes)
        self.bbox_pred = nn.Linear(in_channels, num_classes * 4)

    def forward(self, x):
        if x.dim() == 4:
            assert list(x.shape[2:]) == [1, 1]
        x = x.flatten(start_dim=1)
        scores = self.cls_score(x)
        bbox_deltas = self.bbox_pred(x)
        return scores, bbox_deltas
```


Faster R-CNN: 损失函数

多任务损失

$$L(p, u, t, t^*) = L_{cls}(p, u) + \lambda[u \geq 1]L_{loc}(t, t^*)$$

在 $u \geq 1$ 时取值为1，否则为0

➤ 第一项：分类损失

- u 是目标的真实类别标签， p 是类别预测概率 $p = (p_0, \dots, p_K)$
- $L_{cls}(p, u) = -\log p_u$ 是对类别 u 的log损失

➤ 第二项：回归损失

- t^* 是真实框G相对于候选包围框P的映射参数， $t^* = [t_x^*, t_y^*, t_w^*, t_h^*]$
- t^u 是对于真实类别 u 的包围框预测参数， $t^u = [t_x^u, t_y^u, t_w^u, t_h^u]$

$$L_{loc}(t, t^*) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i^u - t_i^*)$$

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Faster R-CNN: 训练

➤ 输入

- 特征图
- RPN输出的候选框信息（训练时2000个，预测时300个）

➤ 输出

- K+1个类别的softmax预测概率: $p = (p_0, \dots, p_K)$
- K个目标类别的包围框回归输出: 第k类预测包围框的平移和尺度缩放参数（**相对候选框**）: $t^k = (t_x^k, t_y^k, t_w^k, t_h^k)$

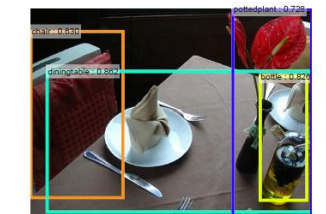
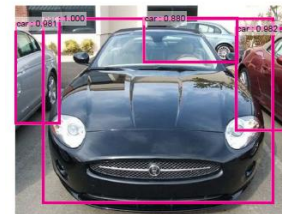
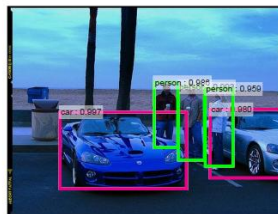
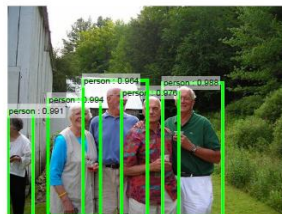
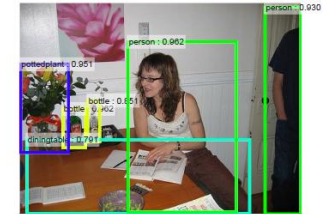
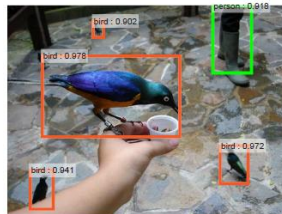
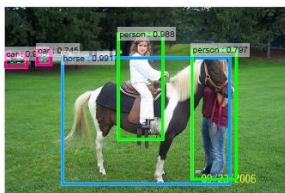
➤ Faster R-CNN的训练数据

- 在RPN输出的2000个候选框中，根据和GT包围框的IoU是否大于0.5，划分正负样本
- 随机筛选512个候选框，其中正负样本比例1: 3，正样本不够用负样本补充

3) Faster R-CNN

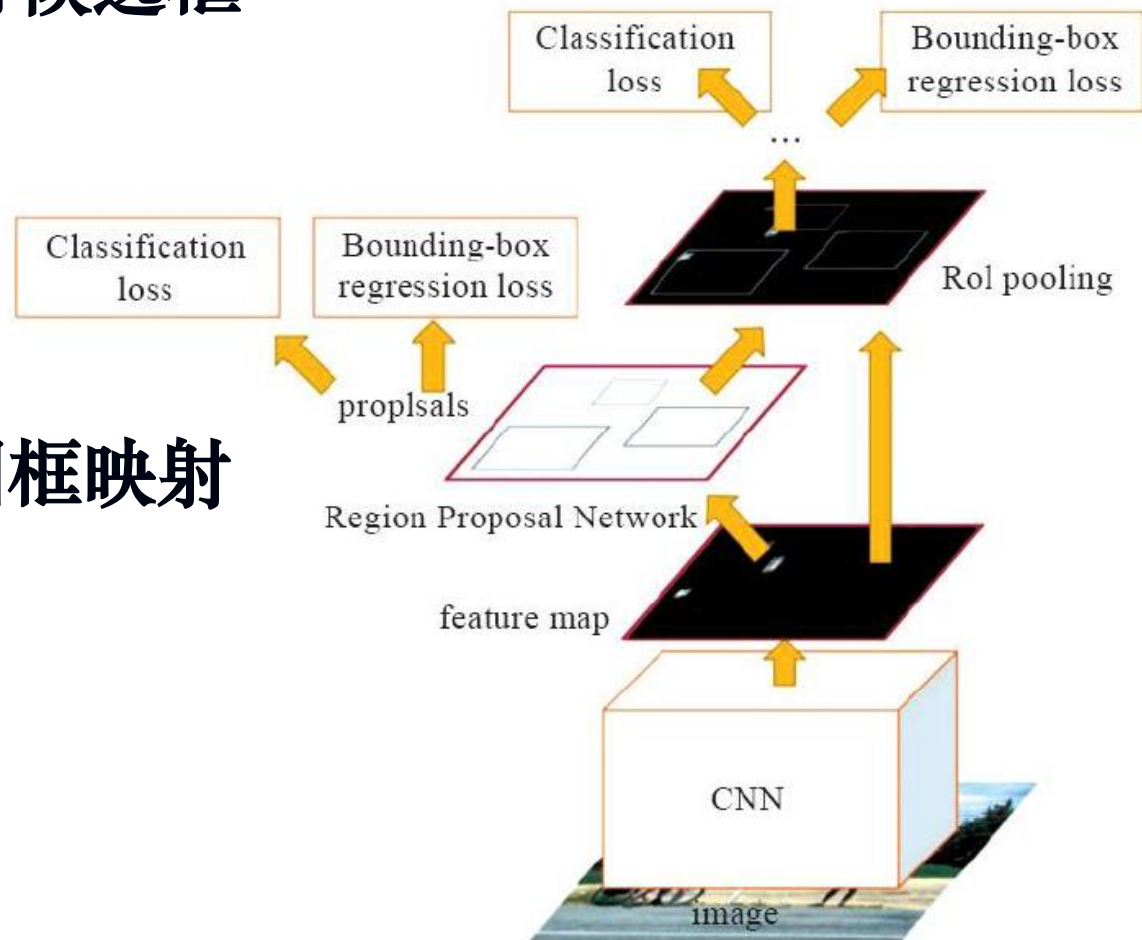
Faster R-CNN预测阶段的具体步骤

1. 将图像输入网络得到对应的特征图feature maps
2. 使用RPN生成300个候选框，将候选框投影到特征图上，获得相应的特征矩阵
3. 将每个特征矩阵通过ROI pooling缩放到7*7大小的特征图
4. 将特征图通过一系列全连接层，得到类别预测结果和针对每个类别的包围框回归结果



3) Faster R-CNN总结

- 借助Anchor获得候选框
- 网络结构
 - 特征提取
 - RPN
 - 目标识别
- 包围框回归-包围框映射
- 损失函数
- 训练与预测



编程讲解说明



➤ https://www.bilibili.com/video/BV1of4y1m7nj/?spm_id_from=333.1387.collection.video_card.click



tensorflow官方提供的目标检测与分类API

2020-03-02



PASCAL VOC2012数据集讲解与制作自己的数据集

2020-05-14



COCO数据集介绍以及pycocotools简单使用

2021-04-13



目标检测mAP计算以及coco评价标准

2020-07-04



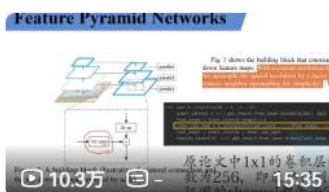
0.目标检测前言

2020-05-02



1.1Faster RCNN理论合集

2020-05-03



1.1.2 FPN结构详解

2021-04-19



1.2Faster RCNN源码解析(pytorch)

2020-05-17



2.1SSD算法理论

2020-07-24



2.1.2 RetinaNet网络结构详解

2021-04-21



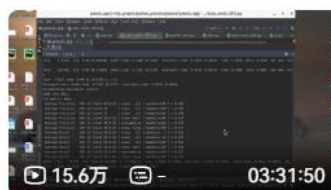
2.2 SSD源码解析(Pytorch)

2020-07-12



3.1 YOLO系列理论合集(YOLOv1~v3)

2020-08-23



3.2 YOLOv3 SPP源码解析(Pytorch版)

2020-10-30



YOLOv4网络详解

2022-03-09



YOLOv5网络详解

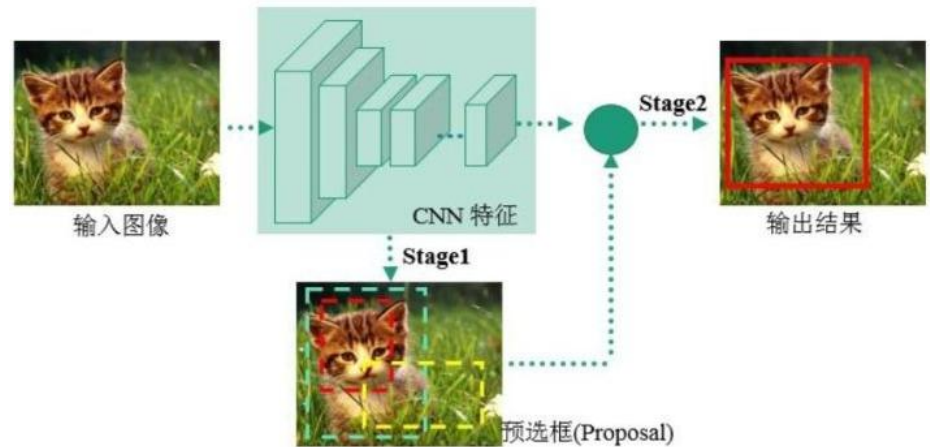
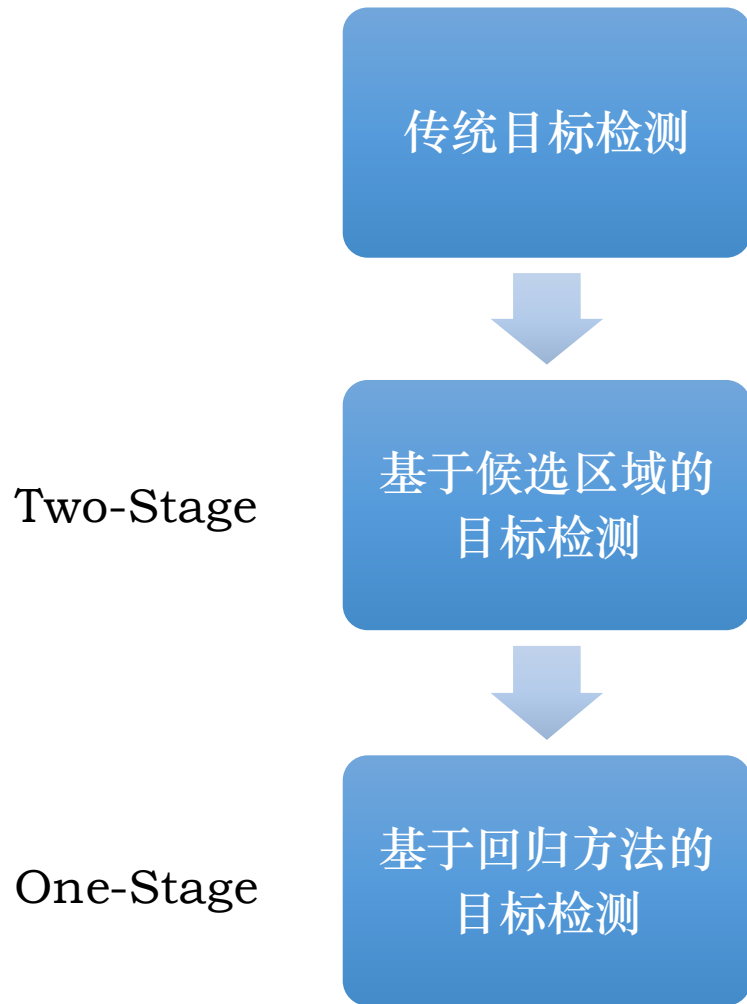
2022-03-21

8.1.2 Two-stage目标检测方法

- 从R-CNN到Fast R-CNN，再到Faster R-CNN，目标检测的四个基本步骤（候选框生成、特征提取、分类、包围框回归）被逐渐统一到一个深度网络框架中
- Faster R-CNN的计算没有重复，可完全在GPU中完成，大大提高了运行速度
- 目标检测速度达到5fps，在PASCAL VOC上准确率73.2%

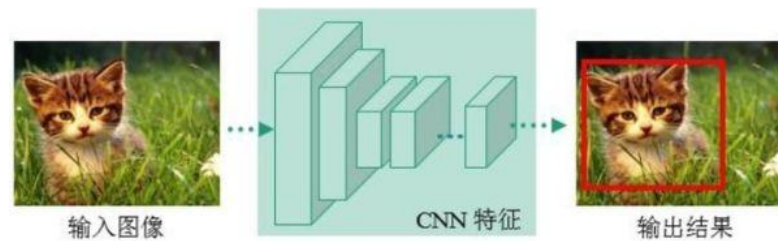
预测时长：49s	2.3s	0.2s												
<table><tr><td colspan="2">region proplsal (SS)</td></tr><tr><td colspan="2">feature extraction (Deep Net)</td></tr><tr><td>classification (SVM)</td><td>rect refine (regression)</td></tr></table>	region proplsal (SS)		feature extraction (Deep Net)		classification (SVM)	rect refine (regression)	<table><tr><td colspan="2">region proplsal (SS)</td></tr><tr><td colspan="2">feature extraction classification + rect refine (Deep Net)</td></tr></table>	region proplsal (SS)		feature extraction classification + rect refine (Deep Net)		<table><tr><td colspan="2">region proposal feature extraction classification + rect refine (Deep Net)</td></tr></table>	region proposal feature extraction classification + rect refine (Deep Net)	
region proplsal (SS)														
feature extraction (Deep Net)														
classification (SVM)	rect refine (regression)													
region proplsal (SS)														
feature extraction classification + rect refine (Deep Net)														
region proposal feature extraction classification + rect refine (Deep Net)														
RCNN	fast RCNN	faster RCNN												

8.1 目标检测



Two Stage结构

知乎 @Hero



One Stage结构

8.1.3 One-stage目标检测方法

You Only Look Once: Unified, Real-Time Object Detection

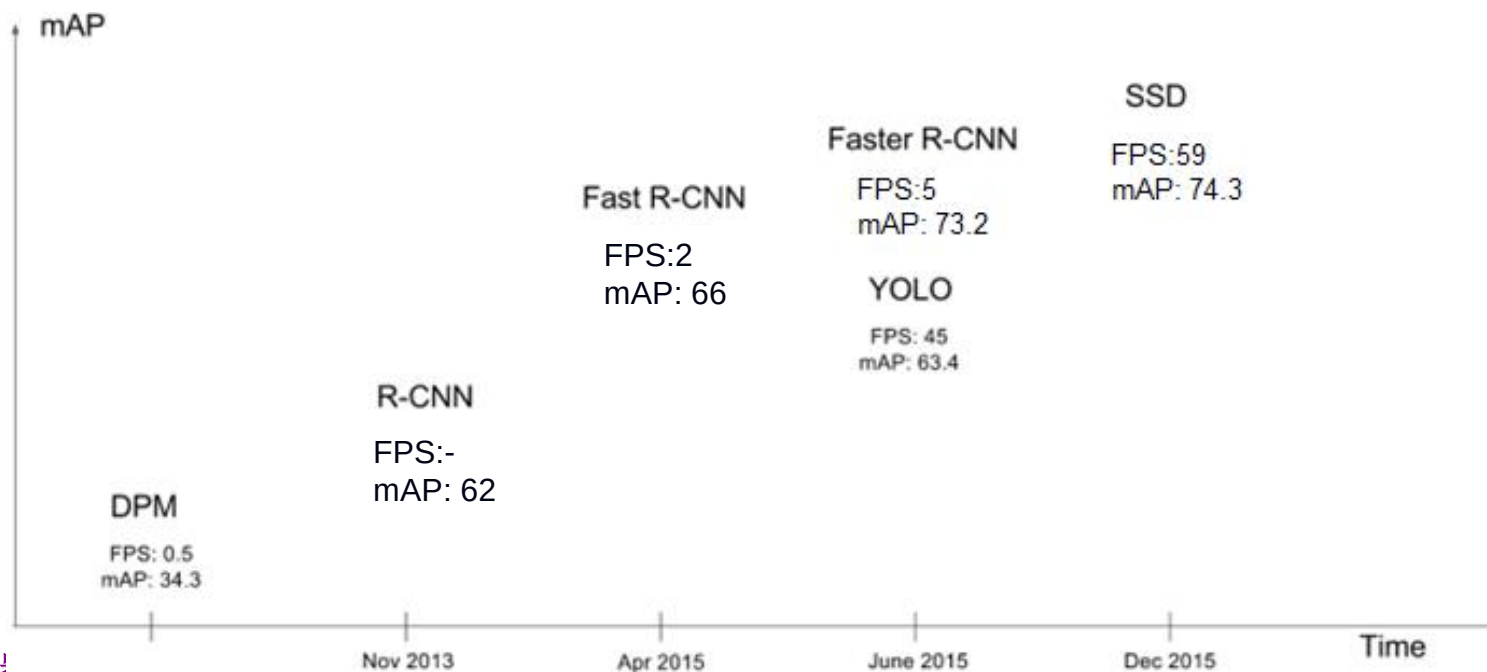
CVPR2016

Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi

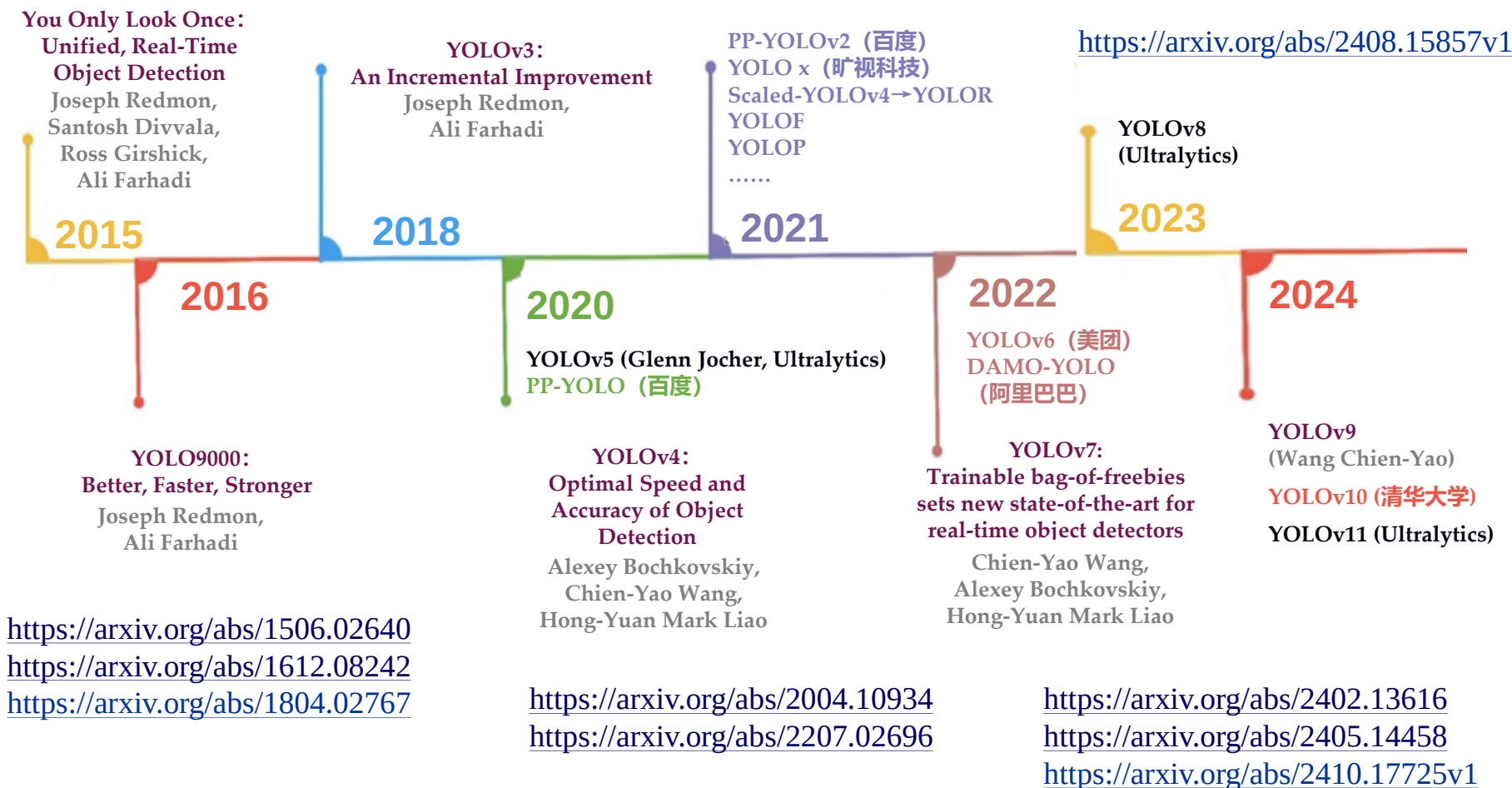
SSD: Single Shot MultiBox Detector

ECCV2016

Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, etc.



YOLO系列



YOLO系列

<https://docs.ultralytics.com/zh/models/>



- YOLOv1-v3(15/16/18): Joseph Redmon, 以其高效的实时目标检测能力而闻名
- YOLOv4-v7-v9(20/22/24): 台湾中央研究院资讯科学研究所
- YOLOv5-v8-11(20/23/24): Ultralytics 公司, YOLO11在包括检测、分割、姿势估计、跟踪和分类在内的多个任务中提供最先进 (SOTA) 的性能
- YOLO26(26): 是 YOLO 系列实时对象检测器的最新演进, 从头开始专为边缘和低功耗设备而设计
- YOLOv6(22): 美团, 应用于该公司的许多自动送货机器人中
- YOLOv10(24): 清华, 具有无 NMS 训练和效率驱动的架构, 可提供最先进的性能和延迟
- YOLOv12(25): 布法罗大学、中国科学院大学, 通过在注意力机制和整体网络架构方面的创新方法, 实现了最先进的目标检测精度, 同时保持了实时性能

YOLOv3

<https://arxiv.org/abs/1804.02767>

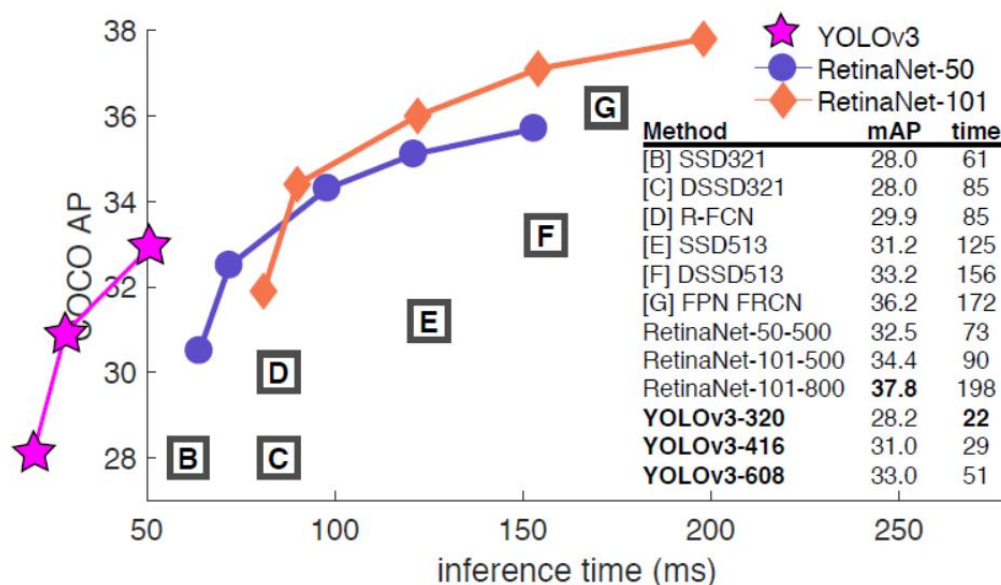


Figure 1. We adapt this figure from the Focal Loss paper [9]. YOLOv3 runs significantly faster than other detection methods with comparable performance. Times from either an M40 or Titan X, they are basically the same GPU.

[9] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. *arXiv preprint arXiv:1708.02002*, 2017. 1, 3, 4



机器人与信息自动化研究所

Abstract

We present some updates to YOLO! We made a bunch of little design changes to make it better. We also trained this new network that's pretty swell. It's a little bigger than last time but more accurate. It's still fast though, don't worry. At 320×320 YOLOv3 runs in 22 ms at 28.2 mAP, as accurate as SSD but three times faster. When we look at the old .5 IOU mAP detection metric YOLOv3 is quite good. It achieves 57.9 AP₅₀ in 51 ms on a Titan X, compared to 57.5 AP₅₀ in 198 ms by RetinaNet, similar performance but 3.8× faster. As always, all the code is online at <https://pjreddie.com/yolo/>.

1. Introduction

Sometimes you just kinda phone it in for a year, you know? I didn't do a whole lot of research this year. Spent a lot of time on Twitter. Played around with GANs a little. I had a little momentum left over from last year [12] [1]; I managed to make some improvements to YOLO. But, honestly, nothing like super interesting, just a bunch of small changes that make it better. I also helped out with other people's research a little.

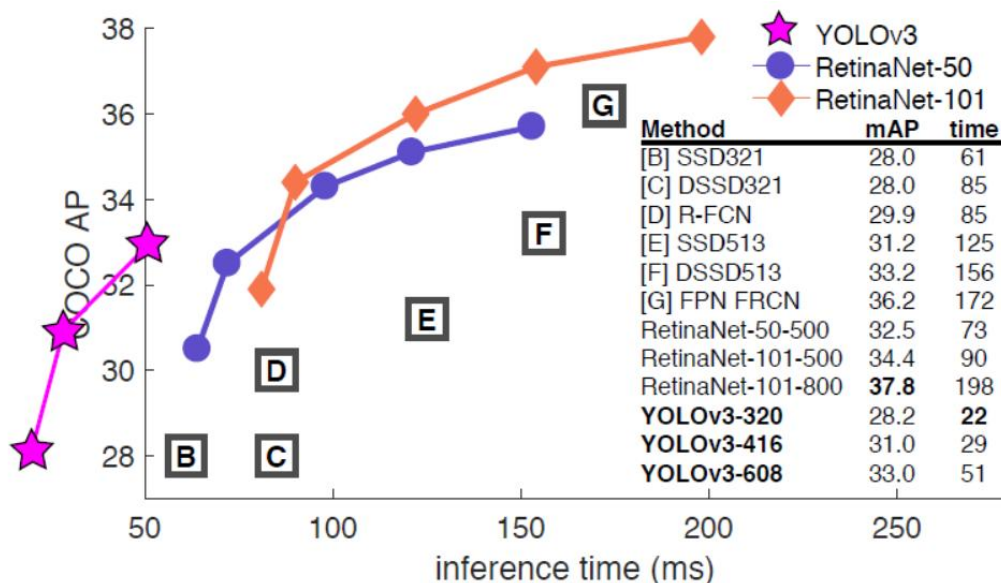


Figure 1. We adapt this figure from the Focal Loss paper [9]. YOLOv3 runs significantly faster than other detection methods with comparable performance. Times from either an M40 or Titan X, they are basically the same GPU.

[9] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. *arXiv preprint arXiv:1708.02002*, 2017. 1, 3, 4

- 网络结构
- Anchor
- 包围框映射
- 包围框置信度
- 损失函数
- 实验结果

YOLOv3: 网络结构 backbone

	Type	Filters	Size	Output
	Convolutional	32	3×3	256×256
	Convolutional	64	$3 \times 3 / 2$	128×128
1x	Convolutional	32	1×1	
	Convolutional	64	3×3	
	Residual			128×128
	Convolutional	128	$3 \times 3 / 2$	64×64
2x	Convolutional	64	1×1	
	Convolutional	128	3×3	
	Residual			64×64
	Convolutional	256	$3 \times 3 / 2$	32×32
8x	Convolutional	128	1×1	
	Convolutional	256	3×3	
	Residual			32×32
	Convolutional	512	$3 \times 3 / 2$	16×16
8x	Convolutional	256	1×1	
	Convolutional	512	3×3	
	Residual			16×16
	Convolutional	1024	$3 \times 3 / 2$	8×8
4x	Convolutional	512	1×1	
	Convolutional	1024	3×3	
	Residual			8×8
	Avgpool		Global	
	Connected		1000	
	Softmax			

Darknet-53

Darknet-53特点:

- 弃用池化层: Darknet-53 没有使用任何最大池化层来进行特征图的降维。它完全通过步长为 2 (stride=2) 的卷积层来实现下采样。好处是保留了更多的空间信息, 避免了池化操作带来的信息丢失
- 全卷积结构: 整个网络几乎完全由 3×3 和 1×1 的卷积层堆叠而成, 没有使用全连接层, 这使得它在处理不同尺寸输入时更加灵活
- 1×1 卷积降维: 在残差块内部, 通常先使用 1×1 卷积来压缩特征图的通道数, 再通过 3×3 卷积提取特征, 最后再恢复通道数。这种“瓶颈 (Bottleneck)”设计在保证特征表达能力的同时, 有效降低计算量

YOLOv3: 网络结构 backbone

	Type	Filters	Size	Output
1x	Convolutional	32	3×3	256×256
	Convolutional	64	$3 \times 3 / 2$	128×128
	Convolutional	32	1×1	128×128
	Convolutional	64	3×3	
	Residual			
2x	Convolutional	128	$3 \times 3 / 2$	64×64
	Convolutional	64	1×1	64×64
	Convolutional	128	3×3	
	Residual			
	Convolutional	256	$3 \times 3 / 2$	32×32
4x	Convolutional	128	1×1	32×32
	Convolutional	256	3×3	
	Residual			
	Convolutional	512	$3 \times 3 / 2$	16×16
	Convolutional	256	1×1	16×16
8x	Convolutional	512	3×3	
	Residual			
	Convolutional	1024	$3 \times 3 / 2$	8×8
	Convolutional	512	1×1	8×8
	Convolutional	1024	3×3	
4x	Residual			
	Avgpool		Global	
	Connected		1000	
	Softmax			

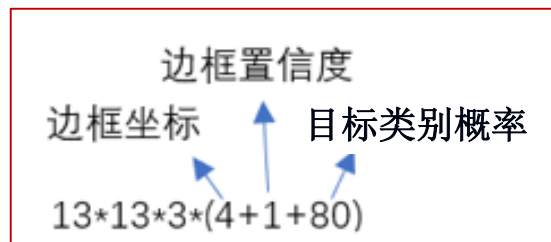
Darknet-53

Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19 [15]	74.1	91.8	7.29	1246	171
ResNet-101[5]	77.1	93.7	19.7	1039	53
ResNet-152 [5]	77.6	93.8	29.4	1090	37
Darknet-53	77.2	93.8	18.7	1457	78

- Top-1 Accuracy是指排名第一的类别与实际结果相符的准确率
- Darknet-53比ResNet-101更好，速度更快1.5倍；Darknet-53与ResNet-152具有相似的性能，速度提高2倍
- Darknet-53也可以实现每秒最高的测量浮点运算。这意味着网络结构可以更好地利用GPU，从而使其评估效率更高，速度更快。这主要是因为ResNets的层数太多，效率不高

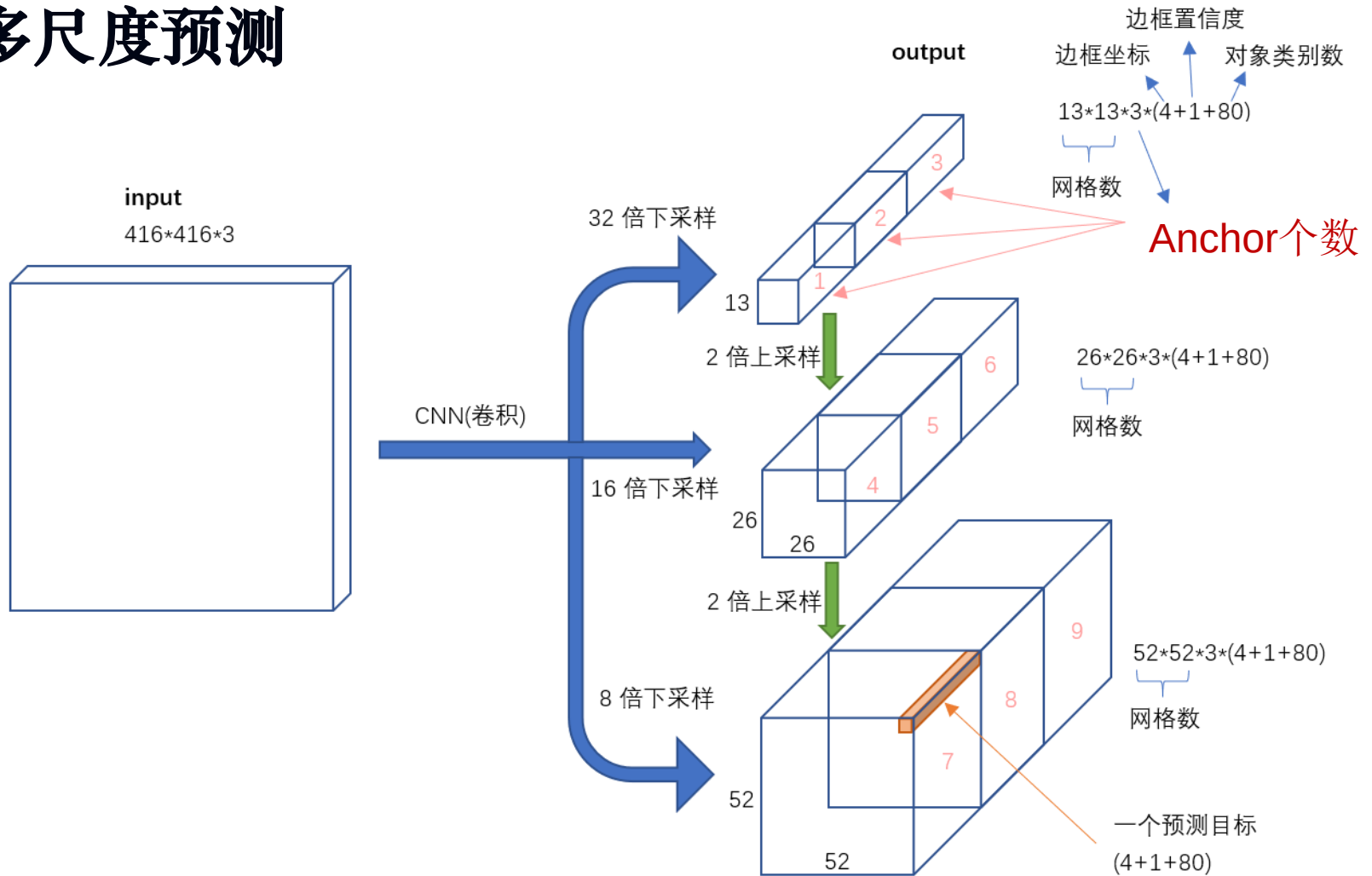
YOLOv3: 网络结构

- YOLOv3借鉴了特征金字塔的思想，使用3个不同尺度的特征图进行目标检测
 - 卷积网络先得到32倍下采样的特征图，对于输入416*416的图像，特征图尺寸为13*13，由于下采样倍数高，这里特征图的感受野比较大，因此**适合检测图像中尺寸比较大的对象**
 - 对该特征图作上采样，得到16倍下采样的特征图，它具有中等尺度的感受野，**适合检测中等尺度的对象**，以实现细粒度的检测
 - 对该特征图再次上采样，得到8倍下采样的特征图，它的感受野最小，**适合检测小尺寸的对象**
- YOLOv3在三个特征图中分别通过 $(4+1+c) \times k$ 个 1×1 的卷积核进行预测，k为Anchor数量(=3)，c为预测目标类别数(=80)
- 最终获得尺寸为 $N \times N \times 3 \times 85$ 的预测结果，N为特征图尺寸(=13,26,52)



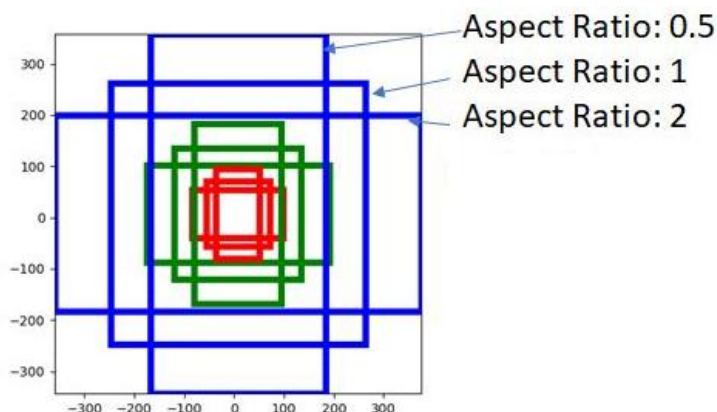
YOLOv3: 网络结构

➤ 多尺度预测



回顾：Faster R-CNN中的Anchor

- Anchor是一组预先定义好的、在原图上位置和大小固定的矩形参考框
- Faster R-CNN在特征图上使用9种面积、尺寸各异的Anchor
 - 三个面积尺寸： $128^2, 256^2, 512^2$
 - 三种不同的长宽比例：1:1, 1:2, 2:1
- Faster R-CNN的Anchor数量： $50 \times 38 \times 9 = 17100$



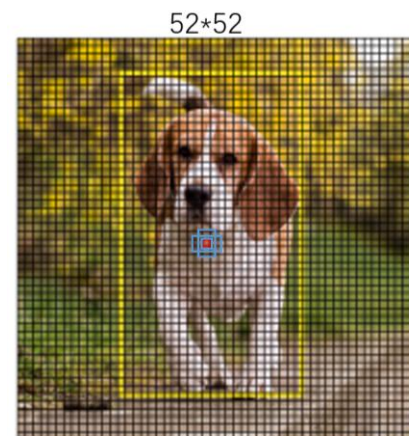
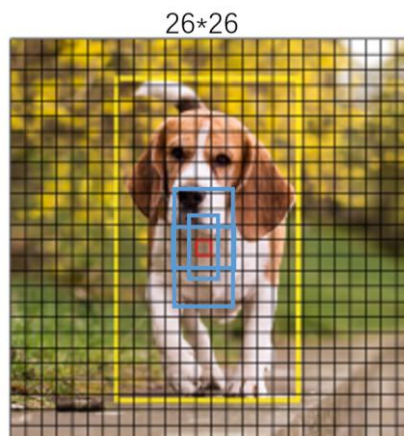
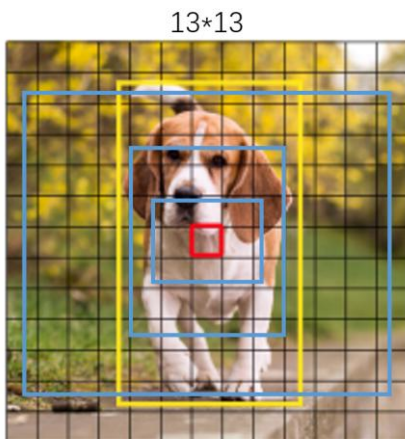
问题：Anchor的尺寸与比例固定，非常依赖于经验调整

YOLOv3: Anchor

- 通过对训练集中标注的GT包围框进行K-means聚类，统计出更符合样本中对象尺寸的Anchor宽高

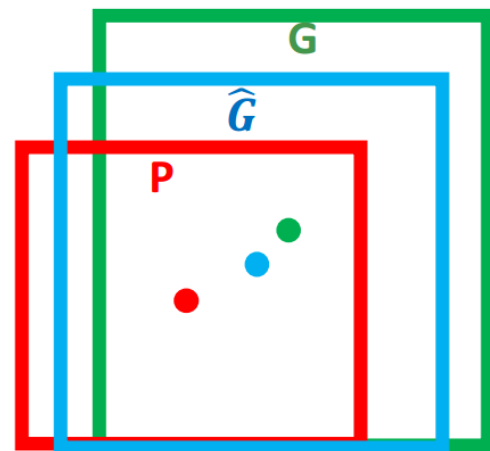
COCO数据集的聚类结果 (Anchor总数: 10647)

特征层尺寸	Anchor尺寸	Anchor总数
13*13	116*90, 156*198, 373 *326	13*13*3
26*26	30*61, 62*45, 59*119	26*26*3
52*52	10*13, 16*30, 33*23	52*52*3



回顾：RPN包围框映射

- 目标：寻找映射关系，使得候选包围框 P (Proposals, 红色) 经过映射得到一个跟真实框 G (Ground truth, 绿色) 更接近的预测包围框 $\hat{P}$ (蓝色) $\hat{P} = [x, y, w, h]$



$$t_x^* = (G_x - P_x)/P_w, t_y^* = (G_y - P_y)/P_h$$

$$t_w^* = \log(G_w/P_w), t_h^* = \log(G_h/P_h)$$

真值
计算得到

$$t_x = (x - P_x)/P_w, t_y = (y - P_y)/P_h$$

$$t_w = \log(w/P_w), t_h = \log(h/P_h)$$



$$x = t_x * P_w + P_x$$

$$y = t_y * P_h + P_y$$

网络预测得到

问题：预测的包围框位置(x, y)没有限制，因此，预测包围框窗口可以出现在图像中的任意位置，导致网络不容易收敛

YOLOv3: 包围框映射

- 将预测包围框的**中心**限制在当前的grid网格中

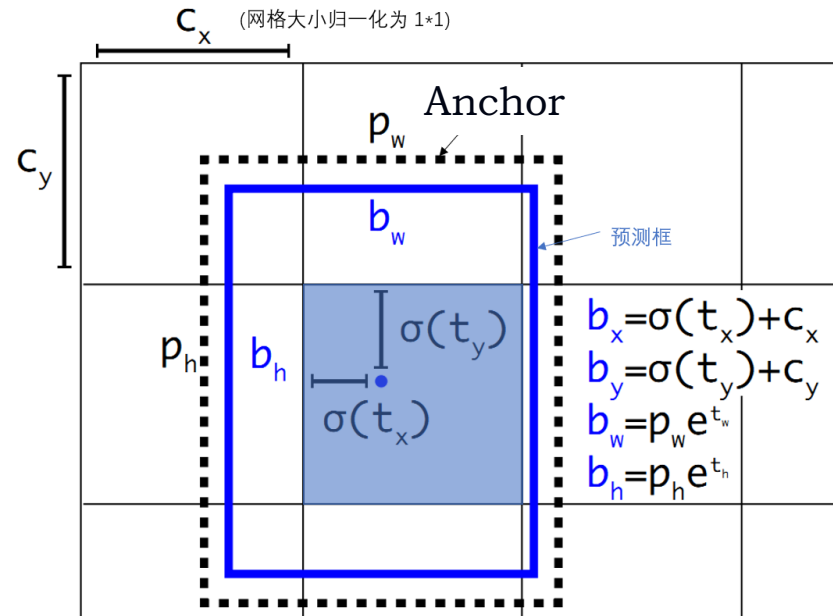
预测的映射参数 $t = [t_x, t_y, t_w, t_h]$

计算得到的
包围框位置
与尺寸

$$\begin{aligned} b_x &= \sigma(t_x) + c_x \\ b_y &= \sigma(t_y) + c_y \\ b_w &= p_w e^{t_w} \\ b_h &= p_h e^{t_h} \end{aligned}$$

其中, c_x, c_y 是当前负责预测该物体的网格单元的左上角在特征图上的坐标索引

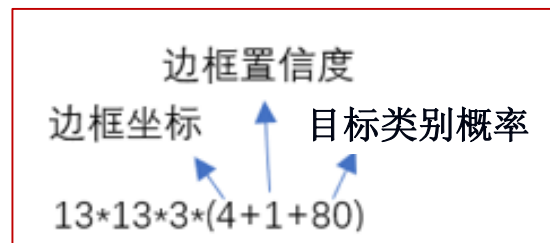
$\sigma(*)$ 是sigmoid函数, 取值范围为0-1, $\sigma(t_x), \sigma(t_y)$ 将神经网络预测的相对偏移量限制在0-1之间, **确保 b_x, b_y 在当前网格中**



$$\begin{aligned} x &= t_x * P_w + P_x \\ y &= t_y * P_h + P_y \end{aligned}$$

YOLOv3: 包围框置信度

- 置信度 (Confidence) 可理解为模型对这次检测结果的“综合把握程度”，由两个核心部分共同决定：物体置信度与条件类别置信度
- 物体置信/边框置信度
 - 表示预测框内存在任意目标物体的概率，而不区分具体是什么类别，取值范围在 0 到 1 之间，越接近 1，说明模型越确信这个框里不是纯背景，而是包含了一个真实的物体
 - 计算方式：在网络输出时，这是一个直接预测的数值，通常会经过 Sigmoid 函数处理，将其映射到 0~1 的区间内
- 条件类别置信度/目标类别概率
 - 表示在已知框内存在物体的前提下，这个物体属于某个特定类别的概率
 - 计算方式：如果模型要检测 80 种物体，那么每个预测框就会输出 80 个类别概率值



YOLOv3: 损失函数

$$L = \lambda_{loc} L_{loc}(t, g) + \lambda_{cls} L_{cls}(O, C) + \lambda_{conf} L_{conf}(o, c)$$

定位损失
(L_2 loss)

分类损失
(BCE loss)

置信度损失
(BCE loss)

➤ 置信度损失:

$$L_{conf}(o, c) = -\frac{1}{N} \sum_i [o_i \log(\sigma(c_i)) + (1 - o_i) \log(1 - \sigma(c_i))]$$

- objectness score: 预测包围框中含有目标的概率
- o_i : 原文中 $o_i \in \{0,1\}$, 预测包围框i中是否存在目标
- o_i : 通常使用 $o_i \in [0,1]$, 预测包围框i与GT包围框的IoU
- c_i : 预测包围框i中是否存在目标的网络预测值
- $\sigma(c_i)$: 预测包围框i中是否存在目标的Sigmoid概率
- N: 总样本数

YOLOv3: 损失函数

$$L = \lambda_{loc} L_{loc}(t, g) + \lambda_{cls} L_{cls}(O, C) + \lambda_{conf} L_{conf}(o, c)$$

定位损失
(L_2 loss)

分类损失
(BCE loss)

置信度损失
(BCE loss)

➤ 分类损失：只有**正样本**才计算分类损失

$$L_{cls}(O, C) = -\frac{1}{N_{pos}} \sum_{i \in pos} \sum_{j \in cls} [O_{ij} \log(\sigma(C_{ij})) + (1 - O_{ij}) \log(1 - \sigma(C_{ij}))]$$

- $O_{ij} \in \{0, 1\}$: 预测包围框*i*中是否存在第*j*类目标, $\sum_{j=1}^C O_{ij} = 1$
- C_{ij} : 预测包围框*i*中存在第*j*类目标的网络预测值
- $\sigma(C_{ij})$: 预测包围框*i*中存在第*j*类目标的**sigmoid概率** (注意不是softmax)
- N_{pos} : 正样本个数

YOLOv3: 损失函数

$$L = \lambda_{loc} L_{loc}(t, g) + \lambda_{cls} L_{cls}(O, C) + \lambda_{conf} L_{conf}(o, c)$$

定位损失
(L_2 loss)

分类损失
(BCE loss)

置信度损失
(BCE loss)

➤ 定位损失：只有**正样本**才有定位损失

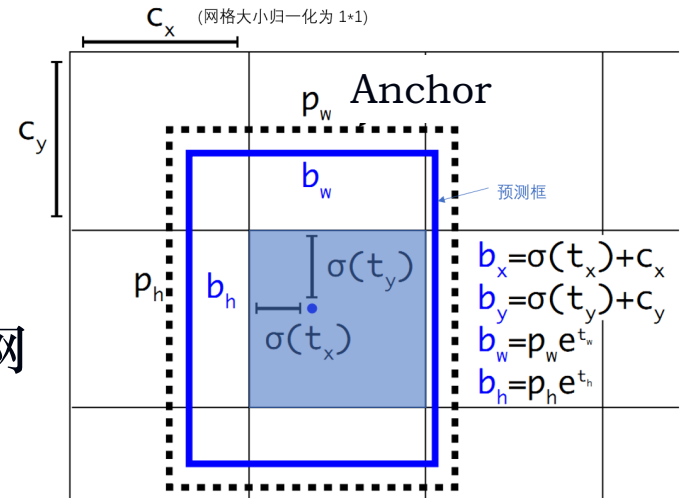
$$L_{loc}(t, g) = \frac{1}{N_{pos}} \sum_{i \in pos} \left[(\sigma(t_x^i) - \hat{g}_x^i)^2 + (\sigma(t_y^i) - \hat{g}_y^i)^2 + (t_w^i - \hat{g}_w^i)^2 + (t_h^i - \hat{g}_h^i)^2 \right]$$

➤ $t_x^i, t_y^i, t_w^i, t_h^i$: 网络预测参数

➤ $\hat{g}_x^i = g_x^i - c_x^i, \hat{g}_y^i = g_y^i - c_y^i$

$$\hat{g}_w^i = \ln\left(\frac{g_w^i}{p_w^i}\right), \hat{g}_h^i = \ln\left(\frac{g_h^i}{p_h^i}\right)$$

➤ $g_x^i, g_y^i, g_w^i, g_h^i$: GT包围框映射到Grid网格的中心点坐标, 包围框宽高



补充：非最大抑制（NMS）

Algorithm 1 Non-Maximum Suppression Algorithm

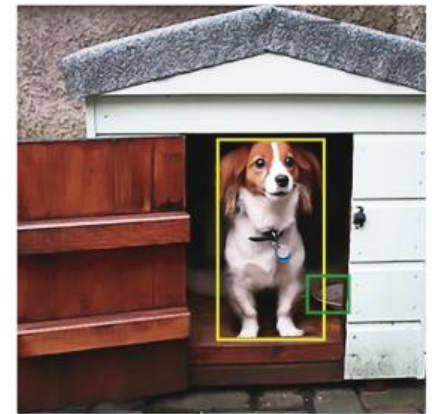
Require: Set of predicted bounding boxes B , confidence scores S , IoU threshold τ , confidence threshold T

Ensure: Set of filtered bounding boxes F

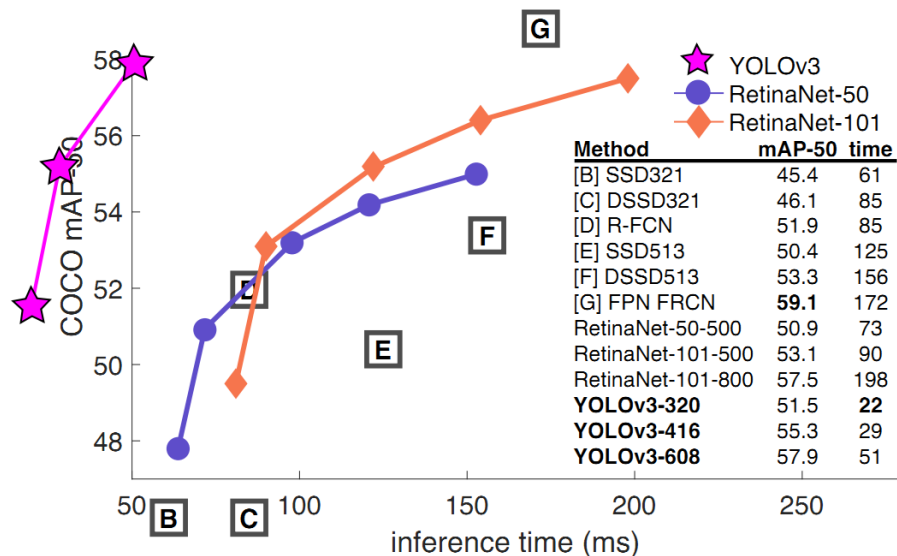
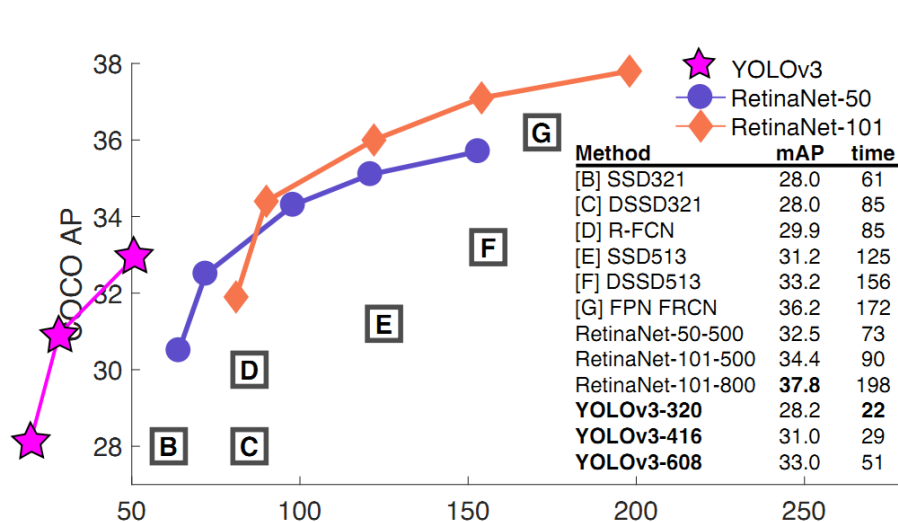
```
1:  $F \leftarrow \emptyset$ 
2: Filter the boxes:  $B \leftarrow \{b \in B \mid S(b) \geq T\}$ 
3: Sort the boxes  $B$  by their confidence scores in descending order
4: while  $B \neq \emptyset$  do
5:   Select the box  $b$  with the highest confidence score
6:   Add  $b$  to the set of final boxes  $F$ :  $F \leftarrow F \cup \{b\}$ 
7:   Remove  $b$  from the set of boxes  $B$ :  $B \leftarrow B - \{b\}$ 
8:   for all remaining boxes  $r$  in  $B$  do
9:     Calculate the IoU between  $b$  and  $r$ :  $iou \leftarrow IoU(b, r)$ 
10:    if  $iou \geq \tau$  then
11:      Remove  $r$  from the set of boxes  $B$ :  $B \leftarrow B - \{r\}$ 
12:    end if
13:  end for
14: end while
```



➔
NMS



YOLOv3: 实验结果



	backbone	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
<i>Two-stage methods</i>							
Faster R-CNN+++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
<i>One-stage methods</i>							
YOLOv2 [15]	DarkNet-19 [15]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [11, 3]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [3]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [9]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [9]	ResNeXt-101-FPN	40.8	61.1	44.1	24.1	44.2	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

编程讲解说明



➤ https://www.bilibili.com/video/BV1t54y1C7ra?spm_id_from=333.788.videoepisode&vd_source=9b166dc770f19e6acbdebfdaa1dd8810



tensorflow官方提供的目标检测与分类API

2020-03-02



PASCAL VOC2012数据集讲解与制作自己的数据集

2020-05-14



COCO数据集介绍以及pycocotools简单使用

2021-04-13



目标检测mAP计算以及coco评价标准

2020-07-04



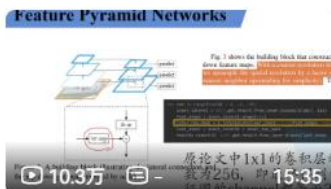
0.目标检测前言

2020-05-02



1.1Faster RCNN理论合集

2020-05-03



1.1.2 FPN结构详解

2021-04-19



1.2Faster RCNN源码解析(pytorch)

2020-05-17



2.1SSD算法理论

2020-07-24



2.1.2 RetinaNet网络结构详解

2021-04-21



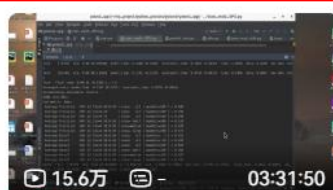
2.2 SSD源码解析(Pytorch)

2020-07-12



3.1 YOLO系列理论合集(YOLOv1~v3)

2020-08-23



3.2 YOLOv3 SPP源码解析(Pytorch版)

2020-10-30



YOLOv4网络详解

2022-03-09



YOLOv5网络详解

2022-03-21

网络结构配置

`./cfg/yolov3-spp.cfg`



COCO数据集的聚类结果 (Anchor总数: 10647)

特征层尺寸	Anchor尺寸	Anchor总数
13*13	116*90, 156*198, 373 *326	13*13*3
26*26	30*61, 62*45, 59*119	26*26*3
52*52	10*13, 16*30, 33*23	52*52*3



```
[yolo]
mask = 0,1,2
anchors = 10,13, 16,30, 33,23, 30,61, 62,45, 59,119, 116,90, 156,198, 373,326
classes=80 目标类别数
num=9
jitter=.3
ignore_thresh = .7
truth_thresh = 1
random=1
```

网络模型

model.py



```
# [bs, anchor, grid, grid, xywh + obj + classes]
io = p.clone() # inference output    p: 网络卷积层输出的原始预测结果
io[..., :2] = torch.sigmoid(io[..., :2]) + self.grid # xy 计算在feature map上的xy坐标
io[..., 2:4] = torch.exp(io[..., 2:4]) * self.anchor_wh # wh 计算在feature map上的wh
io[..., :4] *= self.stride # 换算映射回原图尺度
torch.sigmoid_(io[..., 4:])
return io.view(bs, -1, self.no), p
```

预测的映射参数 $t = [t_x, t_y, t_w, t_h]$

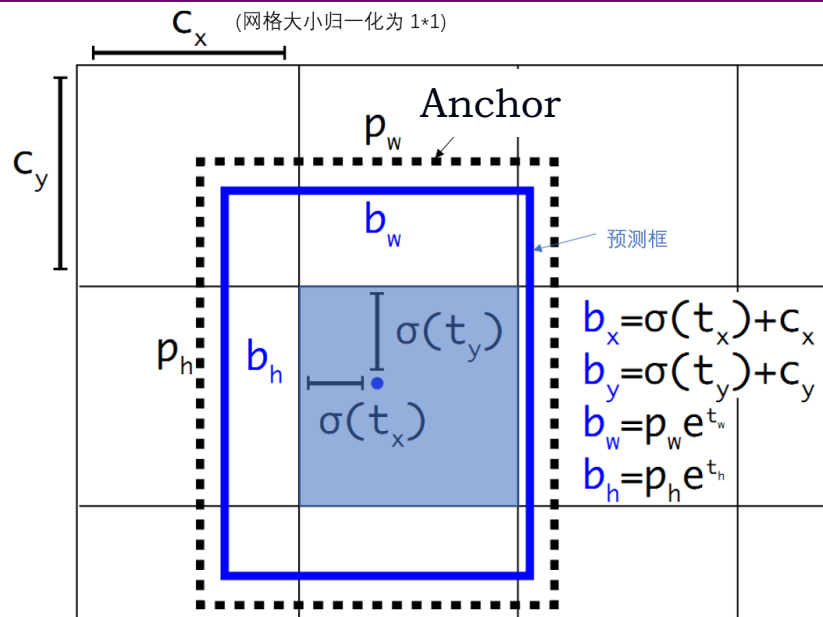
计算得到的
包围框位置
与尺寸

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

$$b_w = p_w e^{t_w}$$

$$b_h = p_h e^{t_h}$$



损失函数

utils.py

compute_loss(p, targets, model)



$$L = \lambda_{loc} L_{loc}(t, g) + \lambda_{cls} L_{cls}(O, C) + \lambda_{conf} L_{conf}(o, c)$$

③ 定位损失
(L_2 loss)

② 分类损失
(BCE loss)

① 置信度损失
(BCE loss)

Define criteria

BCEcls = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([h['cls_pw']]),

BCEobj = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([h['obj_pw']]),

① lobj += BCEobj(pi[..., 4], tobj) # obj loss

② lcls += BCEcls(ps[:, 5:], t) # BCE

pxy = ps[:, :2].sigmoid()

pwh = ps[:, 2:4].exp().clamp(max=1E3) * anchors[i]

pbox = torch.cat((pxy, pwh), 1) # predicted box

giou = bbox_iou(pbox.t(), tbox[i], x1y1x2y2=False, GIoU=True)

③ lbox += (1.0 - giou).mean() # giou loss

loss = lbox + lobj + lcls

return {"box_loss": lbox,

"obj_loss": lobj,

"class_loss": lcls}

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

$$b_w = p_w e^{t_w}$$

$$b_h = p_h e^{t_h}$$

损失函数

utils.py

compute_loss(p, targets, model)



```
# Define criteria
BCEcls = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([h['cls_pw']]),
BCEobj = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([h['obj_pw']]),

① lobj += BCEobj(pi[..., 4], tobj) # obj loss
② lcls += BCEcls(ps[:, 5:], t) # BCE
pxy = ps[:, :2].sigmoid()
pwh = ps[:, 2:4].exp().clamp(max=1E3) * anchors[i]
pbox = torch.cat((pxy, pwh), 1) # predicted box
giou = bbox_iou(pbox.t(), tbox[i], x1y1x2y2=False, GIoU=True)
③ lbox += (1.0 - giou).mean() # giou loss

# loss = lbox + lobj + lcls
return {"box_loss": lbox,
        "obj_loss": lobj,
        "class_loss": lcls}
```

```
# loss
loss_dict = compute_loss(pred, targets, model)
losses = sum(loss for loss in loss_dict.values())
```



反向传播

losses.backward()

主要内容

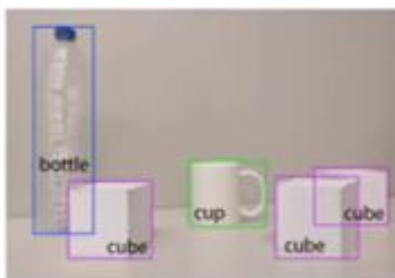
- 第六章 卷积神经网络
- 第七章 图像分类
- 第八章 目标检测与图像分割
 - 8.1 目标检测
 - 8.2 图像分割
 - 8.2.1 语义分割
 - 8.2.2 实例分割

8.2 图像分割

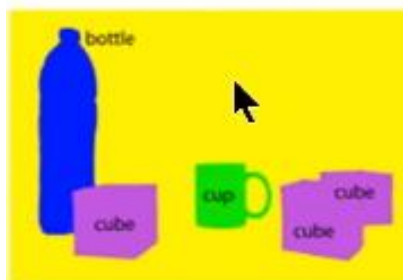
- 语义分割(semantic segmentation): 将图像中的所有像素进行分类（包括背景），但不区分具体目标
- 实例分割(instance segmentation): 对图像中完成像素级的分类，并区分每一个目标



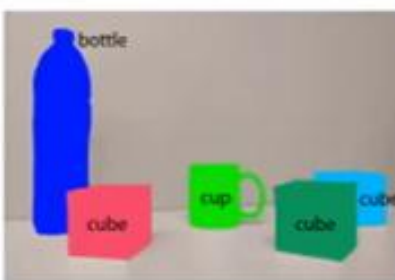
(a) Image classification



(b) Object localization



(c) Semantic segmentation

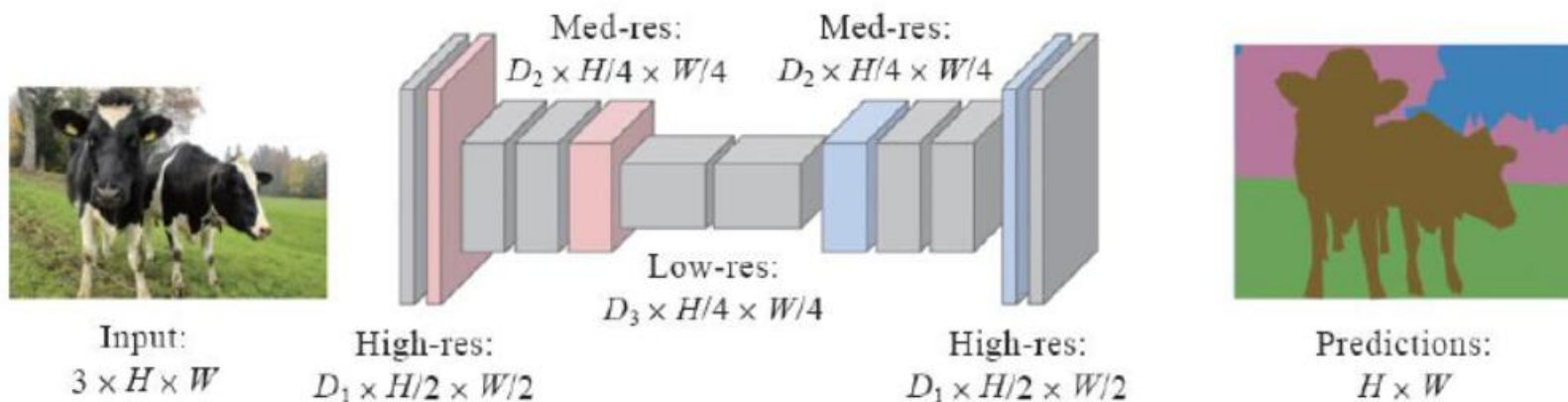


(d) Instance segmentation

8.2.1 语义分割

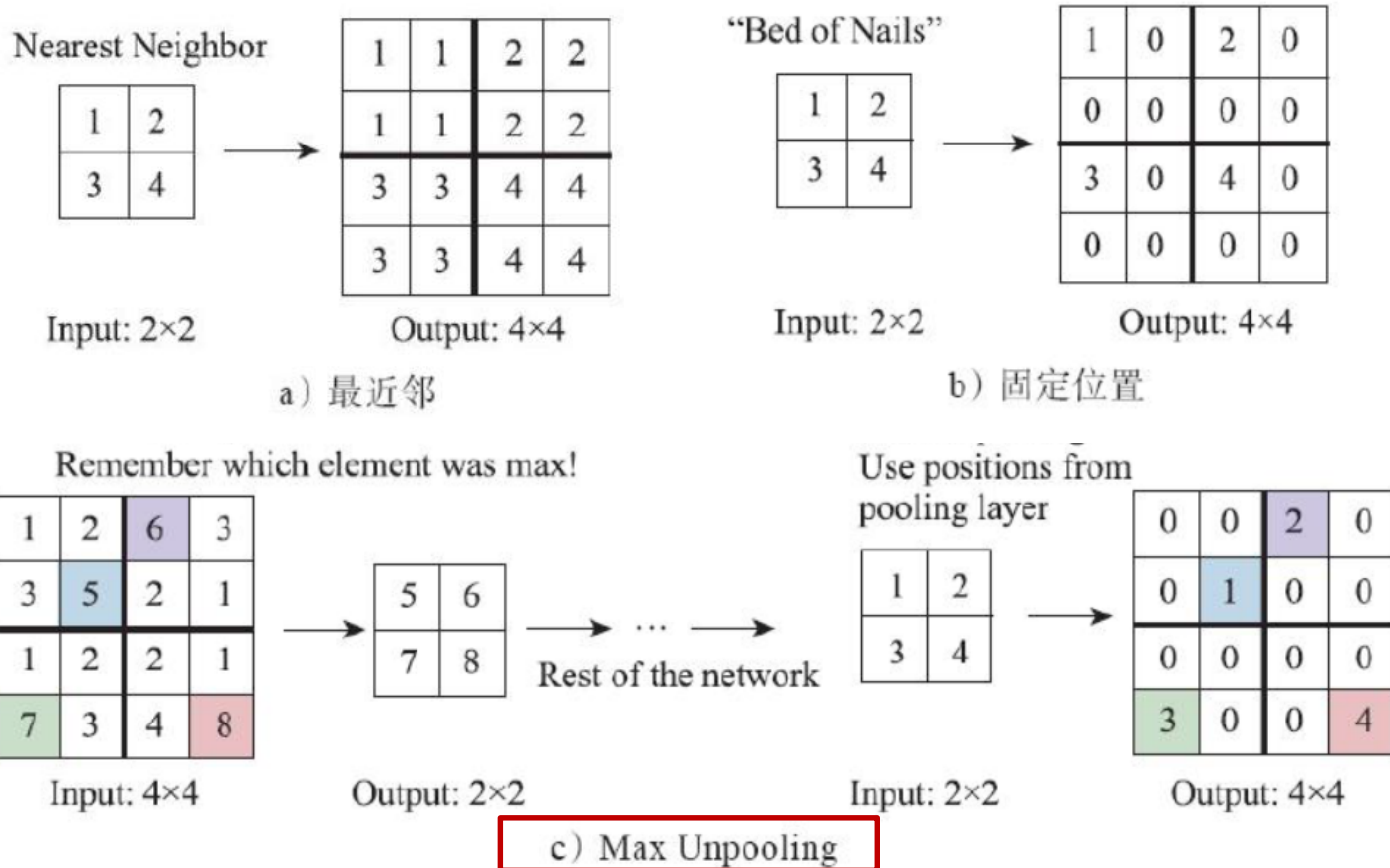
➤ FCN (Fully Convolutional Networks)

- 核心目标是对图像中的每一个像素进行分类，从而实现像素级的精细分割
- 将传统分类网络去掉全连接层，通过全卷积网络接受任意尺寸输入
- 通过反池化(Unpooling)和反卷积(Deconvolution)进行上采样恢复图像尺寸
- 利用跳跃连接融合浅层细节，实现端到端的像素级语义分割



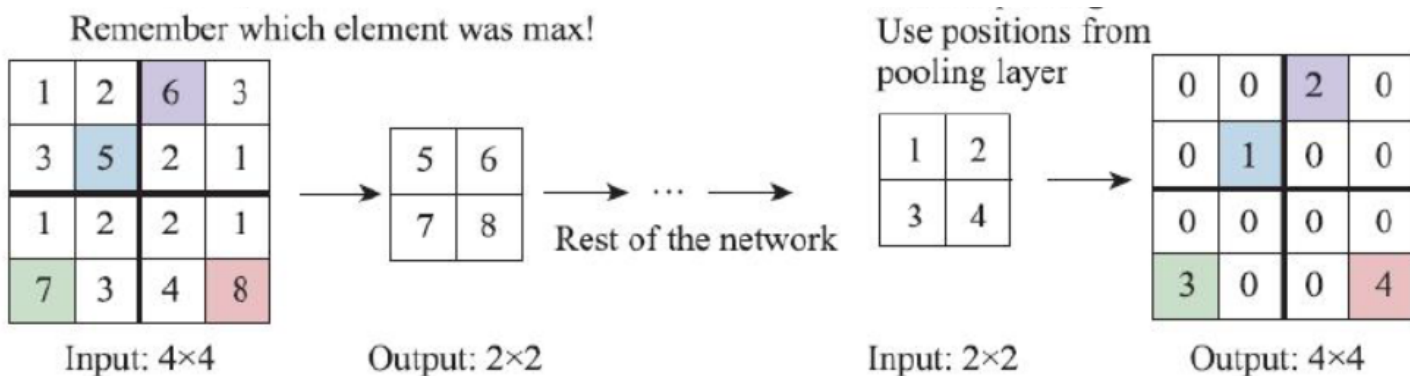
1) FCN

- 反池化(Unpooling): 把缩小了的特征图尺寸重新变大 (恢复尺寸), 下面是几种不同的方式:



1) FCN

- 反池化(Unpooling): 把缩小了的特征图尺寸重新变大 (恢复尺寸)
- 最常用方式: 最大反池化
 - 池化阶段 (记录): 在进行最大池化时, 除了输出最大值本身, 还会额外记录下这些最大值在原始特征图中的位置索引 (Indices)
 - 反池化阶段 (还原): 在进行反池化时, 利用之前记录的索引, 将池化后的数值精准地放回它原本所在的位置, 而其余的所有位置则自动填充为 0



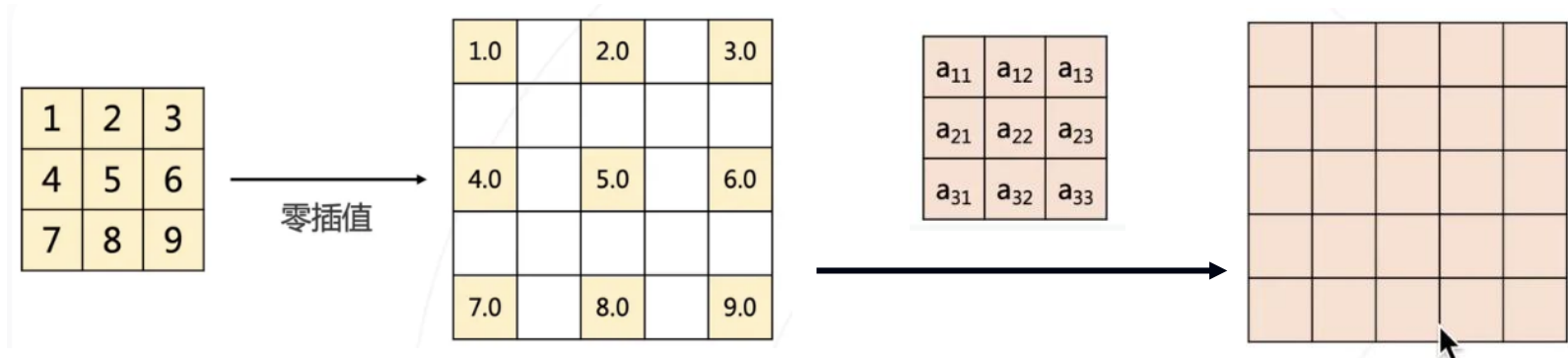
c) Max Unpooling

1) FCN

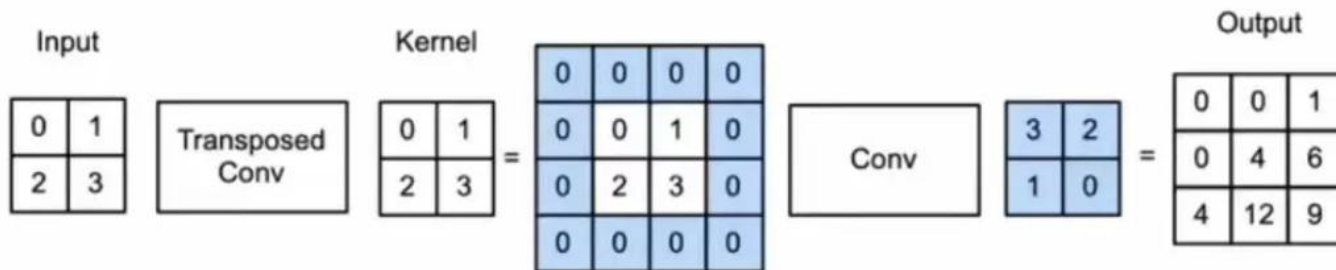
- 反卷积/转置卷积(Transposed Convolution)
 - 如果说普通卷积的作用是提取特征并缩小图像，那么转置卷积的作用就是学习如何填充细节并放大图像
 - **注意：**转置卷积并不是普通卷积的严格数学逆运算，因为它无法完全还原原始像素值，本质是一种可学习的上采样
- 具体步骤
 - 插零扩展：在输入特征图的像素点之间或周围插入零，人为扩大特征图的尺寸
 - 卷积运算：使用一个可学习的卷积核，在这个扩大后的特征图上进行正常的滑动卷积。
 - 重叠求和：卷积核在滑动过程中，输出的某些区域会发生重叠，这些重叠位置的数值会被相加，最终生成平滑且尺寸更大的输出特征图

1) FCN

➤ 反卷积/转置卷积(Transposed Convolution)



特征图像素点之补零



特征图像素点周围补零 (注意: 卷积核翻转)

PyTorch: 转置卷积

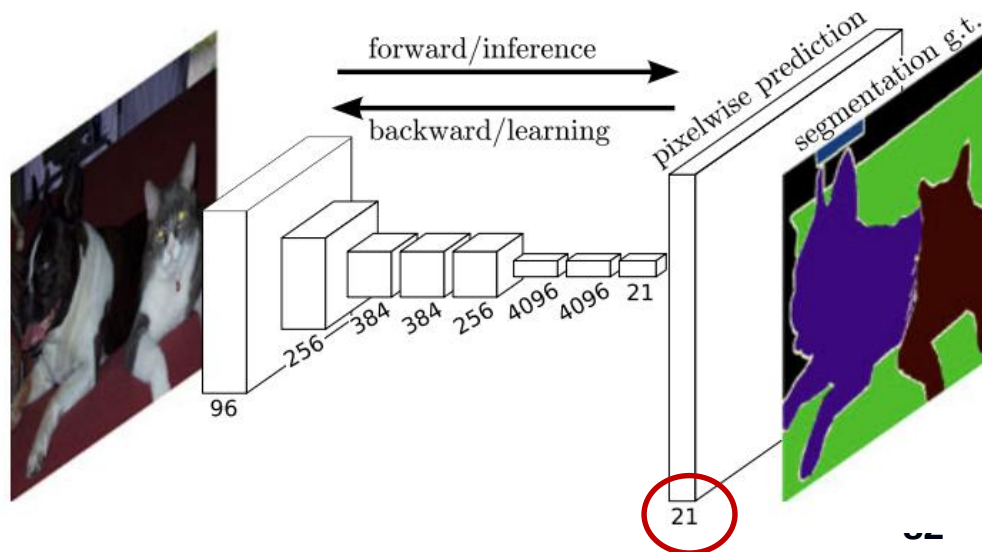
ConvTranspose2d <https://docs.pytorch.org/docs/2.11/generated/torch.nn.ConvTranspose2d.html#torch.nn.ConvTranspose2d>

```
class torch.nn.ConvTranspose2d(in_channels, out_channels, kernel_size, stride=1,  
padding=0, output_padding=0, groups=1, bias=True, dilation=1, padding_mode='zeros',  
device=None, dtype=None) # \[source\]
```

- in_channels: 输入特征图的通道数
- out_channels: 输出特征图的通道数
- kernel_size: 卷积核的大小
- stride: 步长, 控制上采样的倍率, $\text{stride} > 1$ 时会显著增加输出特征图的尺寸
- padding: 在输入特征图四周隐式添加的零填充。注意: 在转置卷积中, padding 越大, 输出尺寸反而越小 (可以理解为在还原标准卷积时, 要把当初加的 padding 减掉)
- output_padding: 输出填充。在输出特征图的一侧额外添加的零填充, 用于微调输出尺寸, 解决由于步长和卷积核大小导致的尺寸计算歧义
- dilation: 空洞卷积的扩张率, 用于扩大感受野
- bias: 是否为该层添加可学习的偏置项, 默认为True

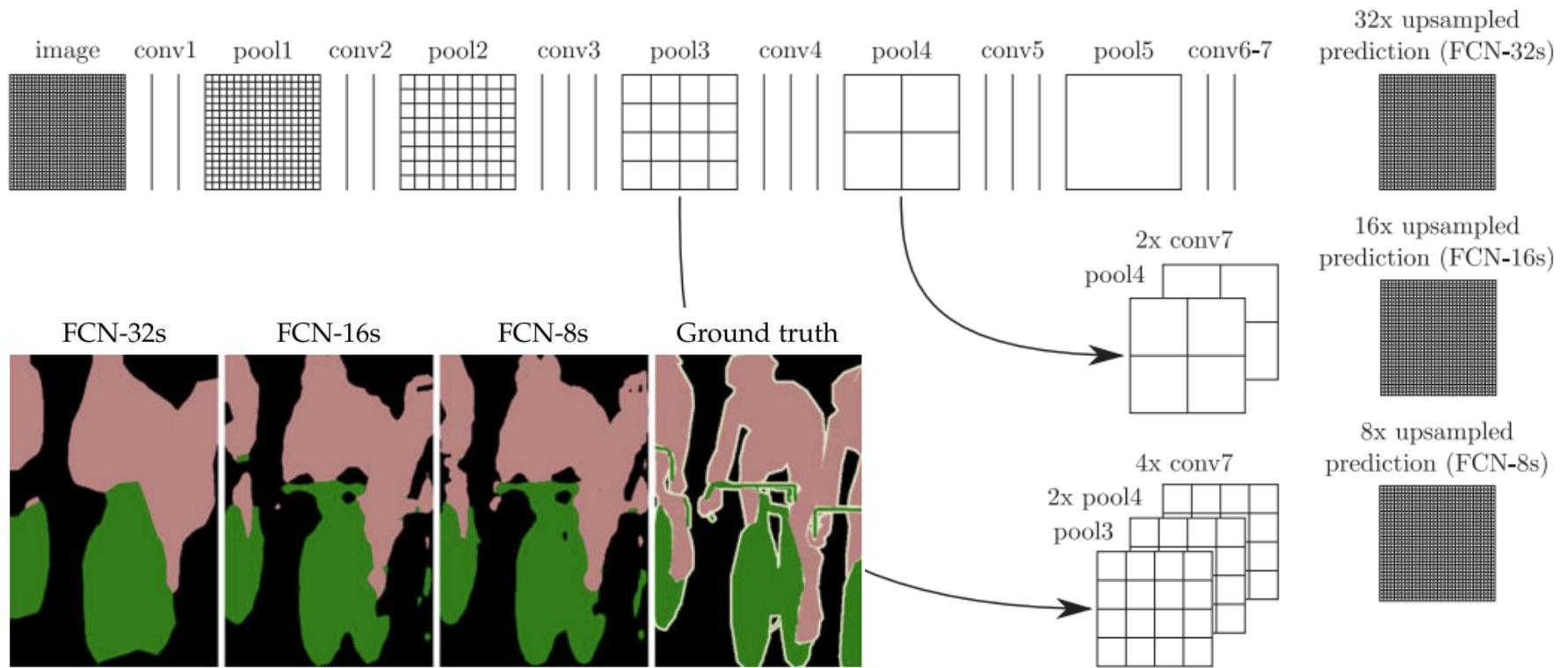
1) FCN

- FCN把语义分割任务看作图像分类的逻辑，在每一个像素点上重复执行分类任务
- 损失函数：逐像素(Pixel-wise)的多分类交叉熵损失
 - 训练时：FCN会输出一个和原图尺寸相同的特征图，其中每个像素位置都包含了一个属于各个类别的概率向量
 - 计算 Loss：把每个像素的预测概率分布与该像素的真实标签进行对比，计算交叉熵损失
 - 最终Loss：将整张图像中所有像素的损失值取平均或求和，得到最终Loss



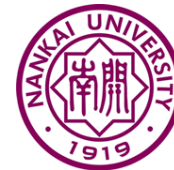
1) FCN

- 不论是Uppooling方式还是Deconvolution的方式，网络结构的关键层都是以卷积的方式进行操作，不涉及类似全连接这种操作，因此我们通常称这种网络为**全卷积网络**（Full Connected Network, FCN）



2) Unet

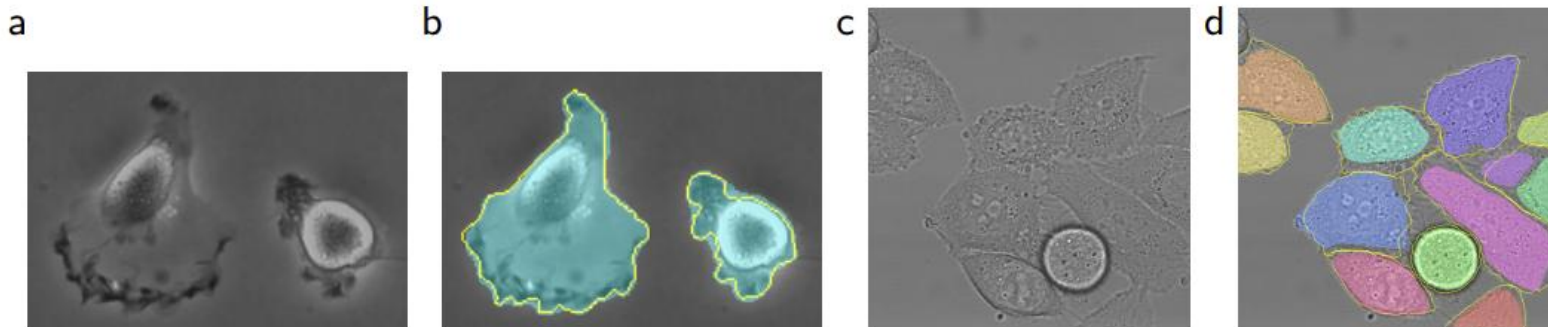
<https://arxiv.org/abs/1505.04597>



U-Net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

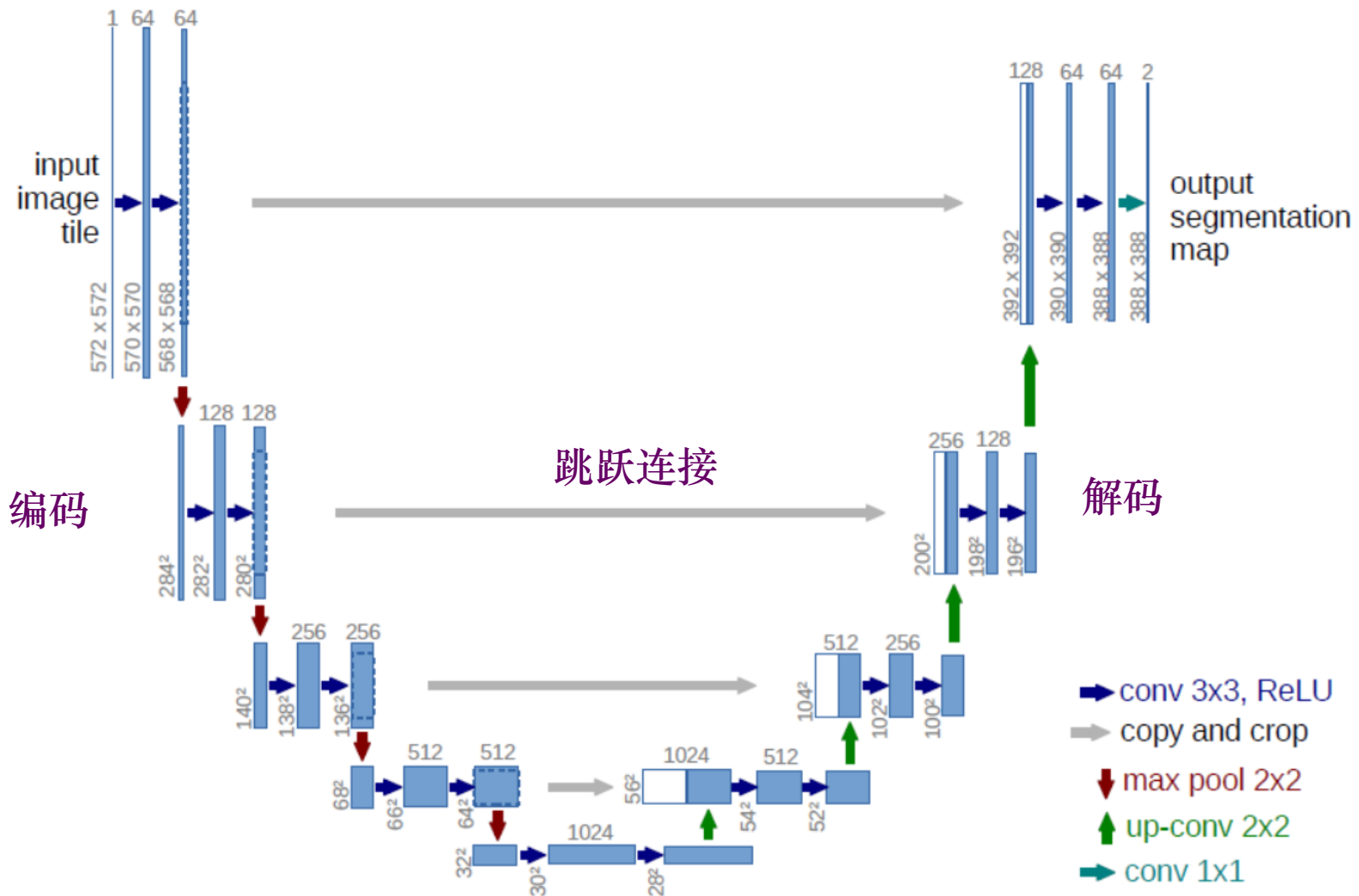
Computer Science Department and BIOSS Centre for Biological Signalling Studies,
University of Freiburg, Germany



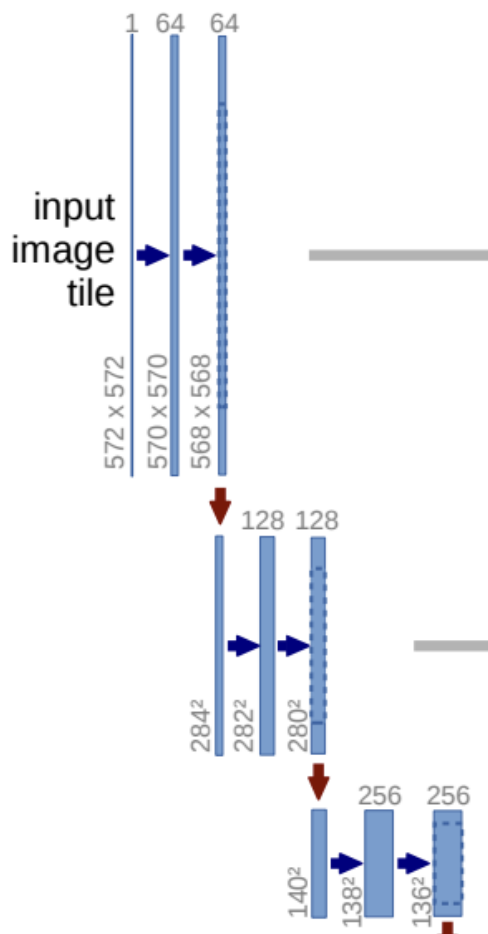
注意：随机染色，Unet是语义分割方法

Fig. 4. Result on the ISBI cell tracking challenge. (a) part of an input image of the “PhC-U373” data set. (b) Segmentation result (cyan mask) with manual ground truth (yellow border) (c) input image of the “DIC-HeLa” data set. (d) Segmentation result (random colored masks) with manual ground truth (yellow border).

2) Unet: 网络结构



2) Unet: 网络结构



1. 编码器(左侧收缩路径): 特征提取

- 作用: 像传统的卷积神经网络一样, 逐步缩小图像的尺寸, 同时增加特征的通道数。目的是提取图像的高层语义信息
- 结构: “卷积块 + 最大池化层” 组成
 - 卷积块: 通常包含两次 3×3 的卷积 (Convolution+ReLU 激活函数+批量归一化 (BatchNorm))
 - 下采样: 使用 2×2 的最大池化 (Max Pooling), 每次操作会让特征图的长宽减半, 通道数翻倍

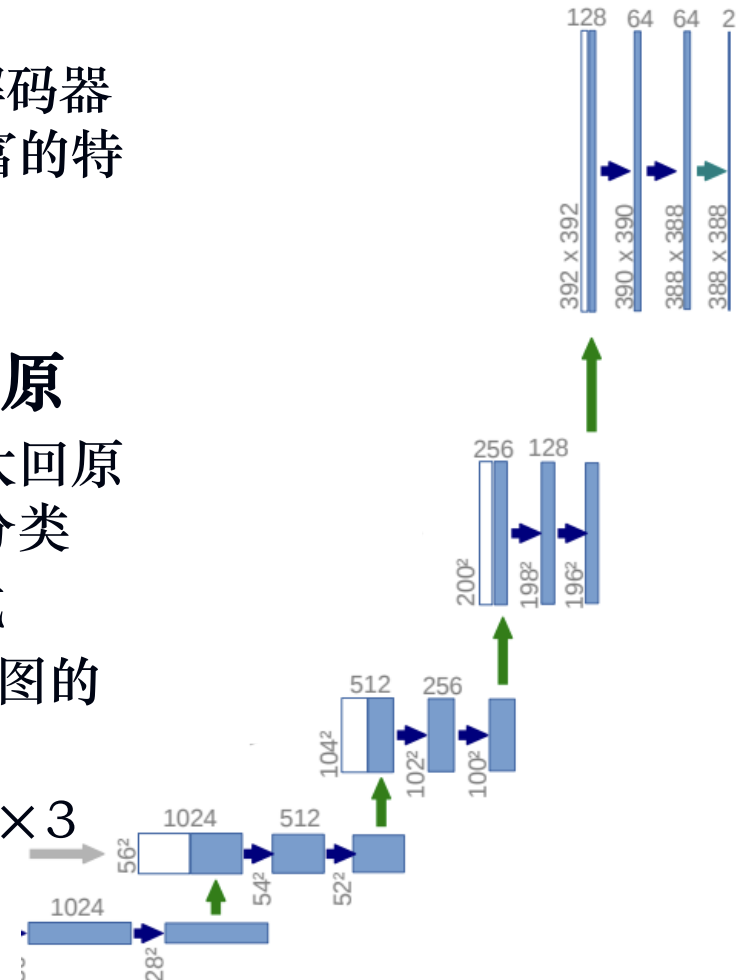
2) Unet: 网络结构

2. 瓶颈层（底部）：语义核心

- 作用：位于U型最底部，连接编码器和解码器，保留了分辨率最低、但语义信息最丰富的特征图
- 结构：两个 3×3 的卷积层，无池化

3. 解码器(右侧扩张路径)：像素级还原

- 作用：将瓶颈层提取的抽象特征逐步放大回原始图像的尺寸，从而实现对每个像素的分类
- 结构：“上采样 + 拼接 + 卷积块”组成
 - 上采样：通常使用转置卷积，将特征图的长宽翻倍，通道数减半
 - 卷积块：上采样后，同样经过两次 3×3 卷积+ReLU+BatchNorm细化特征



2) Unet: 网络结构

4. 跳跃连接 (Skip Connection)

- 这是 U-Net 区别于普通“编码器-解码器”结构最关键的地方
- 核心痛点：解码器在通过上采样恢复图像尺寸时，虽然有了高层的语义信息，但之前被池化层丢弃的边缘、纹理等低层空间细节很难完全找回，这会导致分割出来的物体边缘模糊
- 解决方案：U-Net 在解码器的每一步上采样后，会将特征图与编码器中对应层级的特征图进行“通道拼接 (Concatenate)”
- 效果：解码器不仅拥有了高层的“语义理解”还直接拿到了编码器传过来的低层“高清细节”。这种多尺度特征的融合，使得 U-Net 的分割边界极其精准



2) Unet: 损失函数

➤ 细胞图像分割的独特问题

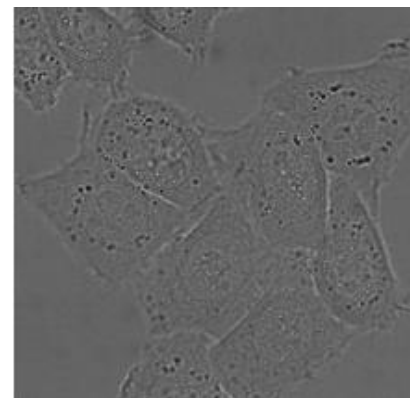
- 类别极度不平衡：背景像素远远多于细胞（前景）像素
- 相邻细胞边界难以区分：同一类别的细胞经常紧密接触，网络很难精准分割出它们之间的细微缝隙

➤ 损失函数

➤ 加权的像素级分类交叉熵损失

$$loss = \sum_{x \in \Omega} w(x) \log(p_{l(x)}(x))$$

- Ω ：图像中的所有像素位置
- $p_{l(x)}(x)$ ：像素 x 被预测为真实类别 $l(x)$ 的概率，由softmax函数计算
- $w(x)$ ：像素点权重



2) Unet: 损失函数

像素点权重

$$w(x) = w_c(x) + w_0 \cdot \exp\left(-\frac{(d_1(x) + d_2(x))^2}{2\sigma^2}\right)$$

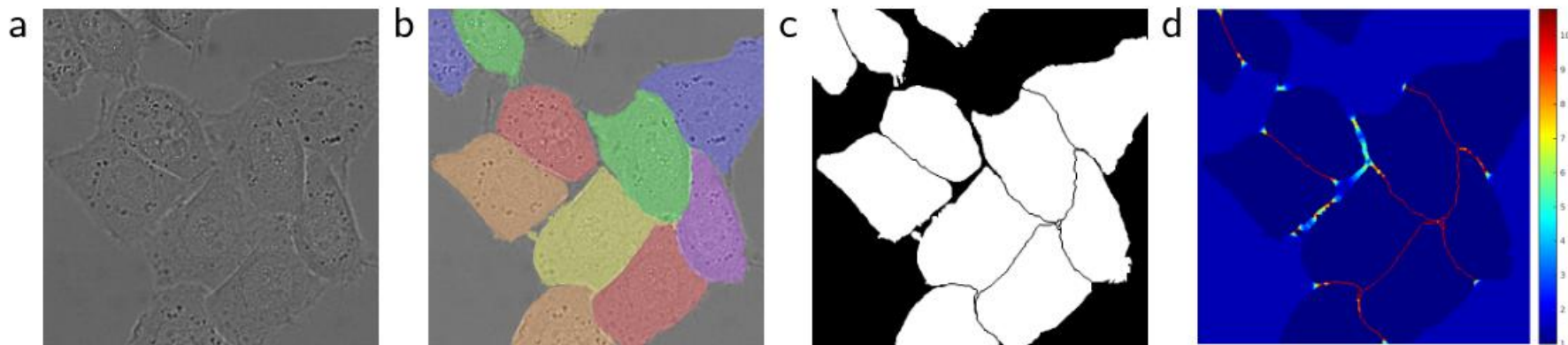
- 第一项 $w_c(x)$: 平衡类别比例的权重, 用来解决类别不平衡问题。对于出现频率低的类别, 比如稀少的细胞前景, 分配更大的权重, 强迫网络在训练时不能只考虑大片背景, 必须重视前景像素
- 第二项: 强化细胞边界的权重, 用于解决相邻细胞难以分割的问题
 - $d_1(x)$: 像素 x 到最近细胞边界的距离
 - $d_2(x)$: 像素 x 到第二近细胞边界的距离
 - 当某个像素正好位于两个细胞的交界处时, d_1 和 d_2 都会很小, 此时指数项的值会非常大, 导致该像素的权重急剧升高
 - 在论文实验中, 作者设定 $w_0 = 10$, $\sigma \approx 5$, 这意味着训练时如果分错了细胞边缘的像素, 会受到极其严厉的惩罚, 从而逼迫网络学会精细地分离紧密接触的细胞

2) Unet: 损失函数

像素点权重

$$w(x) = w_c(x) + w_0 \cdot \exp\left(-\frac{(d_1(x) + d_2(x))^2}{2\sigma^2}\right)$$

- 第一项 $w_c(x)$: 平衡类别比例的权重, 用来解决类别不平衡问题。对于出现频率低的类别, 比如稀少的细胞前景, 分配更大的权重, 强迫网络在训练时不能只考虑大片背景, 必须重视前景像素
- 第二项: 强化细胞边界的权重, 用于解决相邻细胞难以分割的问题



Hela细胞原图

Ground Truth

分割掩膜

像素级损失权重

2) U-Net



2019:

nature | **methods**

BRIEF COMMUNICATION

<https://doi.org/10.1038/s41592-018-0261-2>

Corrected: Author Correction

U-Net: deep learning for cell counting, detection, and morphometry

Thorsten Falk^{1,2,3,20}, Dominic Mai^{1,2,4,16,20}, Robert Bensch^{1,2,17,20}, Özgün Çiçek¹, Ahmed Abdulkadir^{1,5}, Yassine Marrakchi^{1,2,3}, Anton Böhm¹, Jan Deubner^{6,7}, Zoe Jäckel^{6,7}, Katharina Seiwald⁶, Alexander Dovzhenko^{8,18}, Olaf Tietz^{8,18}, Cristina Dal Bosco⁸, Sean Walsh^{8,18}, Deniz Saltukoglu^{2,9,10,11}, Tuan Leng Tay^{7,12,13}, Marco Prinz^{2,3,12}, Klaus Palme^{2,8}, Matias Simons^{2,9,10,14}, Ilka Diester^{6,7,15}, Thomas Brox^{1,2,3,7} and Olaf Ronneberger^{1,2,19*}

¹Department of Computer Science, Albert-Ludwigs-University, Freiburg, Germany. ²BIOSS Centre for Biological Signalling Studies, Freiburg, Germany.

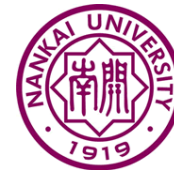
³CIBSS Centre for Integrative Biological Signalling Studies, Albert-Ludwigs-University, Freiburg, Germany. ⁴Life Imaging Center, Center for Biological Systems Analysis, Albert-Ludwigs-University, Freiburg, Germany. ⁵University Hospital of Old Age Psychiatry and Psychotherapy, University of Bern, Bern, Switzerland. ⁶Optophysiology Lab, Institute of Biology III, Albert-Ludwigs-University, Freiburg, Germany. ⁷BrainLinks-BrainTools, Albert-Ludwigs-University, Freiburg, Germany. ⁸Institute of Biology II, Albert-Ludwigs-University, Freiburg, Germany. ⁹Center for Biological Systems Analysis (ZBSA), Albert-Ludwigs-University, Freiburg, Germany. ¹⁰Renal Division, University Medical Centre, Freiburg, Germany. ¹¹Spemann Graduate School of Biology and Medicine (SGBM), Albert-Ludwigs-University, Freiburg, Germany. ¹²Institute of Neuropathology, University Medical Centre, Freiburg, Germany.

¹³Institute of Biology I, Albert-Ludwigs-University, Freiburg, Germany. ¹⁴Paris Descartes University-Sorbonne Paris Cité, Imagine Institute, Paris, France.

¹⁵Bernstein Center Freiburg, Albert-Ludwigs-University, Freiburg, Germany. ¹⁶Present address: SICK AG, Waldkirch, Germany. ¹⁷Present address: ANavS GmbH, München, Germany. ¹⁸Present address: ScreenSYS GmbH, Freiburg, Germany. ¹⁹Present address: DeepMind, London, UK. ²⁰These authors contributed equally: Thorsten Falk, Dominic Mai, Robert Bensch. *e-mail: ronneber@informatik.uni-freiburg.de

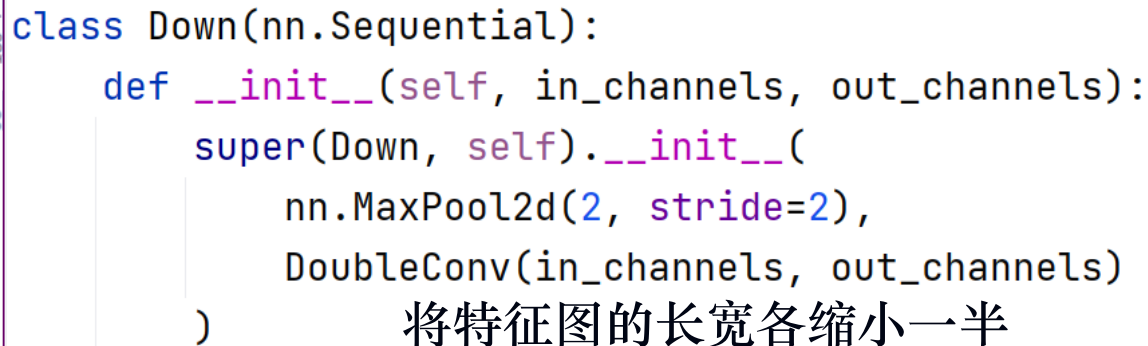
Unet: 双卷积模块

unet.py



```
class DoubleConv(nn.Sequential):  
    def __init__(self, in_channels, out_channels, mid_channels=None):  
        if mid_channels is None:  
            mid_channels = out_channels  
        super(DoubleConv, self).__init__(  
            nn.Conv2d(in_channels, mid_channels, kernel_size=3, padding=1, bias=False),  
            nn.BatchNorm2d(mid_channels),  
            nn.ReLU(inplace=True),  
            nn.Conv2d(mid_channels, out_channels, kernel_size=3, padding=1, bias=False),  
            nn.BatchNorm2d(out_channels),  
            nn.ReLU(inplace=True)  
        )
```

和原文不同，特征图尺寸不变



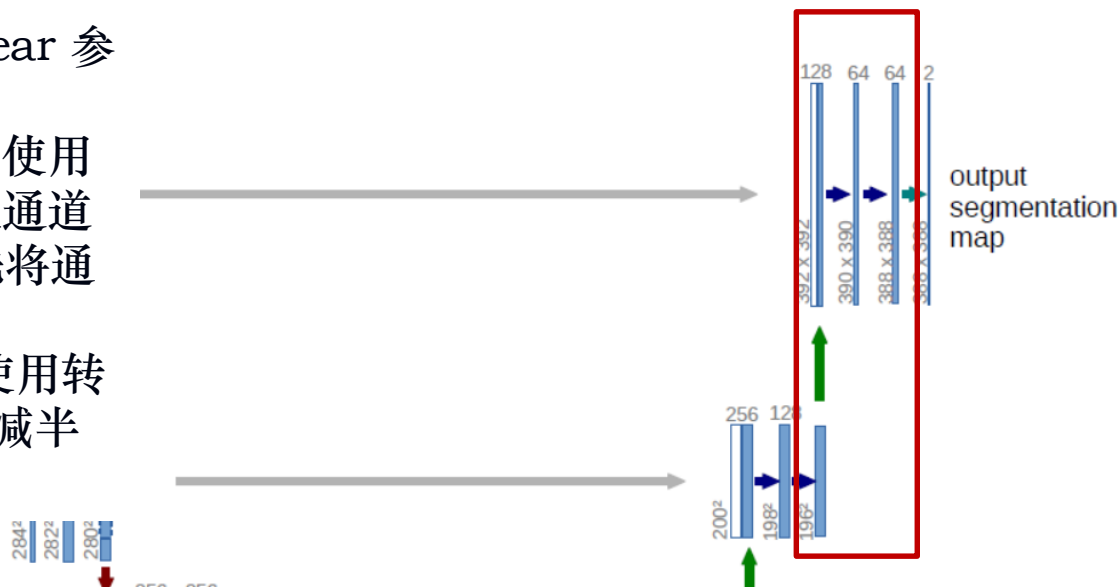
Unet: 上采样模块

unet.py



提供了两种上采样方式，由 bilinear 参数控制：

- 双线性插值 (bilinear=True): 使用 nn.Upsample, 插值不会改变通道数, 后续的 DoubleConv 会先将通道数减半, 再进行特征融合
- 转置卷积 (bilinear=False): 使用转置卷积同时完成“放大尺寸 + 减半通道数”的操作



```
class Up(nn.Module):
    def __init__(self, in_channels, out_channels, bilinear=True):
        super(Up, self).__init__()
        if bilinear:
            self.up = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
            self.conv = DoubleConv(in_channels, out_channels, in_channels // 2)
        else:
            self.up = nn.ConvTranspose2d(in_channels, in_channels // 2, kernel_size=2, stride=2)
            self.conv = DoubleConv(in_channels, out_channels)
```

Unet: 上采样模块

unet.py



forward函数接收两个输入:

- x1: 来自解码器上一层特征
- x2: 来自编码器对应层的跳跃连接特征



```
def forward(self, x1: torch.Tensor, x2: torch.Tensor) -> torch.Tensor:
```

```
    x1 = self.up(x1)  上采样
```

```
    # [N, C, H, W]
```

```
    diff_y = x2.size()[2] - x1.size()[2]
```

尺寸对齐

```
    diff_x = x2.size()[3] - x1.size()[3]
```

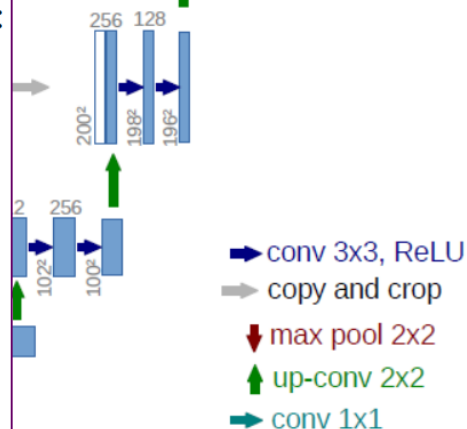
```
    # padding_left, padding_right, padding_top, padding_bottom
```

```
    x1 = F.pad(x1, [diff_x // 2, diff_x - diff_x // 2,  
                  diff_y // 2, diff_y - diff_y // 2])
```

```
    x = torch.cat([x2, x1], dim=1)  特征拼接
```

```
    x = self.conv(x)  DoubleConv特征提取
```

```
    return x
```



Unet

U-Net 的特征通道数会随着下采样逐层翻倍 (64 -> 128 -> 256 -> 512 -> 1024)

```
class UNet(nn.Module):
    def __init__(self,
                  in_channels: int = 1,
                  num_classes: int = 2,
                  bilinear: bool = True,
                  base_c: int = 64):
        super(UNet, self).__init__()
        self.in_channels = in_channels
        self.num_classes = num_classes
        self.bilinear = bilinear

        self.in_conv = DoubleConv(in_channels, base_c)
        self.down1 = Down(base_c, base_c * 2)
        self.down2 = Down(base_c * 2, base_c * 4)
        self.down3 = Down(base_c * 4, base_c * 8)
        factor = 2 if bilinear else 1
        self.down4 = Down(base_c * 8, base_c * 16 // factor)
        self.up1 = Up(base_c * 16, base_c * 8 // factor, bilinear)
        self.up2 = Up(base_c * 8, base_c * 4 // factor, bilinear)
        self.up3 = Up(base_c * 4, base_c * 2 // factor, bilinear)
        self.up4 = Up(base_c * 2, base_c, bilinear)
        self.out_conv = OutConv(base_c, num_classes)
```

unet.py


```
def forward(self, x: torch.Tensor) -> Dict[str, torch.Tensor]:
    x1 = self.in_conv(x)
    x2 = self.down1(x1)
    x3 = self.down2(x2)
    x4 = self.down3(x3)
    x5 = self.down4(x4)
    x = self.up1(x5, x4)
    x = self.up2(x, x3)
    x = self.up3(x, x2)
    x = self.up4(x, x1)
    logits = self.out_conv(x)
```

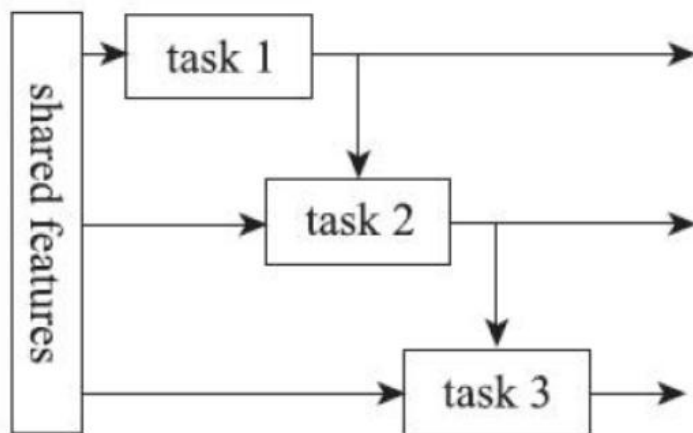
- 输入x下采样，依次经过 in_conv -> down1 -> down2 -> down3 -> down4，在此过程中，每一层的输出都被保存下来，作为后续解码器的跳跃连接
- 从最深层x5开始上采样，依次调用up1到up4，每次调用都会传入当前层的特征和编码器对应层的特征
- 输出：最后经过out_conv得到最终的预测结果 logits



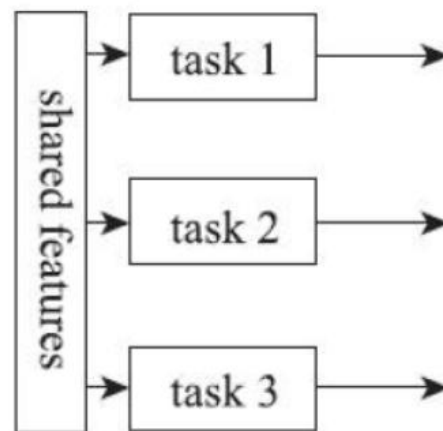
```
class OutConv(nn.Sequential):
    def __init__(self, in_channels, num_classes):
        super(OutConv, self).__init__(
            nn.Conv2d(in_channels, num_classes, kernel_size=1)
        )
```

8.2.2 实例分割

- ▶ 实例分割需要识别出“每一个物体”，又需要得到像素级别分割的结果，是一个多任务学习的方法
- ▶ 多任务学习通常具有两种方式
 - ▶ 堆叠式：将不同的任务通过某种方式连接起来进行训练
 - ▶ 扁平式：每个任务单独训练



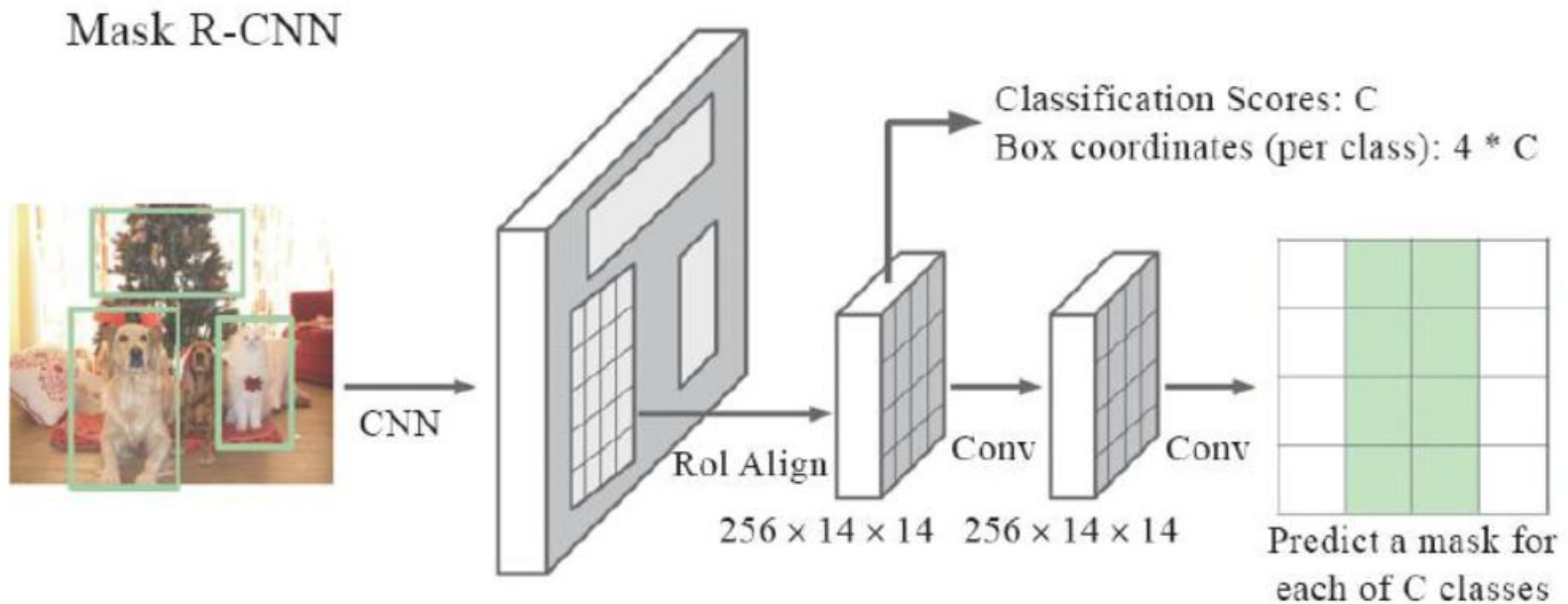
a) 层叠式



b) 扁平式

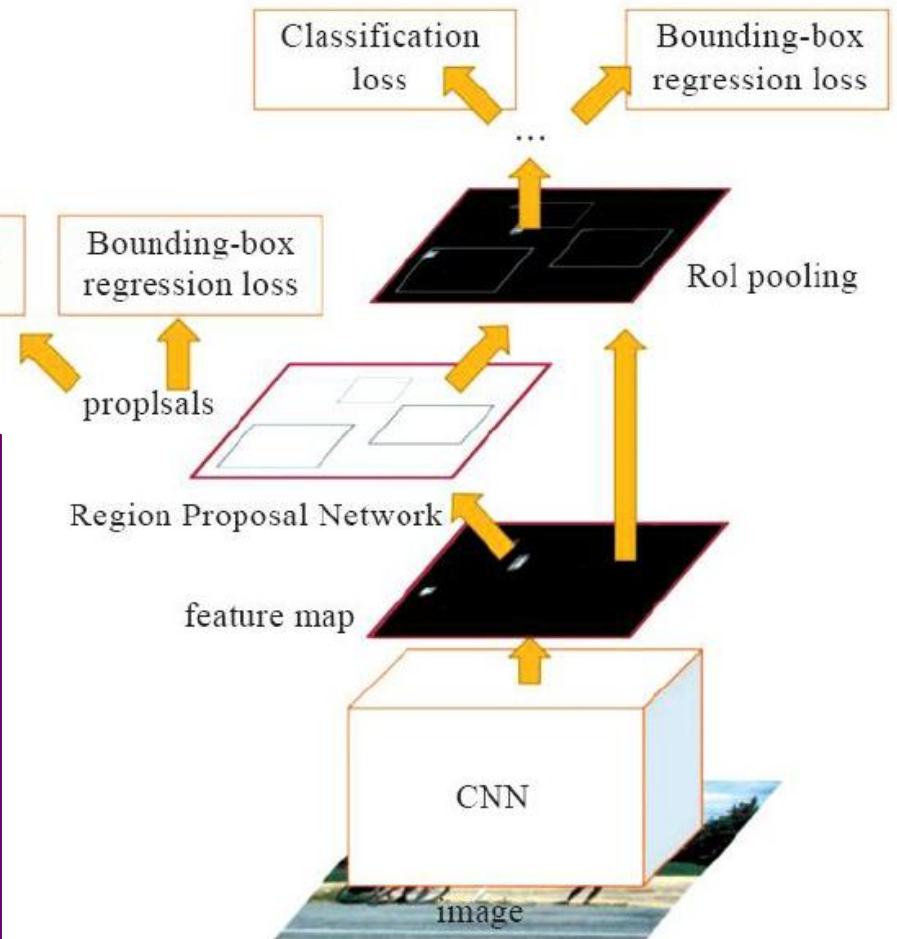
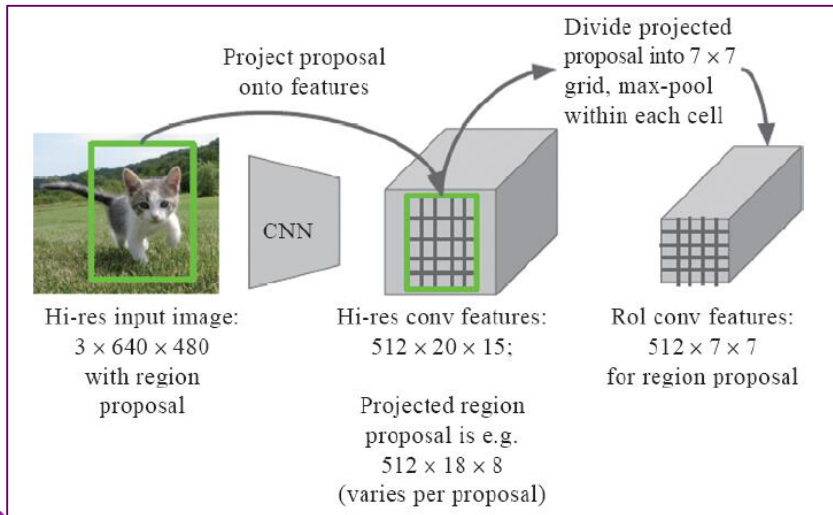
Mask R-CNN

- 由于目标检测的结果既有紧贴着目标的矩形框，又有矩形框里目标的类别，那么是否可以基于目标检测的结果，实现像素级分割呢？
- Mask-RCNN:在目前业界公认的效果较好的检测模型，在 Faster RCNN基础上添加第三个分割分支



回顾：Faster R-CNN

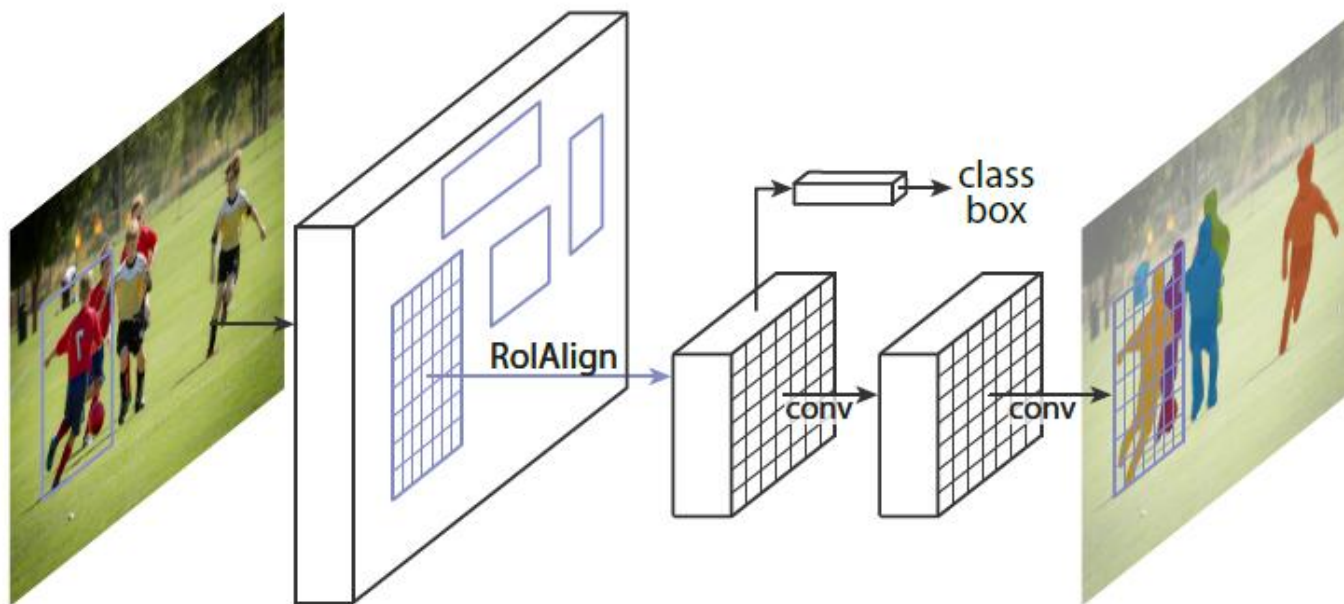
- 借助Anchor获得候选框
- 网络结构
 - 特征提取
 - RPN
 - 目标识别/ROI pooling



Mask R-CNN

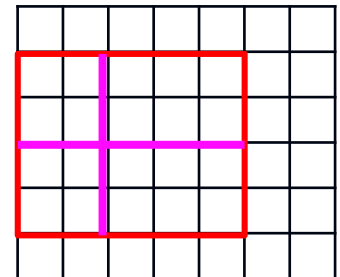
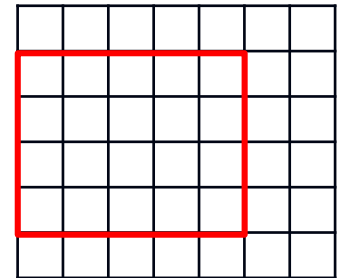
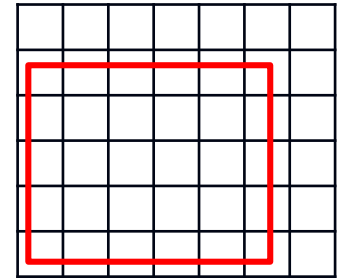
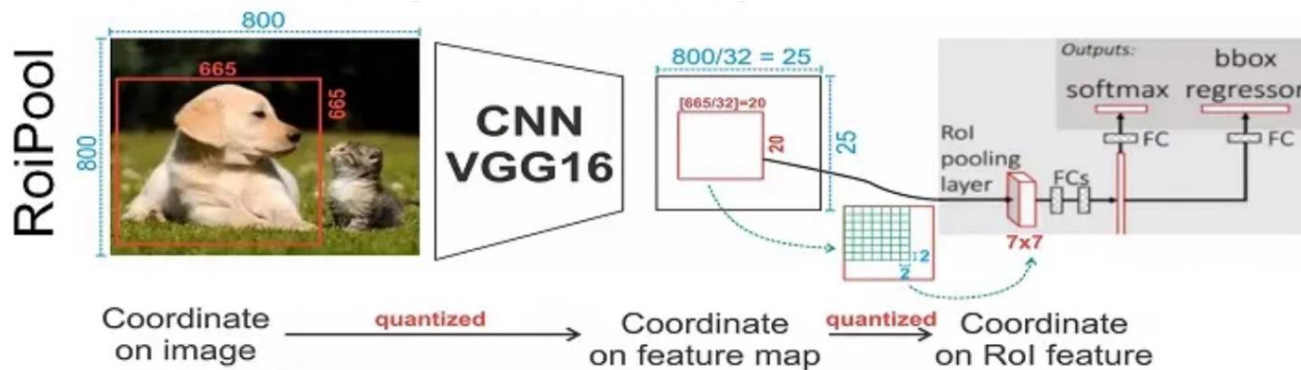
实例分割网络：Mask R-CNN

- ROI Align: 解决ROI Pooling操作中两次整数化操作造成的区域不对准 (mis-alignment)的问题
- Mask分支: 基于目标检测的结果, 完成图像分割



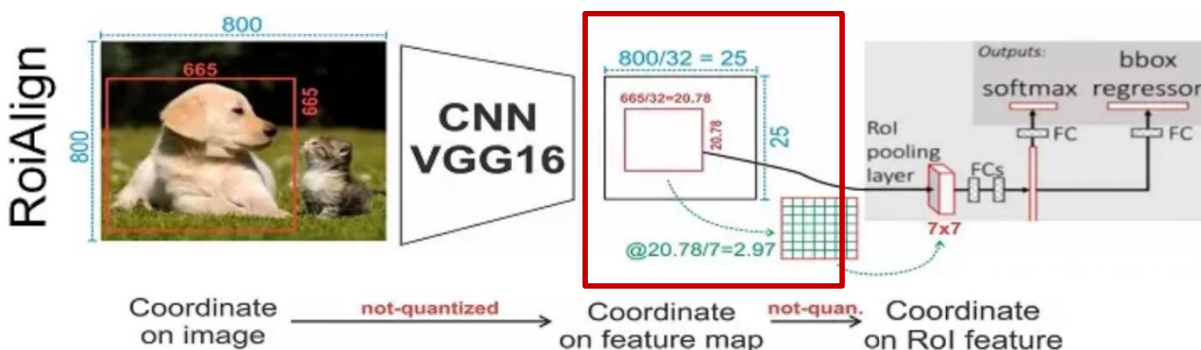
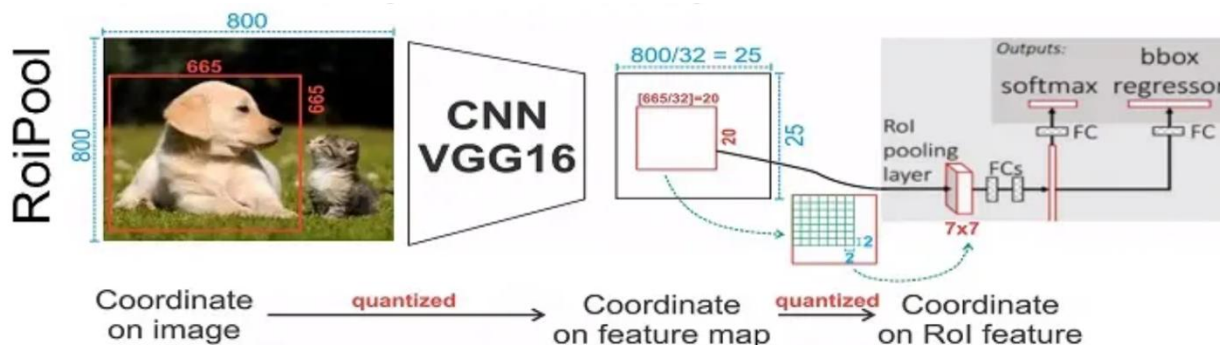
Mask R-CNN: ROI Align

- ROI pooling中的两次整数化操作
 - 候选框位置(x,y,w,h)映射到特征图上，其位置通常是小数，简单起见通常去掉小数部分
 - 在ROI pooling中，整数化后的候选包围框被分成7*7个网格，对每个网格进一步进行整数化
- 经过两次整数化，此时的候选框已经和最开始回归出来的位置有一定的偏差，这个偏差会影响检测或者分割的准确度，区域不对准 (mis-alignment)



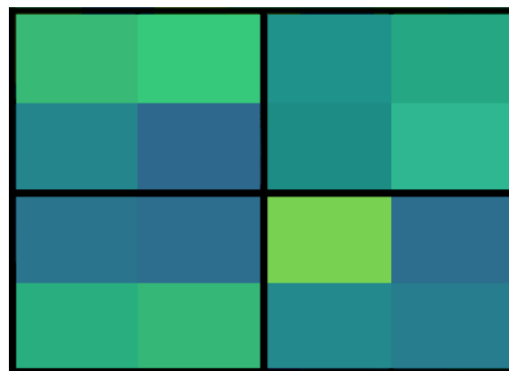
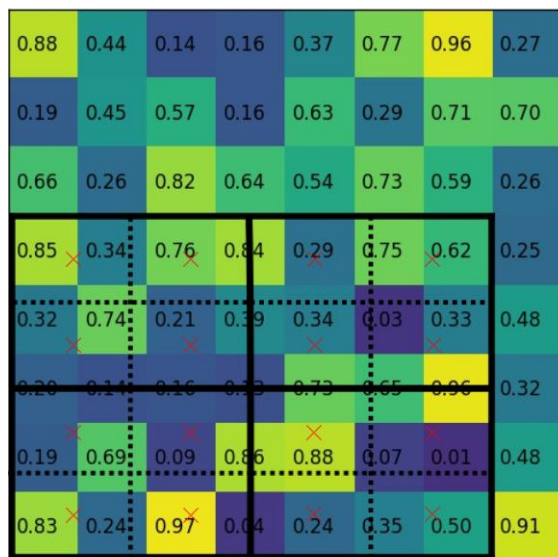
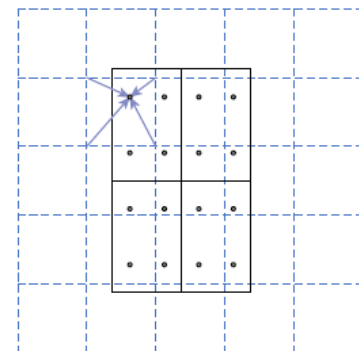
Mask R-CNN: ROI Align

- 遍历每一个候选框，保持浮点数边界不进行整数化
- 将候选框分割成7*7个网格，每个网格的边界也不整数化



Mask R-CNN: ROI Align

- 在7*7的每个网格中考虑四个采样点
- 利用双线性插值从特征图上附近的网格点计算采样点的值
- 对4个采样点进行最大池化，得到ROI网格取值



Bilinear interpolated values



Max pooling output

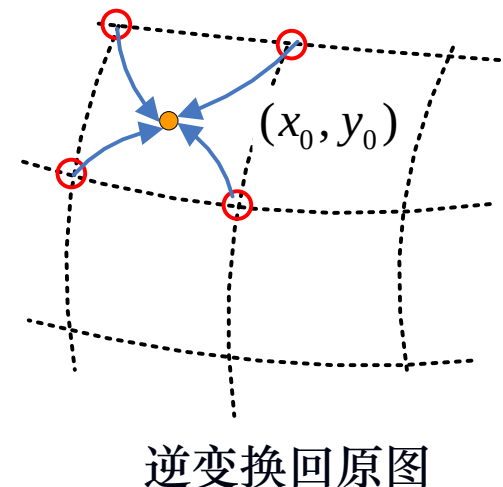
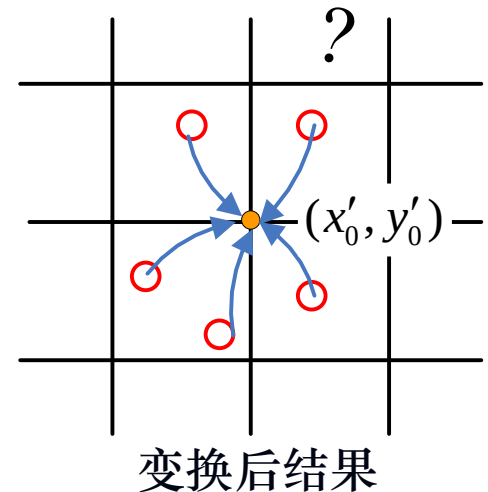
复习：亮度插值

➤ 用对偶方法完成亮度插值

- 确定对应于输出图像离散栅格点在输入图像中原来点的亮度
- 假设计算在输出图像**离散栅格**上像素 (x'_0, y'_0) 的亮度值
- 输入图像中对应点 (x_0, y_0) 的坐标可通过像素坐标逆变换得到

$$(x_0, y_0) = \mathbf{T}^{-1}(x'_0, y'_0)$$

- 对输入图像进行亮度插值，获得 $(x_0, y_0) / (x'_0, y'_0)$ 的亮度值



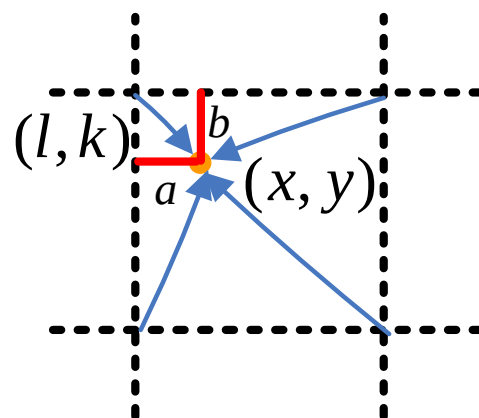
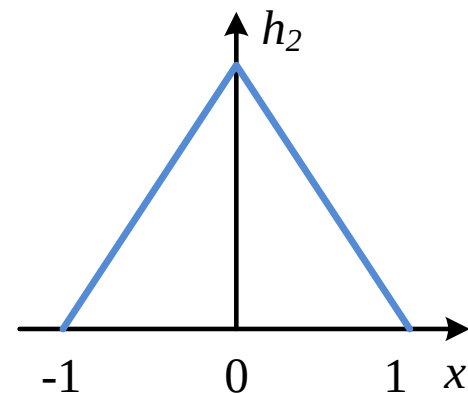
复习：亮度插值

➤ 线性插值

- 考虑像素 (x, y) 的四个相邻点，假设亮度在这个邻域内是线性的

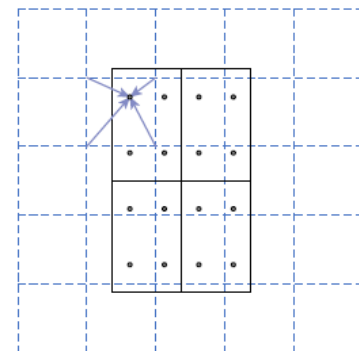
$$\begin{aligned} f_2(x, y) &= (1-a)(1-b)f(l, k) & l &= \text{floor}(x) \\ &+ a(1-b)f(l+1, k) & k &= \text{floor}(y) \\ &+ b(1-a)f(l, k+1) & a &= x-l, b = y-k \\ &+ abf(l+1, k+1) \end{aligned}$$

- 线性插值可能引起较小的分辨率降低和模糊，但减轻了最近邻插值中出现的阶梯状直边界



Mask R-CNN: ROI Align

- 在7*7的每个网格中考虑四个采样点
- 利用双线性插值从特征图上附近的网格点计算采样点的值
- 对4个采样点进行最大池化，得到ROI网格取值



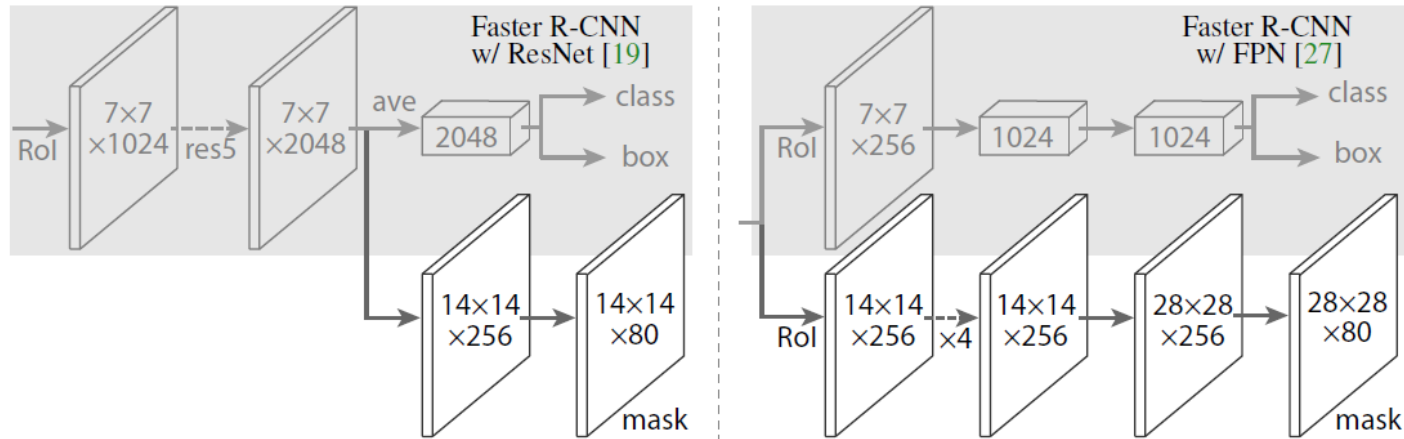
ROI Align的效果

	AP	AP ₅₀	AP ₇₅	AP ^{bb}	AP ₅₀ ^{bb}	AP ₇₅ ^{bb}
<i>RoIPool</i>	23.6	46.5	21.6	28.2	52.7	26.9
<i>RoIAlign</i>	30.9	51.8	32.1	34.0	55.3	36.4
	+7.3	+ 5.3	+10.5	+5.8	+2.6	+9.5

Average Precision (AP):

AP % AP at IoU=.50:.05:.95 (primary challenge metric)
 AP^{IoU=.50} % AP at IoU=.50 (PASCAL VOC metric)
 AP^{IoU=.75} % AP at IoU=.75 (strict metric)

Mask R-CNN: Mask分支

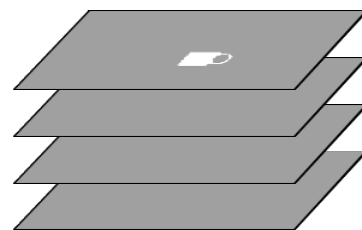
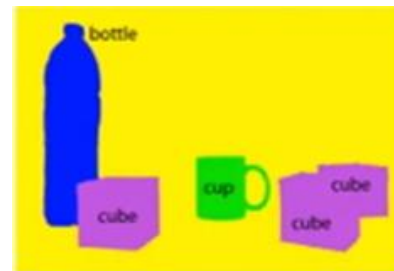


多任务损失函数

$$L = L_{cls} + L_{box} + L_{mask}$$

- 对于每个m*m大小的ROI，Mask分支输出一个K*m*m维的矩阵，表示K个不同的分类
- 对于每一个像素，使用sigmoid函数计算交叉熵， L_{mask} 用平均二值交叉熵损失(average binary cross-entropy loss)定义

Mask R-CNN: Mask分支



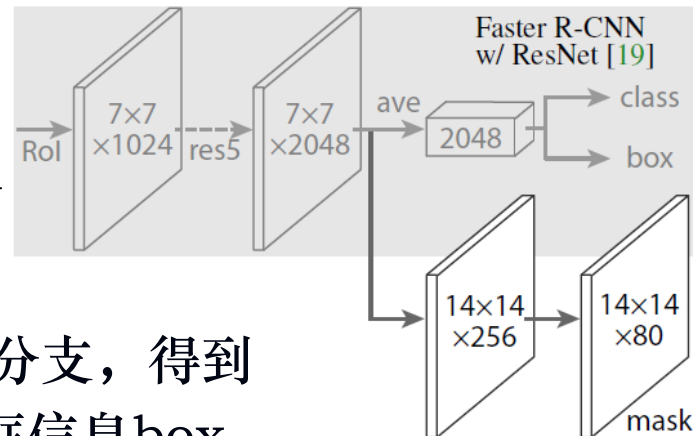
- Mask R-CNN (sigmoid, BCE) : 将预测类别与Mask进行了解耦，由于分类分支已经通过Softmax 确定了这个ROI属于哪个类别，Mask分支的任务专门负责生成该类别的掩码
- 对ROI Align的输出，Mask R-CNN针对每个类别都单独预测一个Mask，最后根据分类分支的预测结果选择对应类别的Mask，这样做避免了Mask分支内部不同类别之间的像素竞争，让分割任务更加专注和精准
- 传统FCN (softmax交叉熵) : 对于图像的每个像素，针对每个类别预测一个分数，通过softmax得到每个类别的概率，将该像素分配给概率最高的类别

	AP	AP ₅₀	AP ₇₅
<i>softmax</i>	24.8	44.1	25.1
<i>sigmoid</i>	30.3	51.2	31.5
	+5.5	+7.1	+6.4

Mask R-CNN

Mask R-CNN预测阶段的具体步骤

1. 将图像输入网络，通过Faster R-CNN分支，得到最终预测的目标类别信息class和包围框信息box
2. 将目标包围框信息box提供给Mask分支，预测得到该目标对所有类别的mask信息
3. 根据目标类别信息class，将对应该类别的mask信息提取出来(shape为 14×14 ，数值在0到1之间)
4. 利用双线性插值将mask缩放到预测目标包围框大小
5. 通过设定阈值将mask转换为二值图，得到前景与背景

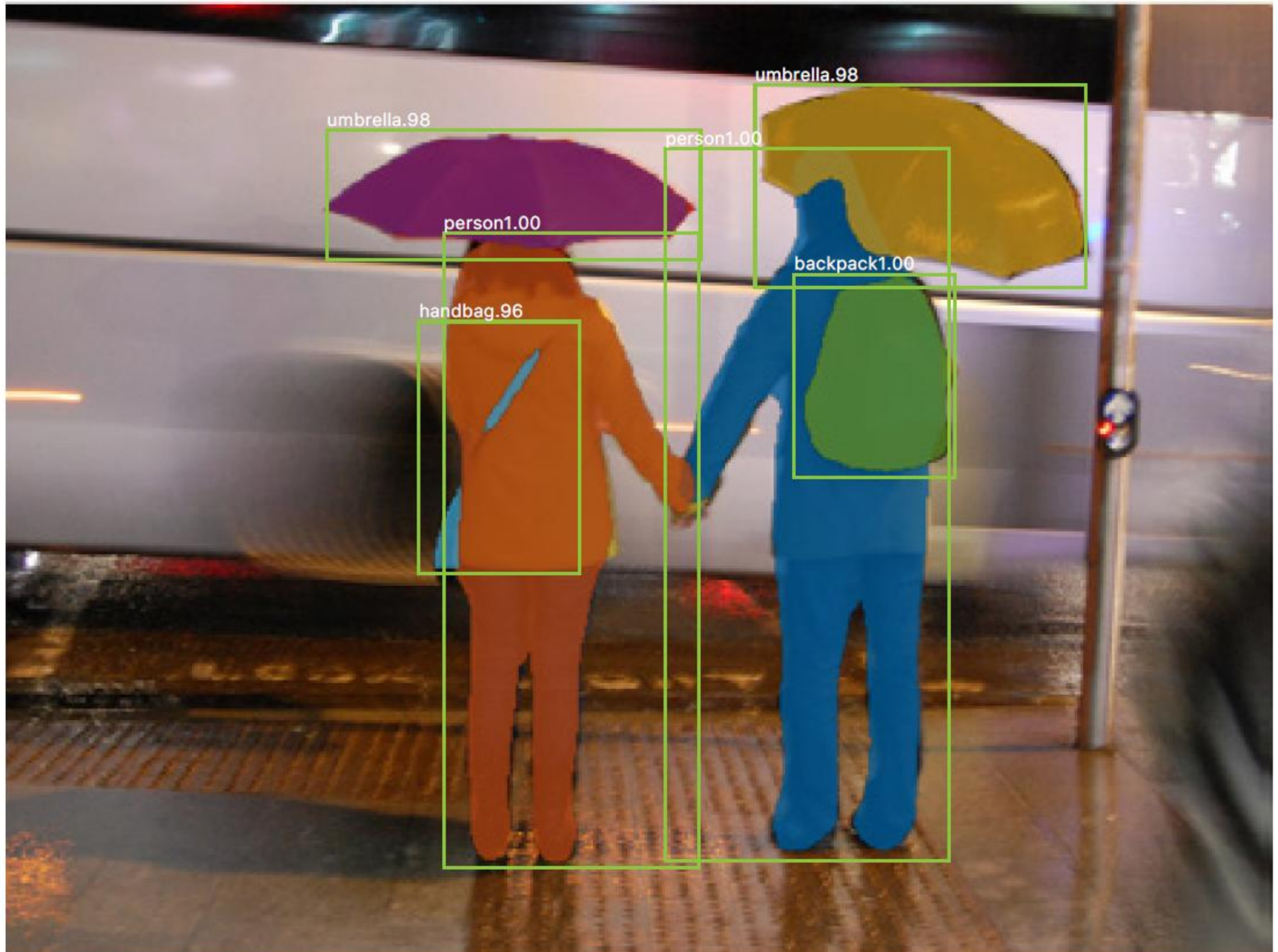


Mask R-CNN



➤ 结果展示





Mask R-CNN

➤ 结果展示：关键点检测



➤ 第八章 目标检测与图像分割

➤ 8.1 目标检测

- Two-stage目标检测方法：RCNN→Fast RCNN
→Faster RCNN
- ROI pooling/Anchor/包围框映射/多任务损失
- One-stage目标检测方法：包围框映射/包围框置信度

➤ 8.2 图像分割

- 8.2.1 语义分割，FCN、Unet：反卷积、像素级分类
交叉熵损失
- 8.2.2 实例分割，Mask RCNN：ROI Align，多任务
损失函数

The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University